

The Thiele Machine

A Complete Account from First Principles

Structural Entitlement, Accountable Computation,
and No Free Insight

Devon Thiele

September 2026 – v3.2.1

Before the book: the short version, and how to check it

I didn't invent a machine. I found one.

That is how I see this work. I think there is structure here that our usual picture of computation leaves out. The conviction brought me to the project; the definitions and proofs are how I put it to the test.

I built this because the answer at the end of a computation did not tell me everything I wanted to know. What structure did the machine use? What did it establish? What did it pay to establish it? I wanted those questions in the mathematics, with definitions somebody else could check.

Certification is one point I could pin down. It gave me a law I could state, a machine I could build, and proofs I could put in front of someone who did not share my intuition. It is one witness in a much larger argument about structure and observation. I am not done looking.

The shadow is real for the projections I proved it for. Drop the relevant distinction and no decoder gets it back from what remains. A different encoding can keep it. That is part of the question: which distinctions deserve to survive our account of a computation?

I think there is something foundational here. I will be straight about the difference between that conviction and a theorem. The results below have statements, premises, and proof terms. The larger claim still has work to do. I built the machine so that work could be done in public, with something firmer than my confidence to argue over.

Abstract

The Thiele framework studies structural state, observation interfaces, and accountable events in computation. Certification is one explicit instance: A2 requires positive cost on a false-to-true transition of a designated predicate. The abstract cost theorem ranges over arbitrary state and instruction types satisfying that law.

The formal results include pricing adequacy relative to certification-flip count, ledger uniqueness for a fixed schedule, separation under specified projections, trace-fold initiality, and specialized algebraic certificate checks. The executable VM and conditional hardware bridges realize parts of this framework. Semantic truth requires the relevant checker soundness; payment alone establishes an accounting event.

The physical interpretation and the choice of universally necessary priced events remain open. The full-ISA F1 physical premise pair is incompatible with zero-cost jumps, as `F1_physical_premises_incompatible` proves. The

observation results do not forbid information-preserving encodings, and the trace-fold universal property does not establish VM-state initiality.

Contents

1	Let me be straight with you	6
2	Things I had to learn first	17
2.1	Bits, bytes, and what a CPU sees	17
2.2	Math notation I will use without explaining it twice	18
2.3	Coq, in just enough detail	20
2.4	Category theory: the part I needed beyond Section 11	21
2.5	Bell-test vocabulary you will meet in Section 16	22
2.6	Hardware and synthesis vocabulary	23
2.7	Other terms I use without rebuilding	24
2.8	The axioms (A1 through A6)	26
3	The structural axis: the projection classical models drop	28
3.1	Substrate-independence: the abstract setup	29
3.2	What is canonical, and under which conditions	31
3.3	The canonical contrast: the Turing machine	31
3.4	The observation boundary	32
3.5	Structural cost and other resource measures	32
3.6	The subsumption theorems, in full	32
3.7	What the projection cannot reverse: three non-existence theorems . . .	36
3.8	What the projection cannot verify: the verifier corollary	40
4	What structure is	42
4.1	Partitions	42
4.2	Axioms carried by modules	43
4.3	Structural gain	45
5	Structural entitlement	45
5.1	The precise vocabulary	46
5.2	What narrowing buys	57
5.3	The boundary	59
6	What insight is, and why it cannot be free	60
6.1	The three tiers of observation	60
6.2	Why insight cannot be free	62
7	The μ ledger	63
7.1	What mu is	63
7.2	Why mu is the unique canonical cost measure	66
7.3	The LASSERT cost formula	67
8	No Free Insight: the first cost law	68
8.1	The abstract versions	68

8.2	The specific versions	71
8.3	What this means in practice	72
9	The formal state record	72
10	The instruction set: all 51 opcodes	75
10.1	Group 1: Partition operations (4 opcodes)	75
10.2	Group 2: Logic and certification (3 opcodes)	76
10.3	Group 3: Compute and data transfer (14 opcodes)	78
10.4	Group 4: Memory and control flow (8 opcodes)	79
10.5	Group 5: Revelation, I/O, heap, and tensor (7 opcodes)	80
10.6	Group 6: Witness and certification (8 opcodes)	81
10.7	Group 7: Categorical morphism operations (7 opcodes)	83
10.8	Why 51 opcodes?	84
11	What a categorical morphism is: the category theory, from scratch	86
11.1	The idea of a category	86
11.2	Why modules as objects	88
11.3	Why this is not just “extra state”	88
12	Categorical separation: what the classical shadow drops	89
13	The knowledge receipt: accounting and structural witnesses	92
14	The substrate, instantiated three times	94
14.1	Layer 1: The Coq kernel	94
14.2	Layer 2: The OCaml extracted runner	96
14.3	Layer 3: The Kami RTL hardware	97
15	Five minimal models inspired by other disciplines	100
15.1	Proof-of-stake finality	100
15.2	Gas metering	101
15.3	Trusted-execution attestation	101
15.4	Certificate transparency	101
15.5	Proof-carrying verification	102
15.6	The same table, five ways	102
16	Binary Game Statistics: where the μ-Ledger Meets the NPA Boundary	102
16.1	The Bell test setup, from scratch	104
16.2	The CHSH inequality: where it comes from	105
16.3	The quantum bound: why $2\sqrt{2}$?	106
16.4	What the cost algebra proves	107
16.5	The bit-commitment requirement	111
16.6	Algebraic characterization of the μ -ledger gate condition	111
16.6.1	The matrix, written out	112

16.6.2	Where the three conditions come from	113
16.7	Lifting the bridge to level 1+AB: the four-slice Schur cascade	122
16.7.1	The 9×9 moment matrix and its five γ parameters	122
16.7.2	Why four slices	123
16.7.3	The compositional Schur cascade (Slice D)	124
16.7.4	What this delivers and what it does not	125
16.8	The full correlator set: the elliptope completion	125
17	Irreversibility accounting and the μ-hierarchy	128
17.1	The Landauer association is motivation only	128
17.2	The μ -hierarchy	129
18	Discrete Geometry over Partition Graphs	131
18.1	Discrete triangulation of the partition graph	131
18.2	The restricted angle-defect identity	132
19	The substrate and its halting-shaped wall	133
19.1	The Substrate typeclass	134
19.2	The shortcut-admittance predicate	135
19.3	The substrate-level limitative theorem	136
19.4	The proof, in prose	139
19.5	The two-axis picture this completes	140
19.6	What this closes	140
19.7	The wall the classical configuration cannot see	141
20	What Coq is, and why I used it	143
20.1	Reproducing the source and proof contracts	144
20.2	The Inquisitor	145
21	Zero Admitted: what it actually means	146
22	How to break this	147
23	The full claim ledger	152
23.1	Main-result contract crosswalk	156
24	Scope of the claim: what is established, what is not	160
25	The pointer-observable conjecture	161
26	Conclusion	163

1 Let me be straight with you

One thing before the story, because people get it backwards. I used language models to write the code and the proofs in this project, and I trust language models about as far as I can throw them, which is exactly why I ran every line through a proof assistant that does not care who or what wrote it. Coq rejects a wrong proof the same whether a tenured professor or a model hands it over, and it accepted these, with none of my own axioms anywhere in them, re-checkable using the reproduction procedure in Section 20.1. Being self-taught and using modern tools did not make this looser. It is the reason I made a machine check all of it, me included. The long version of that is a few pages down, under how the work actually got made; I am putting the short version up here so it lands before anything else does.

What I am is curious, past the point of good sense. I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go. I am not a programmer and I was never formally taught any of this. I was not trying to build a formally verified CPU. I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it. Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own. I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had was not a strategy, it was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in

JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. (The basic furniture of category theory; the first three I build from scratch in Section 11, functors and monoidal products in Section 2.) It came together in pieces and never in chapters, the way a picture resolves out of static instead of getting drawn in left to right. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

Then within weeks I found out the engine I had built to draw pictures was general enough to run a physics simulation on the exact same primitives, without my rewriting any of it. I wrote Newton's laws as a program in it, half expecting nothing, and it ran. I want to be careful about what that was and what it was not, because this is exactly the kind of thing that gets misread. It was not me deriving physics, and I am not claiming the machine explains gravity. It was the compositional structure being general enough to express a physics simulation the same way it had expressed a renderer, and that generality is what told me the structure was the point and the pixels never were. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. I built a small custom language for describing scenes (the name for that is a domain-specific language, a DSL), then a program that read the language and caught my mistakes in it (a linter), then a way to type commands at the whole thing from a terminal (a command-line interface, a CLI). The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how instead of my spelling out every step, I told Julie I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Z3 could settle the small questions and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to, not because it is broken but because it was never built with an eye that points that way. I opened a new repo that same night and called the thing in it the Thiele Machine. It was a placeholder, the name you give a folder so you can save it. It stuck,

and it later turned out to carry more than I meant by it, but that is its own story and I get to it further down.

What survived the whole trip from there to here is not the code. Not one line of it. By the time the Coq proofs finally landed, every line of that first renderer had been thrown out and rewritten at least twice, probably more, and good riddance to all of it. What survived was a single architectural commitment I made early and never once let anyone talk me out of: category theory is the state itself, the actual thing the machine runs on, not a metaphor laid over some realer state underneath. The `Category` structure that came together in that first week and first ran morphisms correctly at two in the morning on January 22 is, in different words and different syntax, the same object `vm_graph` is in the 51-opcode realisation. Finding the different words took fifteen months. I did not write that code, not then and not now. I directed the models and refused everything they handed me until the morphisms composed the way the shape said they had to. The morphisms in that early code were objects that remembered where they came from; the morphisms in the realisation are objects that remember where they came from and that you can walk up to and challenge with `MORPH_ASSERT`. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and language and proof, was me slowly learning to see what I had already built.

I chased that one question for fifteen months. The renderer worked in January. Within weeks the same category structure was general enough to run a physics simulation, which is not the same as deriving physics and I do not claim it is. Formal verification was six months and a graveyard of dead ends further on. The hardware came after that. Every step pulled back to show another layer underneath that I had not known was there, and I mean that as plainly as it sounds: I did not know what a Turing machine was when I started, I did not know what Coq was, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The hands had nothing like this to draw on: no reference implementation, no prior project to point at. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the actual Coq and the actual math, the models produced and the proof checker judged. The proofs compile or they do not. I refused every one that did not. Coq is an obedient idiot. It checks the proof matches the statement. It does not check that the statement means what you think it means. A theorem can claim “every cat that is also a prime number has whiskers” and the proof can be three characters long and Coq will nod and stamp it. So I read every statement. The proofs you can’t fake. The statements

you can fake by accident, which is the more dangerous failure mode. I refused both kinds. However the symbols got onto the page, the argument on it is checkable by reading, and it would be exactly as right or as wrong if a million monkeys had typed it. The monkeys could not have refused.

A word on what I actually do, because it is stranger than the usual split and it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my mood and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, the proof checker takes them or throws them back, and I refuse anything that will not compile or does not match the shape I am holding. That refusal is the one thing in this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it is not background color, it is the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up, and not as a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

If you already know this material and the review of basics grates on you, skip [Section 2](#), which is the vocabulary I had to go look up to do any of this, and jump straight to [Section 9](#), where the formal machine state starts. But if you catch something in the

basics that is wrong, I want to know, and I mean that as a real ask and not a polite one. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

A note before the technical content, on what I do and do not know about prior work. I started this with no background in computer science, math, physics, quantum mechanics, hardware, or category theory. None. I didn't know what a Turing machine was. I didn't know what Coq was. I built this alone. Nobody mentored me or walked me through the field. The only person who has touched this work in any way is a friend who taught me how to use GitHub and set up an IDE. Every name, every comparison, every paper I went looking for, came from me searching on my own. So treat what follows as a list of labels I reached for, not a literature review; where I have mismatched one against the real thing, tell me, because that is a correction I can use.

The comparison with earlier work concerns the object each result studies. Landauer's erasure bound concerns physical dissipation under a thermodynamic model. No-free-lunch results concern averages over specified problem distributions. A2 is a lower-bound contract on designated certification transitions. Exact event pricing additionally imposes a no-overcharge condition. These comparisons distinguish the stated contracts; they do not establish historical priority or a unique choice of certification as the event to price. Primary sources for these limited comparisons are Landauer, "[Irreversibility and Heat Generation in the Computing Process](#)" (1961), and Wolpert–Macready, "[No Free Lunch Theorems for Optimization](#)" (1997).

Here are the labels I went and looked into, with one sentence each on what the label is and why I do not think Thiele reduces to it. Each is a claim inviting correction; if the comparison is wrong, tell me.

- Linear logic (Girard). What I read: a kind of logic where assumptions are consumed when you use them, like physical resources. Compiled into modern programming languages, it shows up as type rules that say "this value can only be used once." My reading of why Thiele is not a restatement: linear logic constrains what the program can write down. Thiele's A2 rule constrains what the machine's step function does. Same intent. Different layer.
- Proof-carrying code (Necula). What I read: when one machine downloads code from another, the code can carry a small proof that the host can check before running it. The proof is data accompanying the program. My reading: Thiele's cost-on-certification is enforced by the machine itself, not by a proof bundled with the program. The program does not carry receipts. The machine produces them. Foundational PCC (Appel) is closer in spirit: proofs about a small trusted base, not certificates bundled with the program. I haven't read enough Appel to say more.
- Session types (Honda). What I read: type systems that constrain how two or more concurrent programs talk to each other so the conversation cannot go off-

script. My reading: Thiele tracks structural facts a single computation has paid for. It does not constrain inter-program communication. Different problem.

- Bennett’s reversibility argument. What I read: Charles Bennett (1973) showed that you can in principle organise any computation so it does not erase information mid-stream. Operations that throw information away are the irreversible ones; everything else can be undone. My reading: in the 51-opcode realisation, the operations that commit information irreversibly, CERTIFY and MORPH_ASSERT and EMIT and REVEAL among them, are charged even at $\delta = 0$, a floor no program can waive; the reversible operations are designed to be free at $\delta = 0$ for the same reason Bennett identified. Positive cost is the larger set, since any instruction can carry a voluntary δ ; the mandatory floor is the part that tracks irreversibility. The shape matches. The dollars-and-joules calibration is a separate named hypothesis.
- Landauer’s bound. What I read: Rolf Landauer (1961) argued that erasing one bit of information must release at least $kT \ln 2$ of heat (Boltzmann constant times temperature times the natural log of 2) into the environment around the chip. My reading: this is the physical-units side of Bennett’s argument; in Thiele the structural side (the substrate tracks each positive-cost transition via μ) is proved, and the conversion to joules is a named hypothesis (see Section 17). I treat Landauer’s claim as motivation, not as something the proofs rest on. Motivation pays no bills in Coq.
- No Free Lunch (Wolpert and Macready, 1997). What I read: averaged over every possible problem, no search method beats blind guessing; whatever an algorithm wins on one set of problems it gives back on another. My reading: this is the same “structure isn’t free” intuition, but it lives as an average over a whole problem space, and the cost is refundable. You buy an edge on your problems by spending prior assumptions, and you pay for it somewhere else. A2 charges one commitment event in one run, with no refund anywhere. Same family. Different shape: an average, not a per-step floor.
- Kolmogorov complexity (Kolmogorov 1965; Solomonoff and Chaitin reached it independently around then). What I read: the amount of structure in an object is the length of the shortest program that prints it, and most objects have no short description, so you cannot squeeze below that floor. My reading: same “structure isn’t free” intuition, but it is a static property of a finished object. No steps. No moment of paying. A2 charges the moment a running machine commits to a certificate. Same family. Different shape: a property of a string, not a law of a step.
- Cook-Levin and P vs NP (Cook 1971, Levin 1973). What I read: checking a solution can be cheap while finding one looks expensive; whether finding is genuinely harder than checking is the open P-vs-NP question. Cook-Levin is the theorem that SAT is NP-complete, not a proof that search is hard. My reading:

same find-versus-check intuition, but it is an asymptotic, worst-case, still-open statement about how difficulty grows with problem size. A2 charges a constant on the one commitment step in a concrete run, whether finding it was easy or hard. Same family. Different shape: an open scaling gap, not a per-step floor. I am not claiming anything about P vs NP; see “What this project is not.”

- Rovelli’s cost of choosing (Rovelli 2024, *The Thermodynamic Cost of Choosing*). What I read: committing to one branch of a binary choice at temperature T dissipates at least $kT \ln 2$, the same floor Landauer found for erasure, now charged on the act of deciding rather than the act of forgetting. My reading: this is the closest existing thing to A2’s claim. It charges a commitment, not a finished object or an average. But it is physical: indexed to a temperature, living in thermodynamics. A2 charges the commitment in a substrate, with no temperature and no physics, blind to how the witness was found. Closest neighbour. Not the same chair.
- CerCo (certified cost annotations through a verified C compiler). What I read: take a C program, run it through a compiler that itself has been formally proved correct, and you can attach cost annotations to the source program that the compiler preserves all the way down to machine code. My reading: CerCo’s cost lives as decoration on the source code that survives compilation. Thiele’s cost lives in the step rule of the abstract machine, and the path down to hardware is a step-by-step bisimulation. Different mechanism. Different layer.
- Kami-RISC-V. What I read: a verified processor design written in Coq using the Kami framework that targets the RISC-V instruction set, which is an open-source CPU architecture (a published list of instructions any chip designer can implement). My reading: I use the same Coq-to-Kami-to-Bluespec pipeline that the Kami-RISC-V work pioneered. The instruction set is mine. The cost discipline is mine. The proofs are mine. The toolchain is shared. Nothing else is.
- NPA hierarchy (Navascués-Pironio-Acín). What I read: a way of bounding what kinds of correlations between two separated experiments are achievable using quantum mechanics. The bound takes the form of a sequence of polynomial inequalities, each tighter than the last. The bounds are usually checked by an external numerical solver. My reading: Thiele bakes one of those bounds into the machine’s step rule itself. The Z-arithmetic check that CHSH_LASSERT performs is proved equivalent to the Q_1 NPA condition by a chain of Coq theorems (Section 16), and lifted to Q_{1+AB} via a four-slice Schur cascade. Two levels of NPA, not the whole tower. It is operational enforcement at the step level, not a bound stated externally and verified out-of-band.

I read enough of each to convince myself Thiele is not a restatement, but I cannot say that with the confidence of someone who has actually read the field. The only things I will tell you with confidence are the ones where the argument actually closes: the structural axis is first-class substrate state, realised in the scaffolding by its own op-

codes; μ is proved unique on every reachable substrate state by `mu_is_initial_monotone`; No Free Insight follows from a one-step cost rule on any abstract certification system by `universal_nfi_any_substrate`; the Categorical Separation Theorem (Section 12) says the structural axis cannot be retrofitted onto a classical substrate without changing what a step is; and the CHSH-to-NPA biconditional drops out of the cost algebra without quantum input. The argument is the part I can defend; the proof corpus is just where I checked I hadn't fooled myself. The literature placement is the part where I am asking for help.

On the name. I wanted to call it the meta machine, and then Facebook bought the word “meta”: a company renaming itself after a term for self-reference and draining it of any in the same motion. So I went one level up, where this machine already lives. The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That's the whole of it, a meta move on a meta machine. It happens to be my dad's name too, and my wife's now, so I asked them before I did it. They said go ahead.

What this project is. The Thiele Machine develops a proposal about structural entitlement: when a computation has evidence and an admissible history supporting use of a structural fact. The abstract laws and the concrete instructions must be distinguished. Generic `CERTIFY` pays to set a flag without checking a proposition. Stronger semantic claims require explicit witnesses and sound checkers. The framework includes such specialized results, while the generic A2 theorem remains an accounting law.

The proved argument, in three steps.

1. A system satisfying A2 pays a positive total cost to move from uncertified to certified.
2. A projection that identifies states disagreeing on the ledger or another queried property cannot recover that property on every state.
3. A verifier restricted to colliding evidence cannot be both sound and complete for the distinguishing claim.

These statements concern explicit laws and observations. A conventional machine may retain richer encodings or enforce an invariant through its transition table. The foundational proposal concerns why the additional distinctions matter; it is not a theorem that all Turing-equivalent systems lack them.

Substrate vs scaffolding. Two layers run through the rest of this document, and I kept conflating them myself until the proofs forced the line; confuse them and the theorems quietly stop meaning what they say. The *substrate* is the general structural layer: a state type and a step relation, extended by different typeclasses for different theorems, and the extensions are not all the same requirement. The A2-respecting

extension, a certification indicator plus the rule that flipping it false-to-true costs at least one, is what No Free Insight needs; the ledger-initiality theorem separately fixes an initial value and increment schedule. These theorems hold for any state type and step relation carrying that extension, no matter what instruction set realizes it. The `WithShortcutPredicate` extension, a monotone-enough state type plus a shortcut-admittance predicate with two canonical witnesses and an internal recursion theorem, is what the structural-axis undecidability theorem in Section 19 needs instead, and that extension does not require A2 at all: the candidate-indexed `nat_substrate` family that supplies its restricted fixed points has no certification content whatsoever, by construction. “Substrate-level” does not mean one fixed interface every theorem shares; it means each result is stated over the particular typeclass extension it actually uses, not folded into the 51-opcode VM. The *scaffolding* is everything that makes a substrate runnable, verifiable, and synthesizable in practice: the 51 opcodes, the Python reference kernel, the extracted OCaml runtime, the Bluespec hardware, the simulator harness. The scaffolding-level constructions (the opcode semantics, the bisimulation between Coq and Verilog, the synthesised gate count) are how this particular substrate is made concrete enough that you can boot it, audit it, and run it on silicon. The 51-opcode VM realizes several of these typeclass extensions at once, an A2-respecting `CertCostMachine` and a `WithShortcutPredicate` instance among them, but a given substrate-level result borrows only the extension its own theorem is stated over. That is the distinction to hold onto: a substrate-level result is about the typeclass it names, and the 51-opcode VM is one instance realizing several such typeclasses simultaneously, not a single flat interface called “the substrate.” Let the distinction collapse and a theorem stated over one extension reads as though it carried premises it does not use.

What “new model of computation” means here. The model adds an explicit structural and accounting discipline to its execution semantics. It does not claim additional computable functions. “Classical shadow” refers to the selected projection or fragment, not all possible representations on conventional machines. The law’s enforcement, its representation, and its visibility to an observer are separate properties.

The renderer experience is the entry point. I started by trying to build a 3D engine where category-theoretic objects were the runtime objects. Modules. Morphisms. Composition. Not data structures simulating those things. The actual things. The classical CPU saw none of that. It shuffled bytes that happened to encode the structural state. Z3 saw formulas that happened to refer to the same shapes. The structural facts the engine was computing with were in the architecture, not in the data. That is the concrete shape I had to formalise. The substrate this document describes is what came out: compositional structure as state, not as an approximation written on top of state.

The formal results give me a precise version of the shadow argument. `thiele_simulates_turing` supplies a lift in its file-local transition model. `degenerate_projection_theorem` characterizes its selected quotient, and the VM separation

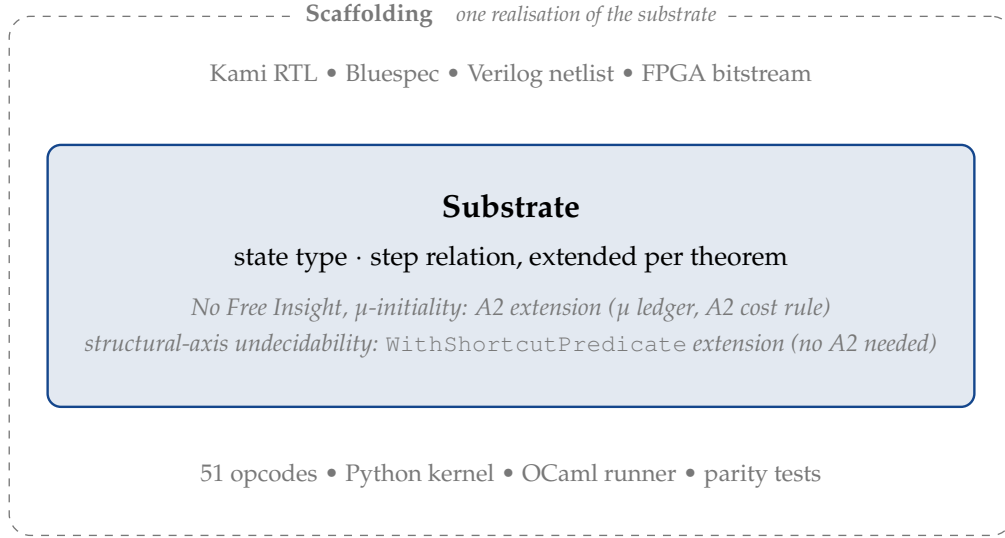


Figure 1: The substrate-versus-scaffolding distinction. The substrate is the general state-type-and-step-relation layer, extended by different typeclasses for different theorems: No Free Insight needs the certification-floor extension; ledger initiality fixes an initial value and increment schedule, structural-axis undecidability needs the separate `WithShortcutPredicate` extension and no A2 at all. Each substrate-level theorem holds for any system carrying the extension it actually uses, regardless of instruction set. The scaffolding is everything that makes this particular substrate runnable, verifiable, and synthesizable. Collapsing the two, or collapsing the two extensions into one, is the most common misread of the document.

witnesses show which structural distinctions its named observations lose. Recovering every original state from a noninjective projection is impossible. Choosing defaults is possible, but it does not tell you what the original state contained.

The irrecoverability theorems concern `vm_certified`, `csr_cert_addr`, and μ under their named projections. Multiple preimages prevent a left inverse recovering every original state. They do not prevent a section choosing defaults, or an information-preserving encoding with a decoder.

What is claimed unique, and what is not. A conversational shorthand for this project says there is only one possible substrate. That shorthand covers three different mathematical claims, and this document does not establish all three.

The first is uniqueness of a law, given the A2 cost rule, is there essentially one way to price certification honestly. Section 7 answers a version of this for a fixed schedule: `instruction_consistent_measure_equals_mu` shows any measure sharing the kernel’s own per-instruction schedule and a zero start is forced to equal μ on every reachable state. It does not show the schedule itself is forced. A different schedule prices differently and is a different, equally honest, accounting.

The second is uniqueness of an abstract structural core up to some stated equivalence: do all A2-respecting systems collapse, under the right notion of sameness, to one

shared mathematical object. This is the strongest reading of “only one substrate,” and it is not a theorem here. It is a proposal, stated so it can be attacked, not a claim this document proves. Call a system *adequate* if it carries an A2-respecting certification indicator, a step relation, and an observation interface rich enough to state No Free Insight, ledger initiality, and the diagonal. The proposed theorem form is

$$\text{Adequate}(M) \implies \text{StructuralCore}(M) \simeq \text{ThieleCore},$$

where `StructuralCore` strips a system down to the state, step, and certification data those theorems actually use, and $\simeq$ is an equivalence still to be pinned down, certainly not literal identity of opcodes, storage layout, implementation language, or clock cycles, since none of those enter any premise of any theorem in this document.

The history-retaining target is the test case that shows why $\simeq$ is hard to pin down and why this stays a proposal rather than a theorem. A target construction discussed later in this document (Section 3.7; the Coq witness is `certification_agreement_does_not_imply_descent`) retains the VM’s full instruction history, charges the same schedule, and reads certification the same way the VM does. It agrees with the Thiele VM on every certification and every charge. It still distinguishes the empty trace from `JUMP 0 0`, two traces the VM identifies, because it keeps the history and the VM does not. A2 and certification agreement alone do not force that extra history scaffolding to collapse away. Any adequacy condition strong enough to make `StructuralCore` identify this target with the VM has to say, explicitly, what makes the retained history inessential to the equivalence, and that is exactly the premise the proposal above leaves open.

The third claim, uniqueness of a particular state-transition object, this exact VM state space up to renaming, is not made anywhere in this document and is not what any theorem needs. The separating witnesses (Section 12’s graph pair, the irrecoverability theorems above) stand on their own regardless of how the second claim resolves. They do not require it, and it does not require them.

So: the first claim is proved, for a fixed schedule. The third claim is not made. The second is the open one, offered here as $\text{Adequate}(M) \implies \text{StructuralCore}(M) \simeq \text{ThieleCore}$, a conjecture with a stated test case it has to survive, not a theorem this document proves.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The CHSH algebraic result is pure math. The Landauer connection is motivation. That is the full extent of the physics content here. The rest is bookkeeping.

2 Things I had to learn first

I had been around computers my whole life and paid attention to people like John Carmack on how graphics actually get made, but I came in with none of the formal fields this work turned out to need. By the time the proofs compiled I had needed to learn enough of half a dozen of them that I have to stop here and put down what those fields had words for, before the rest of the document starts using them.

Everything below is what I understood after I went and read about it. If I have anything wrong, tell me. I am not claiming to teach you these subjects. I am putting down what I had to take from each of them to get this work to compile. If you already know one of these subsections, skip it.

2.1 Bits, bytes, and what a CPU sees

A bit is one digit that is either zero or one, the smallest fact a machine can hold. Everything else is bits stacked up. Eight of them stacked make a byte, which lands you any number from 0 to 255, and the first thing that genuinely surprised me is that the same byte is also a letter: a convention called ASCII (American Standard Code for Information Interchange) simply agrees to call 65 the letter A, 48 the digit 0, 32 a space. Nothing in the byte knows which it is. The meaning is a convention laid on top of the data, and which convention you read it through is your choice, not the byte's. Hold onto that, because it is the whole argument of this document shrunk down to one byte: the data sits there inert, and everything that matters is in the structure you read it through. I learned the byte part in February.

A word is a chunk of bits the CPU treats as one unit. The kernel math uses 64-bit words (each word can hold any number from 0 to $2^{64} - 1$, about 18 quintillion). The synthesised hardware uses 32-bit words (each word holds 0 to $2^{32} - 1$, about 4 billion). A word is just a sequence of bits with a fixed width. The substrate does not care which width.

A register is a single named storage slot inside the CPU that holds one word. The 51-opcode VM has 16 of them, named r0 through r15. Registers are the fastest place to read or write a value. Memory is a longer list of word-sized slots, accessed by index (the "address"). The 51-opcode VM has 128 words of memory. That is the entire address space. The substrate has no address space.

The program counter (PC) is one specific register that tracks where in the program the VM is. After every instruction the PC moves forward by one unless the instruction explicitly says jump to a different address.

A stack pointer is a register that points to the top of a region of memory the program uses for nested function calls. CALL pushes a return address there; RET pops it. In the 51-opcode VM, r15 is the stack pointer by convention. The convention is mine. The substrate is not consulted.

A port is a named channel the 51-opcode VM uses to communicate with the outside world. `READ_PORT` brings a value in, `WRITE_PORT` sends a value out. The kernel does not model the outside world. It just charges μ for the communication and accepts the value the channel encoding declares. The outside world is welcome to comply. The substrate has no ports.

Modular arithmetic means counting wraps around at a maximum. If the maximum is 16, then 17 wraps to 1 and 19 wraps to 3. “Modulo 16” or “mod 16” means “wrap at 16.” The 51-opcode VM wraps register indices modulo 16 and memory indices modulo 128. If you write `r17`, the kernel treats that as `r1`.

Bitwise operations work on the bits of a value rather than treating the value as a number. Bitwise AND of two bits is 1 only when both bits are 1. Bitwise OR is 1 when either is 1. Bitwise XOR (exclusive or, sometimes written $\oplus$) is 1 exactly when the two bits differ. The bitwise version of these does the operation in parallel for every bit position. Bitwise-AND of two 64-bit words gives a 64-bit word. Bit i of the result is the AND of bit i of each input. The bits do not consult each other. The substrate does not see bits.

A shift moves the bits of a value left or right. Left shift by 3 (sometimes written $x \ll 3$) moves every bit three positions to the left, which is the same as multiplying by $2^3 = 8$. Right shift divides by a power of 2. Same idea, opposite direction.

Popcount is the count of 1-bits in a value. Popcount of the byte `11010100` is 4. Sometimes called the Hamming weight; the two names refer to the same thing. Pick one.

Hexadecimal is base 16. Each “digit” is one of 0 through 9 or A through F, where A means 10 and F means 15. Hex is convenient for working with bytes because every two hex digits make one byte. Numbers prefixed with `0x` are hex: `0xFF` is 255, `0xCAFEFACE` is the constant unlock key in the Logic Engine accumulator (the Logic Engine is the machine’s on-chip formula checker; Section 5 builds it). It had to be something.

A trap, in the hardware sense, means the machine routes execution to a fixed address instead of advancing normally. It is the hardware equivalent of “this went wrong, go to the error handler.” The 51-opcode VM traps `LASSERT` length mismatches to address `0xF00` (`LASSERT` is the logic-assertion opcode; Section 10 builds it).

An ISA (instruction set architecture) is the list of all instructions the CPU recognises, plus their formats and costs. Each instruction is identified by an opcode (a numeric tag the hardware decodes to choose which behaviour to run). The kernel has 51 opcodes. The substrate has none. The substrate has a step rule.

2.2 Math notation I will use without explaining it twice

I read the math symbols out loud the way I learned them, which is whatever the source I was reading suggested they were pronounced as. If you have a different convention

in your head, here is what I mean by them.

$\mathbb{N}$ is the natural numbers: 0, 1, 2, 3, and so on. $\mathbb{Z}$ is the integers, which extend the natural numbers with negatives: $\dots, -2, -1, 0, 1, 2, \dots$. $\mathbb{R}$ is the real numbers, all the way along the number line including fractions and irrationals like π and $\sqrt{2}$.

$\in$ means “is an element of.” $x \in S$ reads “ x is in S .”

$\subseteq$ means “is a subset of” (every element of the left side is also in the right). $\subsetneq$ means strict subset: a subset but not the whole thing.

$\cup$ is set union, the set containing everything in either side. $\setminus$ is set difference: $A \setminus B$ is everything in A that is not in B .

$\times$ between two sets is the Cartesian product: $A \times B$ is the set of all (a, b) pairs with $a \in A$ and $b \in B$. Functions from a set A to a set B are written $A \rightarrow B$.

For a finite set S , $|S|$ means the number of elements in S . For a real number x , $|x|$ means the absolute value of x (its distance from zero).

$\sum_{i \in T} f(i)$ is the sum of $f(i)$ taken over every i in T . The big sigma is the summation symbol.

Δx means “the change in x ,” usually the difference between a final and an initial value of x .

$\Rightarrow$ is logical implication. $P \Rightarrow Q$ means: if P is true, then Q is true. $\iff$ is the biconditional, also called “if and only if.” $P \iff Q$ means both $P \Rightarrow Q$ and $Q \Rightarrow P$.

$\forall$ is “for every.” $\exists$ is “there exists.” $\forall x. P(x)$ reads “for every x , $P(x)$ is true.” $\exists x. P(x)$ reads “there is some x that makes $P(x)$ true.”

$\langle X \rangle$ around a random variable X means the expected value, the long-run average value of X . I picked up this notation reading physics papers about Bell tests.

$\circ$ between two functions is composition: $(f \circ g)(x) = f(g(x))$, first apply g , then apply f . The same symbol appears in category theory for arrow composition.

$\oplus$ in this document is bitwise XOR. $\otimes$ in this document has exactly one meaning: the MORPH_TENSOR operator of Section 10, concatenation of two coupling lists into one. Physics writes the same symbol for the tensor product of quantum operators; that sense never appears on these pages, because Section 16 does its quantum work in plain integer arithmetic.

I write A^T for the transpose of a matrix A (rows and columns swapped). I_n is the $n \times n$ identity matrix, which has 1 on the diagonal and 0 everywhere else.

2.3 Coq, in just enough detail

Coq is a proof assistant. It is a programming language. The type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid. Section 20 has the longer build, including why I trust it. Here is the vocabulary that runs through the rest of the document.

A Theorem in Coq is a named statement followed by a proof. Lemma is the same construct, conventionally used for intermediate results. Definition introduces a named value (a number, a function, a record, anything). Inductive defines a type with a finite list of named constructors, the only ways to build a value of that type. Coq's natural numbers, for example, are an inductive type with two constructors: zero, and "successor of another natural number." Two constructors, infinitely many naturals.

A tactic is a step inside a Coq proof. The tactics that appear in this document include: induction (prove the base case, then prove that if the statement holds at step n it holds at step $n + 1$), destruct (split a hypothesis into its cases), exact (provide the literal proof term), reflexivity (prove $x = x$), simpl (simplify the goal by computing), lra (linear real arithmetic; closes goals that are linear inequalities over the reals), nra (nonlinear real arithmetic; handles polynomial inequalities), and psatz (uses Positivstellensatz certificates; it shows a polynomial is non-negative by writing it as a sum of squares). I learned each of them by needing it.

Qed is the keyword that ends a proof. It asks Coq to check the entire chain. If Coq accepts the proof, the theorem is proved.

Admitted is the keyword for "I am claiming this without proof." This is what I am not doing anywhere in the active proof tree. Every Admitted in the kernel would be a bug. Section 21 explains what zero Admitted means. And what it does not.

A Hypothesis (a Coq keyword inside a Section) is a named premise: a statement we accept as given without proof. Hypotheses surface as $\forall$ -quantified premises in the conclusion. I use Hypothesis only at named bridge points (the physical-units calibration in Section 17 is the load-bearing example). An Axiom is the global version: a top-level claim accepted without proof. The kernel tier has zero project-local Axiom declarations. The policy is bookkeeping discipline, not faith.

Print Assumptions is the Coq command that lists every Axiom and Hypothesis a given theorem ultimately depends on. Running Print Assumptions on the kernel tier shows only standard Coq stdlib axioms (functional extensionality, classical logic for the reals). No project-local axiom appears anywhere.

A total function in Coq always returns a value. Coq requires every function definition to be total. `vm_apply` is total. Every state-and-instruction pair has a defined result. Including the error case.

"Parametric in K " means the proof works for any value of K . Register, memory, and

table sizes are explicit constants in the implementation. Changing them requires rebuilding the affected definitions and checking their representation obligations. A proof about the abstract state alone does not establish that every size choice fits a physical target.

2.4 Category theory: the part I needed beyond Section 11

Section 11 builds Category, Object, Morphism, Composition, Identity, and Associativity from scratch. That build is enough for the categorical-separation argument inside this machine. Here is what I had to look up beyond that.

A functor is a structure-preserving map from one category to another. Given two categories $\mathcal{C}$ and $\mathcal{D}$, a functor F from $\mathcal{C}$ to $\mathcal{D}$ sends each object of $\mathcal{C}$ to an object of $\mathcal{D}$. It sends each arrow $f : A \rightarrow B$ in $\mathcal{C}$ to an arrow $F(f) : F(A) \rightarrow F(B)$ in $\mathcal{D}$. It respects composition ($F(g \circ f) = F(g) \circ F(f)$) and identity ($F(\text{id}_A) = \text{id}_{F(A)}$). Think of a functor as a translation rule between two categorical worlds that keeps the laws intact.

An initial object in a category has exactly one morphism into each object. Here `thiele_trace_fold_initial` proves unique evaluation from free instruction lists into a target with a chosen basepoint. It does not prove that the VM state space is initial under certification-preserving state maps. The latter maps may fail to exist; when supplied, their uniqueness on reachable states is a separate theorem.

An embedding is a functor that throws nothing away. That is the whole point of putting the word here: the classical projection of a Thiele state loses things on the way out, and an embedding is the move that does not. Faithful is the precise version of “loses nothing on arrows”: if two arrows between the same pair of objects in $\mathcal{C}$ land on the same arrow under F , they were already the same arrow, so F never quietly fuses two distinct things into one. That is injectivity on each hom-set, checked one pair of objects at a time. Conservative is the version for invertibility: if $F(f)$ has an inverse over in $\mathcal{D}$, then f already had one back in $\mathcal{C}$, so F cannot manufacture an isomorphism that was not there. (An isomorphism is just an arrow with an inverse.) These are different properties, and neither implies the other. The inclusion of the one-object category of the monoid $(\mathbb{N}, +)$ into the one-object category of $(\mathbb{Z}, +)$ is faithful (distinct natural numbers stay distinct integers) but not conservative: 1 has no inverse in $\mathbb{N}$, yet its image in $\mathbb{Z}$ does. Conversely, the functor from the one-object category of the group $\mathbb{Z}$ to the trivial one-object group is conservative (every arrow in the trivial group is already invertible, so nothing needs reflecting) but not faithful: it fuses every integer onto the same single arrow. Both properties rule out a map inventing structure it should not have, an embedding needs faithfulness specifically, not conservativity, but they rule out different structure and neither substitutes for the other.

A monoidal product is a way of combining two objects, or two arrows, into one. In the category of sets, the Cartesian product $A \times B$ is a monoidal product on objects. In the category of Thiele modules, the operation behind `MORPH_TENSOR` is a monoidal

product on morphisms. Take two morphisms with disjoint source and target regions. Pack them into a single morphism whose coupling is the concatenation. The symbol $\otimes$ stands for this in Section 10.

Yoneda’s lemma is named once in the Section 1 origin story and never appears again. You can ignore it. I dropped it during the work because I did not need it for any theorem. I left the name in. That is when I first knew the work was assembling.

2.5 Bell-test vocabulary you will meet in Section 16

Section 16 builds the CHSH game from scratch. A few foundational terms thread through that section that I will not rebuild each time.

A correlation between two outcomes is the average value of their product. If Alice’s outcome a is ± 1 and Bob’s outcome b is ± 1 , the correlation is $\langle ab \rangle$, which lies between -1 and $+1$. A correlation of $+1$ means they always match. -1 means they always disagree. 0 means there is no average correlation. A correlator is one specific correlation in a labelled setting: for example, E_{00} is the correlation when Alice’s setting is 0 and Bob’s setting is 0 .

The marginal of a joint distribution on (a, b) is the distribution on a alone, with b averaged out. Zero-marginal means Alice’s marginal gives $\langle a \rangle = 0$ and Bob’s gives $\langle b \rangle = 0$. Each outcome is $+1$ or -1 equally often, on average. The Section 16 derivation works in this symmetric slice.

A local hidden variable model is a classical-physics-style theory where Alice’s and Bob’s outcomes are determined by a shared random variable λ they agreed on before the game started, plus their own measurement setting. “Local” means no communication during the game. “Hidden” means λ need not be directly observable. “Variable” because λ is random. Bell’s theorem says no local hidden variable model can produce certain correlations that quantum mechanics can.

A no-signaling correlation is one where Alice’s marginal does not depend on Bob’s setting, and Bob’s marginal does not depend on Alice’s setting. You cannot learn Bob’s setting by looking only at Alice’s outputs. Not even in principle. Most correlations we discuss (classical, quantum, and the PR-box below) are no-signaling.

Quantum-realizable, in this document, means “compatible with some quantum-mechanical state-and-measurement description.” The NPA framework (Navascués-Pironio-Acín) gives a precise test for this in terms of positive-semidefiniteness of a moment matrix. The test is decidable. Section 16 builds the test.

The PR-box (Popescu-Rohrlich box) is a hypothetical correlation that wins the CHSH game with probability 1 , achieving $|S| = 4$. It is no-signaling, but stronger than anything quantum mechanics allows. Nature does not provide it. It is the standard counterexample to the false claim “any no-signaling correlation is quantum-realizable.”

The Cauchy-Schwarz inequality is the basic inner-product inequality $|\langle x, y \rangle| \leq \|x\| \cdot \|y\|$. It reads: the inner product of two vectors is bounded by the product of their lengths. It is one of the workhorse algebraic moves in the Section 16 derivation.

The Tsirelson bound is the value $2\sqrt{2} \approx 2.828$, the largest CHSH score $|S|$ that any quantum-mechanical strategy can achieve. The classical bound is 2. The PR-box bound is 4. Tsirelson sits strictly between classical and PR-box. Nature picked one of these three.

The elliptope, named twice in Section 23 and Section 24, is the convex set of valid correlation matrices that arise from quantum states. The zero-marginal NPA slice of the elliptope is the part the proof in Section 16 covers.

2.6 Hardware and synthesis vocabulary

Section 14 builds the scaffolding stack (Coq kernel, OCaml runner, Kami RTL) carefully. Here is the synthesis-tooling vocabulary that Section 14's third subsection uses without expanding each tool.

A hardware description language (HDL) is a programming language for describing circuits. Bluespec SystemVerilog and Verilog are two HDLs. Bluespec is higher-level: you describe rules and the compiler figures out the gate-level details. Verilog is lower-level: you describe gates and wires more directly. Verilog RTL specifically means "register-transfer level" Verilog, which describes how data moves between registers each clock cycle. It is what synthesis tools consume. They do not eat Bluespec.

Synthesis is the process of compiling a hardware description down to actual logic gates (the building blocks of chips). yosys is an open-source synthesis tool. After synthesis, place-and-route assigns the gates to physical positions on a chip and connects them. nextpnr-xilinx is the place-and-route tool for the Xilinx FPGAs this design targets. The last stretch is producing a bitstream, the binary file the FPGA reads to configure itself, and prjxray does it in two moves: fasm2frames packs the routed design into configuration frames, then xc7frames2bit writes the frames out as the bitstream. Four steps, four tools, one chip.

An FPGA (field-programmable gate array) is a chip you can program after manufacturing. The gates are physical. The connections between them are configurable. An ASIC (application-specific integrated circuit) is the non-programmable version: gates fixed at manufacturing time for one specific design.

A LUT (look-up table) is the basic logic primitive in an FPGA: a small memory that maps a fixed-length list of inputs to an output, programmed at configuration time to implement any small boolean function. A flip-flop is a one-bit memory cell that holds its value across clock cycles. A carry chain is specialised wiring for fast addition. Distributed RAM is RAM built from LUTs. Block RAM is dedicated RAM blocks on the chip. DSP slices are specialised hardware for digital-signal-processing arithmetic. The

synthesised K325T design uses 0 DSP slices because yosys is invoked with `-nodsp`, mapping the wide multiplier in the multi-cycle witness checker (Section 14) to LUTs instead. The substrate would also accept a DSP-inferred mapping. The choice is a chip-capacity and place-and-route runtime trade-off, not a correctness one.

A JSON netlist is the gate-level circuit serialised in a JSON text file (JSON is a standard text format for structured data). A VCD waveform is a Value Change Dump, a record of every signal's value at every time step in a simulation. The name describes the file. iverilog is an open-source Verilog simulator. It runs the design and produces a VCD output you can inspect.

A bisimulation is a precise notion of “two systems behave identically.” Given the same starting state and the same input, they produce the same output state, field by field. Section 14 builds it in context. The substrate-to-scaffolding bisimulation is the path the document walks. A parity test is the empirical cousin: run two implementations on the same battery of inputs and check they agree, a correspondence earned by testing rather than by proof. The kernel-to-OCaml boundary is closed that way, and the document says so where it happens.

2.7 Other terms I use without rebuilding

Provenance, in software, means the recorded chain of where an object came from. A morphism's provenance is the specific sequence of MORPH and COMPOSE instructions that built it. You can replay the chain.

Simulation, in computer science, means one system imitating another's input-output behaviour. A Turing machine simulates a Thiele Machine when it produces the same outputs on the same inputs, by writing the whole Thiele state out across its tape and grinding through it. That is real, and it matters, and it is exactly the thing people reach for when they want to wave the whole question away: if a tape can stand in for the state, what was the difference? The difference is that the tape carries a picture of the state, not the state, the way a photograph of a fire keeps you exactly as warm as any other photograph. The cost ledger and the A2 rule live in the step relation, not in the bytes, and an encoding copies the bytes. Simulating the substrate is not being the substrate.

Projection, in mathematics, is a function that drops information on purpose. The classical projection of a Thiele state is the one that does the dropping I keep pointing at: it throws away μ , certification, and the morphism graph, and keeps registers, memory, and the program counter. That is the exact list. The three it keeps are the three a Turing machine already has, which is the whole reason the projection lands you back in ordinary computation. The three it drops are the three that were doing the new work. Nothing about the projection is wrong; it is a perfectly good function. It just costs you everything that made the state more than a tape.

Quotient, in mathematics, is the operation of declaring two things the same when they

agree on what you decided to look at, and then working with the lumped-together classes instead of the originals. The classical projection is exactly that kind of move: any two Thiele states that match on registers, memory, and the program counter get swept into the same class, even when their μ , their certification, and their morphism graph were completely different. So the projection is not just dropping fields one at a time; it is gluing together states that the dropped fields kept apart. That is the precise sense in which it loses information, and it is comforting that the math already had the word sitting there waiting.

A kernel of a map (math sense) is the set of things the map sends to a designated identity value. “Kernel” wears three different hats in this document and I would rather call that out than let it ambush you: the math sense just defined; the Coq proof core, meaning the kernel-tier theorems in Section 23, which is scaffolding I lean on, not the substrate itself; and the operating-system kernel, which I imply at most by analogy and never actually use. Whenever the word shows up, the surrounding sentence tells you which one is meant.

Embedding is a function that preserves structure and is injective. Faithful embedding additionally preserves all the structure-relevant distinctions.

An injection is a function where no two inputs map to the same output. A surjection is a function whose image covers every possible output. Their negations are noninjective and nonsurjective.

Decidable, in computer science, means there is an algorithm that always halts with a yes-or-no answer. The integer-only Z-arithmetic check in CHSH_LASSERT is decidable; the algorithm computes the result in finitely many steps and returns a Boolean.

Polynomial complexity: an algorithm is polynomial-time if there is some power k such that the algorithm’s running time on inputs of size n is at most a constant times n^k . Polynomial-space is the analogous concept for memory used. The class P is decision problems solvable in polynomial time. The class NP is decision problems where, given a candidate solution, a polynomial-time algorithm can check whether the solution is correct. The famous P-vs-NP question asks whether finding is fundamentally harder than checking. Nobody has proved either direction. Section 24 discusses why my `mu_budget_decidable` / `mu_budget_verifiable` predicates are not a P-vs-NP result.

Determinant: a number computed from a square matrix that says whether the matrix is singular (zero determinant means the matrix collapses some direction to zero). For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ the determinant is $ad - bc$. For larger matrices, you can compute it by “expansion along a row”: pick a row, for each entry take the determinant of the submatrix you get by deleting that entry’s row and column (with alternating signs), multiply by the entry, and sum. The recursion bottoms out at the 2×2 case above.

Section 16 walks the explicit calculations for the 3×3 cases I need.

Schur complement: a way of turning a question about a block matrix into a question about a smaller matrix. If $\begin{pmatrix} A & B \\ C & D \end{pmatrix}$ is a block matrix where A is invertible, the Schur complement of A is $D - CA^{-1}B$. A standard linear-algebra result says the block matrix is positive-semidefinite if and only if both A and the Schur complement are positive-semidefinite. The Section 16 build uses this for the 4×4 block of the NPA moment matrix.

A principal submatrix is the square subblock you get by picking a subset of indices and keeping only the rows and columns at those indices. A standard result is that a matrix is positive-semidefinite if and only if every principal submatrix is positive-semidefinite. Every one. Section 16 relies on this.

2.8 The axioms (A1 through A6)

The Thiele Machine has a small numbered list of substrate-level axioms. They appear by name (“A2,” “A3,” ...) throughout the document and the formalization. The list below is what each name means, where it lives in the kernel, and what its conclusion is. The axioms are substrate-level; the kernel is scaffolding-level; the list bridges the two.

The kernel keeps two parallel axiom-numbering systems because two parallel substrate abstractions are formalised. The `AbstractCertMachine` record carries A1 (the cert indicator exists, by record existence) and A3 (only cert-setters flip it, the `acm_preserve` field). A2 (cert starts unset) and A4 (cert-setters cost ≥ 1) live alongside the record, not in it: A2 is the `acm_cert M s0 = false` hypothesis to `abstract_nf1`, A4 is the standalone lemma `cert_addr_setter_cost_pos`. The consolidated `CertificationSystem` record folds the cost-on-certification semantic content into a single field kept by the name A2 (the load-bearing piece). The `QuantitativeCertificationSystem` extension adds A3–A6, which lift the cost-floor result from ≥ 1 to $\geq K$ for an arbitrary threshold K . The two numbering systems agree on the cost-on-certification semantic content; they disagree on which axiom carries that content (A4 in the `AbstractCertMachine` list, A2 in the consolidated `CertificationSystem` list). When the rest of this document says “A2,” it means the cost-on-certification rule, in the consolidated `CertificationSystem` sense.

A1 (state includes a certification indicator).

An `AbstractCertMachine` record bundles a state type S , a step function `acm_step`, and a boolean cert indicator `acm_cert : S → bool`. A1 is the structural requirement that this indicator exists at all. In Coq, A1 is enforced by the record type itself. There is no way to hand Coq an `AbstractCertMachine` value with `acm_cert` left blank; it reads the field as missing and declines to build the record, the way a clerk declines a form with an empty box. A1 is discharged at type-check time.

A2 (cost-on-certification).

In the consolidated `CertificationSystem` record, Coq field `cs_cert_costs`. Statement: for every state s and instruction i , if `cs_cert s = false` and `cs_cert (cs_step s i) = true`, then `cs_cost i` ≥ 1 . In words: any single step that flips the certification predicate from no to yes must have positive cost. This is the load-bearing axiom for No Free Insight. `universal_nfi_any_substrate` concludes from A2 alone that every trace from uncertified to certified has total cost ≥ 1 . Note: in the `AbstractCertMachine` numbering, the same content appears as A4 (cert-setters cost ≥ 1). The `AbstractCertMachine` A2 is the separate precondition that the cert indicator starts unset, which the consolidated system handles as a theorem hypothesis instead of a numbered axiom.

A3 (cost bounds witness growth).

Coq field `qcs_cost_bounds_witness` on the record `QuantitativeCertificationSystem`. Statement: for every state s and instruction i , `qcs_witness s + cs_cost i` $\geq$ `qcs_witness (cs_step s i)`. In words: each instruction's cost is an upper bound on how much the system's accumulated evidence (the "witness") can grow in that step. The cost is the speed-limit on evidence accumulation. (In the `AbstractCertMachine` numbering, A3 is the field `acm_preserve`: non-cert-setter instructions preserve the cert indicator. The two A3s are distinct properties on distinct record types; the monograph's A3 is the quantitative one above.)

A4 (witness nondecreasing).

Coq field `qcsf_witness_nondecreasing` on the record `QuantitativeCertificationSystem_full`. Statement: for every s and i , `qcs_witness s` $\leq$ `qcs_witness (cs_step s i)`. A3 and A4 point in opposite directions: A3 caps how fast the witness can grow ($w' \leq w + \text{cost}$), A4 stops it from shrinking ($w \leq w'$). Neither implies the other. $(w, \text{cost}, w') = (1, 0, 0)$ satisfies A3 with non-negative cost yet violates A4, and $(0, 0, 1)$ does the reverse, so A4 is its own field, not a consequence of A3. It is also idle: the $\geq K$ floor telescopes through A3, A5, and A6 alone and never touches A4. The field is kept so downstream consumers that want witness monotonicity in hand have it without re-deriving it; the consolidated `QuantitativeCertificationSystem` record drops A4 entirely, carrying only A3 and A5.

A5 (certification requires witness).

Coq field `qcs_cert_threshold_witness` on `QuantitativeCertificationSystem`. Statement: `cs_cert s = true` $\rightarrow$ `qcs_witness s` $\geq$ `qcs_threshold`. In words: certification cannot fire until the system has accumulated at least `qcs_threshold`-worth of evidence. This is what lets the quantitative theorem push the cost floor from ≥ 1 to $\geq K$, where K is the threshold. Pay more, certify more.

A6 (witness initial zero).

The hypothesis that the run starts with zero witness, stated as a precondition on the quantitative theorem rather than a record field. Without A6 the telescoping bound becomes $K - \text{initial_witness}$ instead of K . Zero is a clean place to start.

A1 through A6, as the consolidated records state them, are independent of any specific instruction set; the `AbstractCertMachine` variant is the one exception, typing its step over the VM's own instruction type. They are conditions on Coq record types; supplying instances of those record types is the only way to invoke the corresponding theorems. The 51-opcode VM supplies them by construction: the kernel builds two concrete `CertificationSystem` values for the VM (one for the `csr_cert_addr` channel, one for the `vm_certified` channel), each discharging A2 from the existing `no_free_certification` kernel theorems. A separate substrate-level abstraction (the `Substrate` typeclass) carries a related but distinct field `mu_monotone` stating that the cost ledger never decreases on a step. `mu_monotone` is a strictly weaker general-purpose monotonicity property than A2: A2 specifically asserts the cert transition costs ≥ 1 , whereas `mu_monotone` only asserts the ledger does not go down. The two coincide on the 51-opcode VM, where every instruction adds non-negative cost and the cert transition adds at least 1. Two locks on the same door, by different keys.

That is the vocabulary. From here on, the document uses these words without rebuilding them. Section 9 is where the formal machine state begins; experts should jump there. Everyone else stays for the walk.

3 The structural axis: the projection classical models drop

Why this section exists. This section exists because a renderer I was building in January 2025 walked me into a wall, and the wall turned out not to be made of what I thought.

The renderer was a categorical engine. I had built it so the things running at runtime were the category-theory objects themselves: modules were objects, transformations between them were morphisms, composing two transformations was literally composition, and running two independent parts side by side was the monoidal product (Section 2 builds all of these). I was not simulating category theory on top of ordinary data. The structure was the thing the engine ran on. And it worked, which is the part I still find a little funny: within weeks the same engine that was supposed to draw pixels was running physics on the identical primitives, and I had not planned that and did not at first believe it.

Then I tried to prove something about what it was doing, and the floor went out. The machine underneath, the ordinary CPU, had seen none of the structure the entire time. (That ordinary architecture, one processor and the memory passing data back and forth across a single bus, is the von Neumann machine, after John von Neumann who wrote it down in 1945; every laptop and phone and server you have ever touched is one.) It had been faithfully shuffling the bytes my modules and morphisms happened to be spelled out in, faithful the way a printing press is faithful to a poem it cannot read. When I brought in Z3 to verify the morphism chains the engine was reasoning with, I met the same blindness in different clothes: Z3 saw formulas, true ones, that happened to refer to the shapes, and it never once saw a shape. For a long stretch I

took this for a Z3 problem, a tooling problem, the kind you solve by getting cleverer with the solver. It was not a Z3 problem. The structure I wanted checked was real and was carrying the whole computation, and there was nowhere in the machine for it to sit as a thing the machine itself had to answer for. I was asking a tool to certify a fact about the one part of the computation the computation had no place to keep. You do not get cleverer your way into a slot that was never built.

The Thiele Machine is that slot. It is the substrate the engine had needed all along and could never be handed on the hardware it was running on: a machine where the structure is first-class state instead of a pattern in the bytes, where composing that structure is a step the machine takes rather than something you read back out afterward, and where committing to a structural fact is an act the machine is charged for and cannot quietly perform for free. Everything past this point is that substrate's correctness conditions, laid out in English you can follow and check yourself, with a Coq proof standing next to each one. The Coq is not what makes any of it true. It is there because my own eye slides straight over the gaps in an argument I am hoping is right, and I wanted a reader that would not let me. This section is the long-form answer to a question a stuck renderer put to me and would not drop until I quit trying to out-clever it and went looking for what was actually missing.

A2 is expressed in the transition interface, as the proof field `cs_cert_costs`. Within the VM, it is established from the instruction semantics. Observers that discard the relevant fields cannot reconstruct them from the remainder. The irredundancy result is relative to those projections, not a requirement for separately named fields in every implementation.

The law is meaningful beyond this ISA: the abstract state type and predicate are parameters. This supports studying many instances. It does not establish that all observable changes have positive cost or that arbitrary predicates are independent geometric dimensions.

3.1 Substrate-independence: the abstract setup

The kernel theorem `universal_nfi_any_substrate` is written to be as hard to wriggle out of as I knew how to make it, and the way you make a theorem hard to wriggle out of is to make it ask for almost nothing. It takes any abstract computational system at all, and by any I mean it genuinely does not care what the system is made of or whether it looks anything like the machine in this document. It asks for five pieces and refuses to ask for a sixth. A set of states, and a state can be whatever it is for you: bits on a tape, words in a register file, cells in memory, it makes no difference to the theorem. A set of instructions, the moves the thing can make. A step rule, which takes a state and an instruction and hands back the next state. A cost function, which pins a non-negative integer to each instruction. And a certification predicate, a plain yes-or-no on states that says whether a given state counts as certified. That is the entire list. The reason I keep leaning on how short it is: any objection that begins "sure, but

that only holds for your particular machine” has nothing to attach to once the result has agreed to run on something this bare.

One premise, labelled A2 and named `cs_cert_costs` in the kernel: if any single step flips the certification predicate from no to yes, the instruction that did it cost at least 1. One conclusion: every trace that starts uncertified and ends certified has total cost at least 1.

And it does not care how powerful the thing is, in either direction. It never asks whether your system can compute everything a Turing machine can (the word for that is Turing-complete: you are Turing-complete if you can simulate a Turing machine, at most Turing-equivalent if a Turing machine can simulate you). It never asks for the reverse either. Stronger than a Turing machine, weaker, dead even, none of it reaches the floor. Bring the five pieces, charge A2 on the flip, and you are in scope. The floor does not check your computational class at the door, because it was never about what you can compute. It is about what your step is allowed to do for free.

The categorical-separation result in Section 12 concerns the specified observable tuple. Two states may agree there and differ in their graph. A full conventional encoding could preserve the graph; a decoder could then inspect it. The theorem identifies what this observation discards.

The abstract theorem requires a designated predicate and the A2 cost premise. An arbitrary Turing-machine trace is not an instance merely because it is a computation. It becomes an instance when the required state interpretation, step function, cost, and proof are supplied.

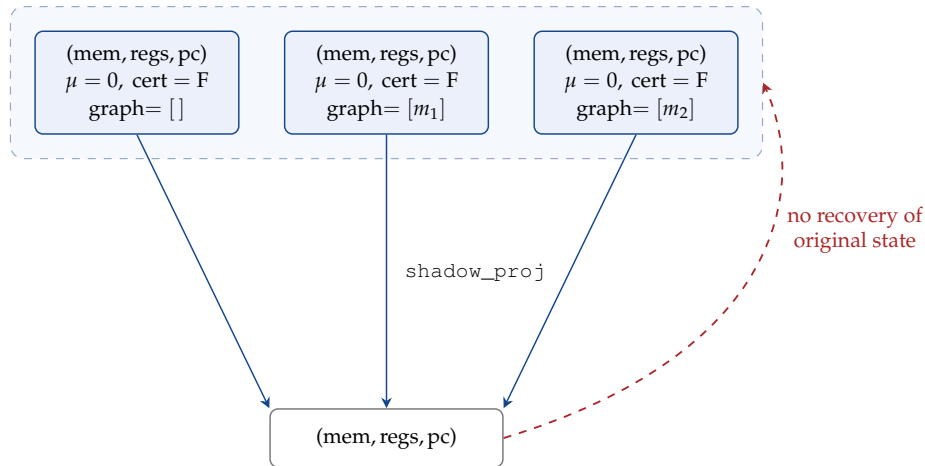


Figure 2: Selected projections identify richer states. Multiple preimages rule out recovering every original state from its projection. A section choosing default metadata remains possible; it does not recover the omitted original data. The diagram suppresses some retained fields for legibility.

3.2 What is canonical, and under which conditions

Three results should be kept separate. `universal_nfi_any_substrate` proves a floor for every system satisfying A2. `mu_is_initial_monotone` proves ledger uniqueness on reachable states given the initial value and exact instruction increments. The projection theorems identify information lost by specific observations.

The abstract `CertificationSystem` carries an arbitrary state type, instruction type, step function, nonnegative instruction cost, Boolean predicate, and proof of A2. Its state need not have separate cost and certification fields. A conventional encoding may carry both. The classification is about the supplied semantics, not the physical names or layout of registers.

A fixed machine can enforce an invariant through its prescribed transitions. A simulator preserves that invariant when its correctness contract establishes the correspondence. A defective simulator does not establish that every simulator or every conventional machine lacks enforcement. Equally, a protected internal law does not automatically authenticate a transcript presented by an external party.

Certification is one instance of this framework. A proposed family of observables $\alpha_i : S \rightarrow X_i$ needs explicit events and cost laws for each claimed guarantee. Merely collecting predicates supplies neither a topology nor a metric, nor independence among the observations. The present ISA permits zero-cost changes even to partition structure.

3.3 The canonical contrast: the Turing machine

Here is what I had to learn about Turing machines to compare what I built to one. I went and read about them on my own. I'm walking through it because I had to walk through it. If I have anything wrong below, I'm relaying what I read, so tell me.

What I read says Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine, as I understand it, is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells. Each cell holds one symbol from a finite alphabet, say $\{0, 1, \text{blank}\}$.
- A read/write head. One cell at a time. It reads the symbol, overwrites it if the table says to, then moves left or right.
- A finite set of states, $Q = \{q_0, q_1, \dots, q_n\}$. The machine is in exactly one state at a time. q_0 starts the run. Some states halt it.
- A transition table. Given the current state and the symbol under the head, it says: write this, move left or right, enter that state.

That is the entire machine. Nothing else.

Here is what I read as the remarkable thing about this simple machine: it can compute

anything that can be computed by following a definite procedure. The sources I dug up claim every algorithm that has ever been written, in every programming language that has ever existed, can be translated into a Turing machine. The label I found for this is the Church-Turing thesis. The original claim (Church 1936, Turing 1936), as I read it, is specifically about *effective calculability*: anything that can be computed by following a mechanical, finite, step-by-step procedure, a Turing machine can also compute. I did not find a counterexample. I also have not verified the thesis myself. I am reporting the ground I had to stand on before I could push against it.

The claim about physical hardware is a related but separate conjecture, called in the sources I read the *physical Church-Turing thesis*: every physical computing device is computationally equivalent to a Turing machine. As I read it, this is an empirical claim about physics, not proven mathematics. I did not find a built physical counterexample. Your laptop, your phone, every server in every data center: in the usual computability sense, they compute no more functions than a Turing machine computes. The differences I found are resource differences. Speed. Space. Energy. Cost. The class of computable functions stays the same.

3.4 The observation boundary

A Turing-machine transition reads a local symbol and control state; a computation can nevertheless derive global properties or retain prior evidence in its state. The relevant impossibility here concerns a particular projection: when two full states have equal observations and different structural properties, no function of that observation recovers the differing property universally.

A structural opcode can expose or check a property through the richer state. Its guarantee must be read from its semantics. An accumulated ledger alone records a total; locating an event in history requires the trace as well. A positive payment alone does not establish semantic truth.

3.5 Structural cost and other resource measures

This framework tracks its own event-sensitive cost schedule alongside ordinary execution. It does not claim that complexity theory studies only two resources or cannot model verification costs. The intended contribution is the particular composition of transition laws, structural state, observation interfaces, and accounting results, with the conditions stated in each theorem.

3.6 The subsumption theorems, in full

Section 1 framed classical computation as the degenerate fragment of the Thiele substrate, the corner where the structural axis lies dormant and nothing ever lights it up. That framing is four theorems in the kernel, and I am setting their statements and proof sketches down below in plain math, so you can hold the framing up against the proofs and decide for yourself, without taking my word for either one and without leaving this page to do it.

Theorem 3.1 (D2 faithfulness). *For every classical program prog (a sequence of `vm_instruction` values each satisfying `is_classical_opcode`) and every starting state s_0 , let s_n be the state after running prog on s_0 . Then (a) `shadow_proj`(s_n) extracts exactly the six projected fields of s_n (`vm_regs`, `vm_mem`, `vm_pc`, `vm_mu`, `vm_err`, `vm_certified`); and (b) the structural layer is preserved: $s_n.\text{vm_graph} = s_0.\text{vm_graph}$, $s_n.\text{vm_csrs.csr_cert_addr} = s_0.\text{vm_csrs.csr_cert_addr}$, and $s_n.\text{vm_certified} = s_0.\text{vm_certified}$.*

Proof. Part (a) unfolds `shadow_proj`: by definition it extracts exactly those six fields, and reflexivity closes it. Part (b) is D3 conservativity (next theorem) iterated over the program. $\square$ $\square$

| Plain reading. Classical programs leave the structural axis alone. The shadow you see equals the full state’s projected fields. The graph and cert channels stay at whatever they were at the start.

Theorem 3.2 (D3 conservativity). *If `is_classical_opcode(i) = true` and $s' = \text{vm_apply}(s, i)$, then $s'.\text{vm_graph} = s.\text{vm_graph}$, $s'.\text{vm_csrs.csr_cert_addr} = s.\text{vm_csrs.csr_cert_addr}$, and $s'.\text{vm_certified} = s.\text{vm_certified}$. By induction over the program list, classical programs preserve all three.*

Proof. Case analysis on the instruction i . The predicate `is_classical_opcode(i) = true` holds exactly for the compute, memory, control-flow, ordinary I/O, heap, and MORPH_GET / TENSOR_GET opcodes: the ones whose `vm_apply` rule mutates only `vm_regs`, `vm_mem`, `vm_pc`, `vm_err`, `vm_mu`, `vm_logic_acc`, `vm_mu_tensor`, or `vm_mstatus`. Twenty-two opcodes return false from `is_classical_opcode` and are excluded by the premise, but not all for the same reason. PNEW, PSPLIT, PMERGE, TENSOR_SET, MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, and MORPH_TENSOR mutate graph or module-tensor state. MORPH_ASSERT can write the cert-address channel on success; CERTIFY writes the top-level certification flag; CHSH_TRIAL mutates the witness counters. LASSERT, LJOIN, EMIT, REVEAL, PDISCOVER, CHSH_LASSERT, and the four Q_{1+AB} CHSH_LASSERT variants are also outside the classical fragment because they carry validation, revelation, discovery, tensor-charge, or mandatory-floor semantics. The D3 proof only needs the forward direction: for each opcode that `is_classical_opcode` admits, direct inspection of the apply rule leaves `vm_graph`, `csr_cert_addr`, and `vm_certified` fixed. Iterating over the program list by induction closes the trace-level preservation. $\square$ $\square$

| Plain reading. The classical opcodes are defined by what they don’t touch. They may change the projected working fields, including the ledger charge assigned to the instruction, but they leave the graph and certification channels alone. A program built only out of them cannot disturb those channels, even if you wanted it to. That dormancy is by construction, not by accident.

Theorem 3.3 (Thiele simulates Turing). *Lift a Turing configuration by keeping the TM part and setting μ to zero: $\text{lift_config}(c) := \{ \text{th_tm_config} = c; \text{th_mu} = 0 \}$. Then for every fuel $n \in \mathbb{N}$, transition table δ , and TM configuration c :*

$$(\text{thiele_run } n \delta (\text{lift_config } c)).\text{th_tm_config} = \text{tm_run } n \delta c.$$

Proof. Induction on the fuel n .

Base ($n = 0$). Both $\text{thiele_run } 0$ and $\text{tm_run } 0$ return their inputs unchanged. Reflexivity.

Step ($n + 1$). Unfold one step on both sides. Let $c' = \text{tm_step } \delta c$ when defined. On a lifted configuration, thiele_step 's TM portion is exactly $\text{tm_step } \delta c$: structural fields are not consulted, and lifted μ stays at 0 because TM steps are classical. Apply the induction hypothesis to the next n steps from $\text{lift_config}(c')$. $\square \quad \square$

| Plain reading. Take any Turing-machine configuration and drop it into the Thiele substrate untouched: the TM part stays exactly as it was, $\mu = 0$, the structural fields empty. Now run thiele_run on it. Nothing happens that wasn't going to happen anyway. Step for step the TM portion does precisely what the standalone machine would have done, and because every move it makes is a classical move, the ledger never comes off zero. The induction on fuel just carries that out to the whole trace. The Turing machine is sitting inside the larger object running its same computation on its same tape to its same answer, having paid nothing, and from where it stands there is no way to tell it has been subsumed at all. That is what subsumption is supposed to look like: the old thing keeps working, identically, for free.

Theorem 3.4 (Degenerate projection theorem (four-part)). *The following four propositions hold simultaneously:*

1. Faithful embedding. *For every fuel n , transition table δ , and TM configuration c :*

$$(\text{thiele_run } n \delta (\text{lift_config } c)).\text{th_tm_config} = \text{tm_run } n \delta c.$$

2. shadow_proj is the quotient map. *For all Thiele states s_1, s_2 :*

$$\begin{aligned} \text{shadow_proj}(s_1) = \text{shadow_proj}(s_2) \\ \iff \text{eq_on_classical_shadow}(s_1, s_2). \end{aligned}$$

3. Classical traces respect the compatibility relation. *For every classical program prog and every two Thiele states s_1, s_2 with $\text{shadow_proj}(s_1) = \text{shadow_proj}(s_2)$ and*

$$s_1.\text{vm_graph} = s_2.\text{vm_graph} \text{ and } s_1.\text{vm_csrs} = s_2.\text{vm_csrs}:$$

$$\text{shadow_proj}(\text{run}(\text{prog}, s_1)) = \text{shadow_proj}(\text{run}(\text{prog}, s_2)).$$

Shadow agreement alone is not a strong enough hypothesis. `TENSOR_GET` and `MORPH_GET`, both classical opcodes, read `vm_graph` into a visible register; `HEAP_LOAD` and `HEAP_STORE` read `csr_heap_base` out of `vm_csrs`. Two states can agree on all six projected fields and still disagree on the graph or the heap base, and either instruction then writes a different value into a projected register, breaking the shadow-only version of this claim. The extra agreement on `vm_graph` and `vm_csrs` closes that gap.

4. The projection is strictly lossy. *There exist Thiele states s_1, s_2 with $\text{shadow_proj}(s_1) = \text{shadow_proj}(s_2)$ and $s_1 \neq s_2$.*

Proof. The theorem is four lemmas stitched into one conjunction.

- Part (1) is `thiele_simulates_turing` (preceding theorem).
- Part (2) is definitional. `shadow_proj` extracts its six projected fields. Equal projections mean those six fields agree. The Coq witness is `shadow_proj_kernel_is_eq_on_classical_shadow`.
- Part (3) follows from D2 and D3. Run a classical program: the output shadow depends only on the input shadow. Equal shadows in, equal shadows out. The Coq witness is `D2_classical_shadow_preserved`.
- Part (4) is `shadow_strictly_lossy`. The Categorical Separation construction gives the witness: s_1 has one identity morphism in `pg_morphisms`; s_2 has none. The six projected fields agree. The morphism lists do not.

The conjunction of the four pieces is the theorem. □ □

| Plain reading. Four parts about the same projection. Part 1: every TM configuration lifts faithfully into the Thiele substrate; same computation, with $\mu = 0$ and no structural content. Part 2: the projection from Thiele back to classical IS the classical-equivalence quotient; two Thiele states have the same shadow exactly when they agree on the six projected fields. Part 3: classical programs commute with the projection; you can project before running or after, you get the same answer. Part 4: the projection genuinely loses information; there are Thiele states that look identical from the classical side but differ on the structural axis. Put together, classical computation is exactly the image of Thiele computation under the structural-axis projection. The projection drops real information.

3.7 What the projection cannot reverse: three non-existence theorems

The subsumption theorems above establish what the Thiele substrate carries over and above its classical projection: fields the projection drops. The natural next question is whether a classical observer could reconstruct those fields by some clever derived predicate on the surviving shadow. The answer is no, and it is a structural no, the kind you cannot argue your way around. The kernel proves three irrecoverability theorems (each one a non-existence of a function from the bare classical projection back to a Thiele-only field) and packages the corollaries the subsumption argument relies on. Every one is proved with zero axioms and no admits, fully constructively.

The projection in this subsection is the four-field map `forget : VMState → TMSnapshot`, keeping `(pc,  $\mu$ , regs, mem)`. It is already forgetful, but it still retains μ . For the cost-ledger separation, the stricter three-field projection `bare_observable` drops μ as well, keeping only `(pc, regs, mem)`: the observable the standard complexity-theory cost model sees.

This document uses several distinct execution models and several distinct projections, not one model viewed from different angles. Table 3.7 is the registry: which state type each one runs over, what counts as one step, what an observer sees, and which theorems are stated over it. Consult it whenever a later section says “run,” “classical shadow,” or “simulation”; the domain is not always the same one.

Model	State type	Execution function	Observation	Key theorem(s)
File-local TM lift	th_tm_config	thiele_run/tm_run, fuel-indexed TM step	TM configuration, th_mu = 0	thiele_simulates_turing
VM instruction-list fold	VMState	run, folds a fixed instruction list once, in order	full VMState	D2/D3, degenerate projection theorem
PC-indexed runner	VMState	vm_apply, single-step transition driven by vm_pc	full VMState	μ -conservation, instruction_cost agreement
Bounded evaluator	VMState	vm_run, fixed 1000-step fuel, always returns Some	full VMState, fuel-truncated	structural-axis undecidability, VM instance (conditional, Section 19)
Hardware retirement model	Kami/RTL state	WFDrivenRun, clock-driven, multi-cycle per instruction	full VMState at instruction retirement	rtl_contract_trace_agreement
bare_observable	3-tuple	(a projection, not an execution function)	(pc, regs, mem)	mu_not_function_of_bare_observable
forget	4-tuple	(a projection)	adds μ to the above	cert_not_function_of_forget
shadow_proj	6-tuple	(a projection)	adds vm_err, vm_certified; still drops the graph	degenerate projection theorem, shadow_strictly_lossy

Table 3.7: model registry. The file-local TM lift is a proof device local to `thiele_simulates_turing`; it is not a claim that the bounded evaluator implements arbitrary Turing machines on the 51-opcode hardware. Connect two rows by an explicit correspondence theorem only where a claim in this document actually depends on that connection; otherwise treat them as different objects answering different questions.

Theorem 3.5 (Cert irrecoverability, `cert_not_function_of_forget`). *There is no function $\phi : \text{TMSnapshot} \rightarrow \text{bool}$ such that, for every Thiele state s ,*

$$\phi(\text{forget } s) = s.\text{vm_certified}.$$

Proof. The witnesses are `trace_witness_A` (certified, defined) and `forget_witn`

`ess_uncert` (not certified, defined). Direct computation shows the two states have equal `forget` images. Suppose ϕ existed: then $\phi(\text{forget trace_witness_A}) = \text{true}$ and $\phi(\text{forget forget_witness_uncert}) = \text{false}$. But the inputs are equal, forcing the outputs to be equal too. Contradiction. $\square$ $\square$

| Plain reading. The Thiele `vm_certified` flag is not a function of the four fields kept by `forget`. Two Thiele states can agree on $(pc, \mu, \text{regs}, \text{mem})$ and still differ on whether they are certified. No formula computed from that four-tuple alone can decide certification. The certification bit lives outside the four-tuple.

Theorem 3.6 (Mu irrecoverability from bare observable, `mu_not_function_of_bare_observable`). *There is no function $\phi : \text{BareTMObservable} \rightarrow \text{nat}$ such that, for every Thiele state s ,*

$$\phi(\text{bare_observables } s) = s.\text{vm_mu}.$$

Proof. The witnesses `mu_separation_witness_zero` and `mu_separation_witness_one` (defined) differ only in their μ field (0 vs 1) and have identical bare observables. A ϕ satisfying the equation would force $0 = 1$. Contradiction. $\square$ $\square$

| Plain reading. The Thiele μ counter is not a function of the three-field classical observable $(pc, \text{regs}, \text{mem})$. Two Thiele states can agree on the classical observable and still have different μ values. The cost-ledger separation between Thiele states that the classical observable cannot tell apart is not a derived fact that a clever classical observer could compute; it is a state distinction the classical observer's signature is structurally blind to.

Theorem 3.7 (Cert-addr irrecoverability, `cert_addr_not_function_of_forget`). *There is no function $\phi : \text{TMSnapshot} \rightarrow \text{nat}$ such that, for every Thiele state s ,*

$$\phi(\text{forget } s) = s.\text{vm_csrs.csr_cert_addr}.$$

Proof. The witnesses `cert_addr_witness_zero` and `cert_addr_witness_one` (defined) differ only in their `csr_cert_addr` field (0 vs 1) and have identical `forget` images. A ϕ satisfying the equation would force $0 = 1$. Contradiction. $\square$ $\square$

| Plain reading. The `csr_cert_addr` channel cannot be recovered from the four fields kept by `forget`. This is a state-observation theorem. The conditional program-level diagonal in Section 19 uses a separate outcome-equality predicate; transferring this collision to that predicate would require an additional correspondence theorem.

The three subsumption-witness corollaries.

The irrecoverability theorems above ground three statements the subsumption argument relies on. These are the corollaries the monograph cites as “the Thiele content the bare projection structurally cannot carry.”

1. *A2 cannot be stated as a constraint on the bare projection alone.* A2 (`cs_cert_costs`, a typed field on `CertificationSystem`) says that a cert-predicate flip from false to true forces instruction cost ≥ 1 . By cert irrecoverability, any cert-predicate phrased as a function of the four-field `TMSnapshot` cannot agree with Thiele’s `vm_certified` on every Thiele state. So A2 on the bare classical signature is not Thiele’s A2. It constrains a different predicate. The Coq corollary is `no_classical_a2_cert_predicate`.
2. *The cost-ledger observability separation has no analog in the bare projection.* The separation above names two Thiele states with identical $(pc, regs, mem)$ but different μ : the witnesses `mu_separation_witness_zero` and `mu_separation_witness_one` from the μ -irrecoverability theorem. As a predicate on the bare classical observable, the separation would have to read μ from an observable that does not contain it. μ -irrecoverability forbids that. The separation lives at the Thiele level by structural necessity, not by drafting choice. The Coq corollary is `no_classical_mu_separation_predicate`.
3. *The CSR channel does not factor through the named projection.* `no_classical_cert_addr_predicate` excludes recovery of the specified CSR field from four-field `forget`. It does not identify this state predicate with the program-level diagonal or exclude conventional encodings that retain additional information.

Recovery and sections of a projection.

`fiber_has_two_preimages` exhibits multiple full states above a projected state. A left inverse recovering every original full state is impossible. A section that chooses a default graph, ledger, and flag may still exist. Noninjectivity alone does not formalize or refute every notion of a canonical choice.

Start with the question the observer promises to answer. If two states look identical to that observer but require different answers, no decoder of that observation can answer correctly on both. `decoding_requires_fiber_constancy` proves that for arbitrary state, observation, and answer types. Give me an explicit representative of each observation class and the converse holds too: if the answer is constant on the class, I can compute it on the representative. No certification bit is needed to state either result. This is the precise sense in which a distinction becomes necessary once a model promises to answer a question about it.

The trace-fold universal property.

`thiele_trace_fold_initial` states that, for a `CertCostMachine` M and base-point s_0^M , the fold of its step function is the unique function from instruction lists that

preserves the basepoint and commutes with instruction extension. Its domain is traces; the proof uses list induction. `thiele_is_initial_a2_substrate` is an alias for the same trace-fold theorem.

`thiele_morphism_unique_on_reachable` proves that two supplied certification-preserving VM-state maps agreeing at the initial state agree on reachable states. It does not establish their existence. A one-state target whose predicate is always false satisfies A2 but admits no such map from certified VM states. More generally, descent from traces requires compatibility with equal VM outcomes and with the certification predicate.

The state-map question has an exact condition too. If two instruction lists reach the same VM state, they must evaluate to the same target state before trace evaluation can descend to a state map. `trace_descent_unique_value_iff` proves equivalence with a unique target value at every reachable VM state. An executable construction additionally takes a representative trace for each reachable state. Under that explicit selector, `reachable_simulation_exists_iff` characterizes certification-preserving reachable simulations by trace compatibility and certification agreement. The Boolean certification quotient supplies a concrete target that works.

And there is a target that fails for a more informative reason than never certifying. It retains the full instruction history, charges the VM schedule, and reads certification using the VM evaluator. Certification agrees exactly. But the empty trace and `JUMP 0 0` reach the same VM state and leave different histories. `certification_agreement_does_not_imply_descent` proves the obstruction. A2 and certification agreement do not force a target to forget those histories.

Neither this universal property nor the projection theorem selects the event to price or establishes physical foundationality.

3.8 What the projection cannot verify: the verifier corollary

The non-existence theorems above proved the projection is structurally lossy: μ , `vm_certified`, and `csr_cert_addr` are not functions of the bare classical observable. The natural next move is to ask what that lossiness means for verification. If a verifier sees only the projection (a list of `StrictClassicalState` snapshots, the Turing-classical trace), what can it soundly decide?

Answer: not the claims that depend on the dropped fields.

Theorem 3.8 (Bare-setting verifier impossibility, `bare_setting_no_sound_complete_verifier`). *There is no verifier $V : \text{list } \text{StrictClassicalState} \rightarrow \text{bool}$ that is simultaneously sound and complete on the claim “ $\text{vm_}\mu s = 1$.”*

Proof. The two single-step witness states from the kernel’s receipt theorem (`ReceiptTheorem`: no function of the strict three-field shadow recovers μ ; its witnesses are

`pol_state_A, cost-1 CERTIFY 0`, and `pol_state_B, cost-0 PNEW [] 0`) project to the same strict-shadow trace. State *A* satisfies the claim; state *B* does not. A complete verifier must accept the honest run from *A*. On the same accepted transcript, soundness forces the claim to hold for every state that could explain the transcript, including *B*. The first requirement gives acceptance; the second contradicts *B*'s $\mu = 0$. The Coq check assumes no axioms. $\square$ $\square$

| Plain reading. A verifier that sees only the classical projection of a Thiele execution cannot soundly decide claims about μ . Two different Thiele executions project to identical classical traces and disagree on whether $\mu = 1$. The verifier has nothing to distinguish them by. Soundness and completeness on μ -sensitive claims are mutually exclusive under the bare projection. This is the receipt theorem lifted from “no function recovers μ from the projection” to “no verifier soundly decides μ -sensitive claims from the projection.”

Three escapes, each constructed.

Three structurally distinct ways to clear the impossibility live in the kernel as concrete verifiers, each one a closed Coq theorem.

1. *Substrate-trust.* The transcript carries the full `VMState` and the verifier reads μ directly. `substrate_escape_succeeds` exhibits a verifier with constant cost that is both sound and complete on `vm_mu s = 1`. The verifier's decision function is `Nat.eqb (last t).vm_mu 1`; the proof is one inhabitation per quantifier.
2. *Hardness.* The transcript carries an unforgeable commitment; the verifier accepts the commitment under a hardness hypothesis. `hardness_escape_succeeds` exhibits a constant-cost weak-sound verifier conditional on a `HardnessHypothesis` record. The record is a parameter the verifier is constructed over, not an axiom; whether the hypothesis is realised in nature is a question for cryptography, not this kernel.
3. *Interaction.* The verifier elicits a response from the prover that pins the claim. `interactive_escape_succeeds` exhibits a constant-cost sound and complete verifier when the response set is rich enough to report μ . This is the simplest possible interactive model (one round, one fixed query), and that is all the corollary needs.

The substrate channel is the option the structural axis makes available; hardness and interaction are the two routes classical cryptography and complexity already used. All three are witnessed by closed Coq theorems, each with its own trust profile. What follows is the shared obstruction they all answer to.

Theorem 3.9 (Factorisation impossibility, `V_does_not_factor_through`)

`_classical`). Let T be a transcript type with a projection $\pi : T \rightarrow \text{list StrictClassicalState}$ and a relation $\text{explains}_T : \text{VMState} \rightarrow T \rightarrow \text{Prop}$ connecting VM states to the transcripts that could have produced them. Suppose the bare-setting witness pair (states s_A, s_B with identical bare projections, $\text{vm_mu } s_A = 1$, $\text{vm_mu } s_B = 0$) lifts to T : there exist $t_A, t_B : T$ with $\pi(t_A) = \pi(t_B)$, $\text{explains}_T s_A t_A$, and $\text{explains}_T s_B t_B$. Then no verifier $V : T \rightarrow \text{bool}$ that is sound (every transcript V accepts explains only $\text{vm_mu} = 1$ states) and complete (every transcript explaining a $\text{vm_mu} = 1$ state is accepted) factors through π .

Proof. Suppose V factored through π , i.e. $V = V' \circ \pi$ for some $V' : \text{list StrictClassicalState} \rightarrow \text{bool}$. Since $\pi(t_A) = \pi(t_B)$, $V(t_A) = V(t_B)$. Completeness applied to t_A (which explains s_A , $\text{vm_mu} = 1$) forces $V(t_A) = \text{true}$. Soundness applied to t_B (which explains s_B , $\text{vm_mu} = 0$) forces $V(t_B) = \text{false}$. Contradiction. $\square$ $\square$

| Plain reading. The theorem is conditional on the collision lifting to T : the transcript type has to be rich enough that the two bare-indistinguishable states s_A, s_B still show up as two distinguishable transcripts t_A, t_B under explains_T . That is not a free assumption; it is exactly the fact each of the three escapes below has to establish about its own transcript type, and each one does so by exhibiting t_A and t_B directly rather than by positing that they exist. Given the lift, verification of μ -sensitive claims must access non-classical structure of the transcript: any verifier that pretends to live entirely on the classical projection fails on that same lifted pair. The substrate, the hardness commitment, and the interactive response are three constructed instances of a verifier that does read that structure. Whether some fourth transcript design could expose it a different way is not something this theorem or the surrounding development rules out; the claim is that the three shown here work, each under its own stated trust contract, not that they exhaust every possible design.

4 What structure is

Everyone uses the word “structure,” and the word does the work while meaning nothing in particular, which you get away with right up until you try to build a machine whose entire job is to charge for it. Then the fog has a price. So I am going to pin the word down, further down than feels necessary, because every time I have watched this argument slip, it slipped at a step earlier than anyone was looking.

4.1 Partitions

The smallest piece of structure there is, is a partition. You take everything in front of you and sort it into buckets: nothing in two buckets, nothing left on the floor. That is the whole of the definition, and yes, it is the thing you do with a kindergartner and a pile of blocks. Keep it anyway. The instant the buckets are drawn you have made a claim about the world, and the claim turns out to cost money.

Formally, a partition of a set S is a collection of non-empty pieces $S_1, S_2, \dots, S_k$ that share no elements and together cover all of S , so every element lands in exactly one piece. I write that $S_1 \cup S_2 \cup \dots \cup S_k = S$, with $\cup$ the set union from Section 2. Each piece I call a module, or a block, or a cell, whichever word the surrounding sentence wants.

The runtime partition graph stores module records and permits empty regions through PNEW. Its local region canonicalization does not establish a global disjoint covering partition. Claims about global addresses, nonoverlap, or locality require their own well-formedness and representation conditions. The finite hardware partition wall is a separate enforcement mechanism. In the assembled PNEW format, operand A carries the first listed address and operand B carries the list length. The hardware partition table stores operand B and enforces the local range $[0, \text{length})$; this does not preserve the supplied global addresses or prove a duplicate-free region contract.

Here is the part that is not kindergarten. The partition is not a tidier way to hold information you already had. The partition is itself information, a fact that did not exist until you drew it. Put x in S_1 and y in S_2 and you now know, for nothing and for keeps, that the two live in different groups. Before, you would have had to go and look. After, the answer is a lookup, and the lookup does not cost one cent more when S swells from ten elements to a billion. You bought a question's answer down from a search to a glance. Something paid for that. Somewhere a bill came due, whether or not anyone troubled to write it on a receipt.

Not writing it on a receipt is the classical world's whole way of life. Carve up the memory of an ordinary machine, announce "these addresses are module A , those are module B , and A and B never touch," and you have said something with teeth, something a later step can lean its full weight on and run a hundred times faster for. A Turing machine will happily hold that announcement. It writes the table onto its tape the way it writes anything, as bytes. What it cannot do is hold it as a fact the machine itself is answerable for. Nowhere in the machine is there a place that says this partition is real and somebody paid to build it; the table just sits there, asserted, and the step rule has no opinion about whether it was earned or invented. The Thiele substrate keeps that place. It carries the partition as part of its state, posts every build to the ledger at its declared cost, and will not stamp the structure certified while the bill for the check is still open: the step that stands behind it pays at least one, and there is no other door. That gap, between a structure you merely wrote down and a structure the machine is on the hook for, is the whole reason this section exists, and a good part of the reason the rest of the document does.

4.2 Axioms carried by modules

A partition by itself is just a set of labels, and labels are cheap because labels promise nothing. Calling something module A commits you to exactly zero about what is inside it. Structure that earns the name starts one step later, when you hang a constraint

on a module and make it answer for something.

Two pieces of vocabulary, so I can say that cleanly.

A **predicate** is a yes-or-no question you can put to a thing and get a definite answer back. “ $x > 5$ ” is a predicate about a number. “This memory region holds a sorted list” is a predicate about a region of memory. A predicate is the shape a claim takes once it is sharp enough to be checked instead of merely believed.

A **logical formula** over Boolean (true/false) variables is those variables wired together with AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). Take $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$: it comes out true exactly when at least one of x_1, x_2 holds and, at the same time, at least one of $\neg x_1, x_3$ holds. A choice of true or false for every variable that makes the whole thing come out true is a satisfying assignment; one that makes it come out false does not satisfy it. That is the entire machinery, and it carries everything built on top of it.

Now hang one on a module. “Module A ’s region satisfies the predicate Φ_A .” The capital Greek Φ_A is nothing but a name I am pinning on the predicate I want to attach; mathematicians reach for Greek out of habit and there is no magic in the ink. Φ_A can carry real content: every value in this region sits between 0 and 100, or this region encodes a satisfying assignment to the formula living in module B , or this region is sorted top to bottom.

The module record stores strings called axioms. Storage alone does not prove their meaning or connect them to an interpreted predicate over memory. Such a connection requires an explicit representation and sound checker. The guarded formula instructions provide specific checking behavior; generic module metadata and generic CERTIFY do not establish arbitrary propositions.

I will come back to one concrete encoding in Section 10. In the 51-opcode realisation, LASSERT reads a formula from memory, checks it against two witnesses (one satisfying assignment, one falsifying assignment showing the formula is not a tautology), and either accepts or traps. The hardware checker is the Logic Engine. It is a finite-state controller in the chip: a dedicated circuit that walks the formula word by word and verifies the witnesses without calling an external constraint solver. If validation succeeds, the checked constraint package can feed the later certification path. If validation fails, LASSERT traps (Section 2: routes execution to a fixed error address) and sets an error flag. The μ cost is charged either way.

A trusted execution trace can identify the step at which a particular check ran and the amount charged. A ledger value alone does not identify the checked statement or its position in the trace. Authenticity of an externally supplied history requires its own evidence interface.

4.3 Structural gain

A trace has *structural gain* when some structural fact is uncertified at the door and certified by the time the trace stops. The plainest case: you open holding a partition nobody has vouched for, and you close holding a checked, certified satisfying-assignment witness. Something structural walked into the computation that was not there when it started.

The shallow question is whether the gain happened, and you can settle that by holding the first state next to the last and looking. The substrate is not built for the shallow question. It is built for *where*: which single step in the run is the one that turned belief into certified knowledge. Pin the trace to the board, walk it end to end, and find the step where the fact crossed from uncertified to certified.

There is always such a step, and it always cost something: a step with $\mu \geq 1$, never a free one, and it can be nowhere else. That is No Free Insight, said plainly before I have earned it on paper. The gain has a location, the location has a price, and the price is never zero.

5 Structural entitlement

It would be easy to read this whole project as one sentence, certification has a price, and stop at the doorway. That sentence is true. It is also the first cost law and the shallowest thing in here, the part you can see without going in. The room behind it is bigger, and this section is about what is in the room.

The deeper object is *structural entitlement*. A computation has it when the state contains not just data, but a structural fact later steps are allowed to use as already established. I call that fact *admissible*. Admissible means the trace earned the right: a cost-positive step paid for it, and the proof object establishing it is on hand. Admissible is not just “true.” Admissible is “tracked and paid for.”

Several kinds of object can be admissible in this sense: a partition (Section 4), a decomposition (a split of a problem into independent parts), a morphism (Section 11), a constraint (Section 4), a witness (a value that proves the existence of something), or a narrowed feasible set (defined below). The gap between admissible fact and raw data matters. A raw bit pattern can be copied. A certified decomposition can justify solving two smaller problems instead of one large one. At the raw-data level, a morphism ID can be forged in a register: write 42 into the register and call it “the ID of some morphism.” An asserted morphism in the graph has provenance (Section 2). In the 51-opcode realisation, the trace contains the MORPH and COMPOSE steps that built the morphism, and MORPH_ASSERT then committed it. A formula sitting in memory is text. A checked formula with a satisfying assignment and a falsifying assignment is admissible narrowing: the two witnesses prove the formula is satisfiable and not a tautology, which rules out at least one possible state.

Definition 5.1 (Structural entitlement). A state has structural entitlement to a claim P when three things are present:

- a structural object carried by the state, such as a partition, morphism, witness counter, or formula package;
- a checked relation between that object and the claim P ;
- a trace-visible certification or assertion event that makes later use of P admissible.

Without the third part, the object may exist as raw data. The computation still has not earned the right to use it as certified structure.

Here is how the pieces connect.

5.1 The precise vocabulary

Every term below is an exact Coq definition, not a metaphor. I ran every theorem that follows through Coq, so I won't stamp "machine-checked" on each one; assume I did. That isn't what makes them true. It's how I caught the places I'd talked myself into something.

Feasible set. A feasible set Ω is the list of substrate states still on the table: everything the trace has not yet ruled out, everything still consistent with what it has checked and certified up to this point. In the kernel it is exactly that, a `list VMState`, and the literalness is the whole point. This is not "the space of possible worlds" or any other phrase that lets the possibilities stay vague and uncounted while sounding profound. It is a finite list of concrete states you could print to a page. Early in a run, before the computation has certified anything, the list is long, because almost nothing has been excluded yet. Each time a constraint gets certified, the list gets shorter, because the states that violate that constraint are no longer live and they come off. Learning, on this substrate, is the list getting shorter, and it is concrete enough that you can stand over it and watch the states drop.

Prior and posterior. Two snapshots of that list, before and after one observation or certification event. The *prior* Ω is the list as it stood going in; the *posterior* Ω' is the list coming out. The posterior can only ever be a subset of the prior, $\Omega' \subseteq \Omega$. You can rule a state out and you can never rule one back in, because ruling-in would mean un-learning something the trace already paid to establish, and nothing in the substrate runs that way. When the posterior comes out strictly smaller, $|\Omega'| < |\Omega|$, the event genuinely cut the field: states that were live going in are dead coming out, killed by a structural claim the trace certified, a claim those exact states fail. Whatever is still standing afterward is the posterior.

Distinguishing observation. A distinguishing observation is a function that separates

an eliminated state from the posterior. Formally: there exists at least one eliminated state $s_w \in \Omega \setminus \Omega'$ such that the observation function outputs a value on s_w that no state in Ω' produces. The observation is the “test” that reveals the structure. No named test, no exclusion: the posterior only genuinely rules out something the prior allowed once you can point at the test that does the ruling-out. Name it and the whole thing becomes checkable by anyone, including someone who thinks I am wrong. That is the entire appeal.

Strictly stronger predicate. The word “stronger” here runs backward from how it sounds. In ordinary talk the stronger claim is the bolder one, the one that says more; in logic the stronger predicate is the one that lets fewer things through. P_1 is strictly stronger than P_2 when it accepts a strict subset of what P_2 accepts. Formally in Coq: $P_1 \leq P_2$ means whenever P_1 is true, P_2 is true too, and strictness means there exists an observation where P_2 is true but P_1 is false. So P_1 is the harder claim to satisfy, not the louder one: it rules more states out, and everything that clears its bar clears the lower one for free. If P_2 is “the state is in Ω ” and P_1 is “the state is in $\Omega' \subsetneq \Omega$ ”, then P_1 is strictly stronger, anything in Ω' is in Ω , and not the other way around. The inversion is worth holding onto. Loose talk reaches for “stronger” to mean grander, and here it means smaller, and the whole vocabulary of narrowing only works once you take it the logician’s way round.

Structure-addition event. In the 51-opcode realisation, a structure-addition event is a single execution step where the assertion register `csr_cert_addr` transitions from 0 to nonzero. Precisely: `s.vm_csrs.csr_cert_addr = 0` before the step and `(vm_apply s i).vm_csrs.csr_cert_addr  $\neq$  0` after. This happens only through a successful MORPH_ASSERT whose property string carries a non-NUL byte, which stores that property’s nonzero ASCII checksum in `csr_cert_addr`. Not “structural knowledge was acquired” in fog. This field flipped from zero, at this step, because MORPH_ASSERT asserted such a property against an existing morphism.

Supra-certification (`has_supra_cert`). In the 51-opcode realisation, supra-certification is reached when `csr_cert_addr  $\neq$  0`. The name distinguishes it from `vm_certified`, the top-level flag set by CERTIFY. Supra-certification tracks the MORPH_ASSERT channel: has a morphism property been asserted and its checksum registered? A trace can set `vm_certified = true` without `has_supra_cert` by using CERTIFY and not MORPH_ASSERT; CERTIFY never touches `csr_cert_addr`. Both are positive-cost certification channels. They record different things. Two channels, two costs, one substrate.

Decision tree. A decision tree, in the kernel, is the bare branching skeleton: a leaf, or a node with a left subtree and a right subtree, and nothing written on it. What gets audited is its shape, the depth and the leaf count; the semantic bookkeeping, which observation was made and which states survived it, lives in the observation function

and the posterior-representative reduction standing next to the tree, not inside the tree object. The tree is a proof object, not a runtime structure: it exists so the move from prior to posterior answers to a definite branching shape, with leaves enough to index the posterior and a depth the trace has to pay for, rather than a magic jump. “Realized by the trace” means exactly that payment: the trace fired at least as many cert-flips as the tree is deep. A shortcut isn’t sound on its word: it’s sound once that shape, and a trace that paid for its depth, are on the table, and not one second before.

Posterior-representative reduction. Given a prior state $s \in \Omega$, a posterior-representative reduction provides a function mapping s to a representative $s' \in \Omega'$. This proves that every prior state corresponds to something in the posterior. The purpose is accounting: the posterior was not reached by arbitrary deletion, but by a coherent map where each prior state is represented. Without this, the narrowing can be fake. Delete states, announce a small posterior, and provide no principled connection to the prior.

Theorem 5.1 (Classical blindness as quotient). *The four-field projection keeps only (PC, μ , registers, memory). It is many-to-one: different Thiele substrate states can land on the same image. Section 2 calls that a quotient. The projection identifies states that agree on the kept fields, even if they differ in certification, witness counters, or morphism graph. The richer six-field `shadow_proj` keeps `vm_certified`, but still forgets the morphism graph. No function of either projection alone can separate the morphism-witness states.*

Proof. Section 12 gives the witness pair. s_1 carries `pg_morphisms` = $[(0, \text{id})]$. s_2 carries `pg_morphisms` = $[]$. Everything the projections keep is identical: `vm_regs` = $[]$, `vm_mem` = $[]$, `vm_pc` = 0, `vm_mu` = 0, `vm_err` = false, `vm_certified` = false. The four-field projection and six-field `shadow_proj` both map them to the same image. They are different states with equal projections. Any classifier using only the projection-image must agree on equal images, so it agrees on s_1 and s_2 . $\square$ $\square$

| Plain reading. Same witness as Categorical Separation, read backward. Two Thiele substrate states have identical projections under any projection that drops the morphism graph. They are still different states. So the projection is provably many-to-one. A function defined only on the projection output can’t tell them apart. The projection already collapsed them.

If your state model only records registers, memory, PC, an error bit, and maybe μ , you have already thrown away part of what matters. You have collapsed states that differ on the inside into a single raw state that looks the same from the outside. That collapse is a theorem about a projection map.

Theorem 5.2 (μ -ledger necessity, full pairwise independence). *For two state components X and Y , I will write $X \perp Y$ to mean: knowing Y does not determine X on*

every state. I am redefining $\perp$ locally. Elsewhere it can mean orthogonality; here it means independence. Concretely, there is a witness pair: two substrate states with the same Y and different X . For the μ results the construction is parametric, working from every reachable starting state, not one lucky start; for the certified results it runs from any uncertified state, the clean start included; for the graph result the pair is built directly. Five independence results hold at once, and vm_graph is held fixed on the right-hand side of every one of them, not only in result (5): (1) $\mu \perp (mem, regs, pc, vm_graph)$; (2) $certified \perp (mem, regs, pc, vm_graph)$; (3) $certified \perp (mem, regs, pc, \mu, vm_graph)$; (4) $\mu \perp (mem, regs, pc, certified, vm_graph)$; (5) $vm_graph \perp (mem, regs, pc, \mu, certified)$. This is the full pairwise claim: each of the three structural components, μ , $vm_certified$, vm_graph , fails to be a function of the classical projected fields together with both of the other two, not just of the classical fields alone. Results (1) and (2) sharpen further once (4) and (3) tie down the remaining structural coordinate; read together, no structural component is ever recoverable from the classical fields plus the other two structural components, in any of the three positions.

Proof. Each claim is a witness pair: two reachable Thiele substrate states agree on the named projection-tuple and differ on the named coordinate.

(5) $vm_graph \perp (mem, regs, pc, \mu, certified)$. The witness is Categorical Separation: same projected fields, different graph. s_1 carries $pg_morphisms = [(0, id)]$; s_2 carries $pg_morphisms = []$. The pair is written down directly, field for field, because the claim needs nothing more: “not a function of the projection” is settled by any two states the projection cannot tell apart, however they came to be.

(1)-(2) $\mu, certified \perp (mem, regs, pc, vm_graph)$. Same visible state and same graph, different ledger and cert flag. One branch runs `CERTIFY 0`; the other runs `JUMP 1 0` rather than `PNEW`: `jump_state` passes vm_graph , vm_regs , and vm_mem straight through unchanged and only moves vm_pc and μ , where `PNEW` would have added a partition and moved vm_graph along with it. Registers, memory, pc , and graph all match between the two branches. μ does not: 1 versus 0; nor does $certified$: true versus false. The Coq witnesses are `abs_A_strict` and `abs_B_strict_g`, and the record type carrying all four held-fixed fields is `FullStrictShadow`.

(3) $certified \perp (mem, regs, pc, \mu, vm_graph)$. Case (2) with the ledger and the graph both tied down: `CERTIFY 2` against `JUMP 1 3`. Both charge $\mu = 3$, both leave the visible tuple and the graph identical, and they still disagree on $vm_certified$. The Coq witnesses are `abs_A_cost` and `abs_B_cost_g`, over `FullCostShadow`.

(4) $\mu \perp (mem, regs, pc, certified, vm_graph)$. Same visible tuple, same certification flag, same graph, different ledger: `CERTIFY 0` against `CERTIFY 1`. `CERTIFY` never touches vm_graph , so this pair holds the graph fixed without needing to change instruction; only the declared costs differ, so μ comes out 1 versus 2. The Coq witnesses

are `abs_A_cert` and `abs_B_cert`, over `FullCertShadow`.

The parametric rider belongs to the μ results, and there it is not decoration: from any reachable state the certify-versus-jump split is still available, so no prefix, however long, prevents the μ witness pair. The certified results run from any uncertified state, and that is as far as they can go, for the honest reason that certification is permanent: once a prefix has certified, nothing clears the flag, so both branches of any extension agree on it. The theorem's content never needed more; one witness pair is exactly what “not a function of” takes, and swapping `PNEW` for a graph-inert instruction of matching cost is what lets the same style of witness carry the graph-fixed hypothesis too, closing the gap an earlier pass of this document left open. $\square$ $\square$

| Plain reading. Five independence claims, one witness pair per claim, the μ ones parametric in the starting state. The headline case (5) is the categorical-separation pair: same five projected fields, different morphism graphs. The other four are pairs of states reached by one-step programs from the clean start that match on the named tuple, graph included, but disagree on the named coordinate. The trick behind (1)-(3) is swapping the earlier `PNEW` branch for a cost-matched `JUMP: PNEW` touches the graph, so a `PNEW`-based witness could only ever show independence from the classical fields; `JUMP` touches nothing but `pc` and μ , so the same shape of witness now shows independence from the graph as well. Three structural axes, five witnesses, and this time all three axes come out independent of each other, not just independent of the classical fields they sit beside.

The witness states do the work here: same selected observation, different ledger, flag, or graph. Try to reconstruct the missing datum from that observation and one of the witnesses defeats you. The result is sharp about the map it studies, and it is sharp on all three structural axes at once now, not two out of three. It does not say every conventional encoding must throw the datum away, or that these coordinates form a geometric basis.

Theorem 5.3 (μ -ledger uniqueness, for the kernel's fixed schedule). *Run any trace. The μ it accumulates is just the sum of the step costs, charged by the kernel's own fixed per-instruction schedule `canonical_cost` (the schedule `vm_apply` itself charges, not a stand-in for some other pricing choice). Partition graph, registers, memory, program counter, certified flag: none of them changes that sum. Formally:*

$$\mu_{final} = \mu_{initial} + \sum_{i \in prog} instruction_cost(i).$$

The right side uses the summation symbol from Section 2. It reads: add up `instruction_cost(i)` for every program element i in `prog`. The result does not depend on the starting state's structural fields. Two runs of the same program from any two starting states with equal $\mu_{initial}$ accumulate the same additional μ . Any measure that charges exactly

canonical_cost at every step, the identical schedule, and assigns 0 to *init_state*, agrees with *vm_mu* on every reachable state (*instruction_consistent_measure_equals_mu*). This fixes the accumulated total once the schedule and the zero start are fixed. It is not a claim that *canonical_cost* is the only schedule a system could honestly adopt, and it says nothing about a measure charging a different per-instruction price; Section 7 below keeps that separate question, exact-event pricing among possible schedules, apart from this one. If any step in *prog* has positive *canonical_cost*, then $\Delta\mu$ is strictly positive: this is the theorem's positive-cost-step guarantee, not a claim that no program or no instruction can cost zero.

Proof. Induct on *prog*. At each step, apply μ -Conservation.

Base (*prog* = []). Empty program. No steps. The sum is 0, so both sides reduce to μ_{initial} . The equation holds.

Step (*prog* = *i* :: *rest*). Let $s_1 = \text{vm_apply}(s_0, i)$. By μ -Conservation,

$$s_1.\mu = s_0.\mu + \text{instruction_cost}(i).$$

Apply the induction hypothesis to *rest* starting from s_1 :

$$\mu_{\text{final}} = s_1.\mu + \sum_{j \in \text{rest}} \text{instruction_cost}(j).$$

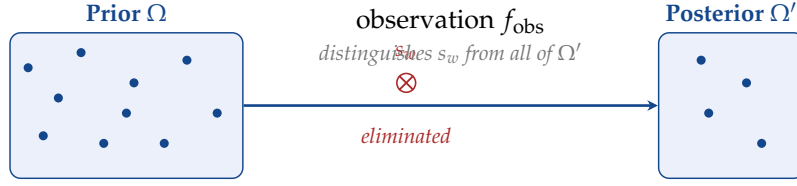
Substitute $s_1.\mu$:

$$\begin{aligned} \mu_{\text{final}} &= s_0.\mu + \text{instruction_cost}(i) + \sum_{j \in \text{rest}} \text{instruction_cost}(j) \\ &= \mu_{\text{initial}} + \sum_{i \in \text{prog}} \text{instruction_cost}(i). \end{aligned}$$

The right-hand expression depends only on *prog* and μ_{initial} . No other starting-state field gets a vote. If any step has positive cost, then $\Delta\mu$ is strictly positive: all terms are non-negative, and that term is ≥ 1 . □ □

| Plain reading. The change in μ across a run is just the sum of the per-step costs. μ -Conservation handles each step. Induction stitches them together. Two different starting states with the same starting μ end up at the same final μ on the same program. The starting graph, registers, memory: none of it matters for the μ count. Only the program does. The cost is attached to the trace, not invented by the state that carries it. The substrate records the receipt; it does not make the receipt up.

The ledger records the sum determined by the trace and the fixed instruction-cost schedule. Any accounting satisfying those same local increments and initial value agrees. Applying another schedule can give another total; the theorem does not assign an intrinsic physical bill to an unannotated classical computation.



$$\Omega' \subsetneq \Omega \text{ and } f_{\text{obs}}(s_w) \notin f_{\text{obs}}(\Omega') \implies \log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \leq \Delta\mu$$

Figure 3: Feasible-set narrowing and the entropy lower bound. A strict subset plus a distinguishing observation yields a strictly stronger posterior predicate (one fewer admissible state, witnessed by s_w). The number of yes/no bits the narrowing buys ($\log_2^\uparrow$ is the base-2 log rounded up, built in full at the representation theorem below) is at most the μ -cost of the trace that produced it. Same event, two sides of one equation.

Theorem 5.4 (Feasible narrowing becomes predicate strengthening). *Suppose a posterior feasible set Ω' is a strict subset of a prior feasible set Ω . Suppose also that an observation distinguishes a removed state from all posterior states. Then the membership predicate for Ω' is strictly stronger than the membership predicate for Ω . (The membership predicate for Ω is the function that takes a state and returns true exactly when the state is in Ω .)*

Proof. Define $P_\Omega(s) := (s \in \Omega)$ and $P_{\Omega'}(s) := (s \in \Omega')$. “Strictly stronger” has two parts.

(a) $P_{\Omega'}$ implies P_Ω . If $P_{\Omega'}(s)$ holds, then $s \in \Omega'$. Since $\Omega' \subseteq \Omega$, we get $s \in \Omega$. So $P_\Omega(s)$ holds.

(b) P_Ω does not imply $P_{\Omega'}$. Since $\Omega' \subsetneq \Omega$, there exists $s_w \in \Omega \setminus \Omega'$. So $P_\Omega(s_w)$ is true and $P_{\Omega'}(s_w)$ is false. That is enough for strictness. The distinguishing observation gives more: an explicit Boolean function witnessing s_w , not just the assertion that some such state exists. $\square$ $\square$

Plain reading. Smaller set, stronger predicate. That is the whole of (a), and it is pure set inclusion doing the work. (b) is where the distinguishing observation earns its rent. Without it you have an existential: somebody, somewhere, lives in the gap. With it you have a name and an address. An existential is a promise. A witness is the proof of the promise.

It feels like two separate things happened. First you did some work and the search space got smaller; then, as a reward, you got to assert something stronger. Two events, cause and effect, the second one earned by the first. That ordering is the illusion. There is no second event. The instant Ω shrinks to Ω' the stronger predicate already exists, because the predicate is nothing but membership in the set, and the set is smaller. Nobody pulled the predicate out of a hat to make the theorem go through; it was sitting there the whole time, defined by the boundary that just got redrawn. The

distinguishing observation is what lets you write the boundary down honestly, but it does not author a new claim on top of the narrowing. The narrowing is the claim. One fact wearing two costumes: shrink the feasible set and tighten the predicate are the same move read off opposite sides of the same equation.

Theorem 5.5 (Strengthening requires structure addition). *If a trace starts with no certification address, ends certified for a strictly stronger predicate, and that certification is tied to the supra-certification channel defined above, then the trace contains a structure-addition event.*

Proof. “No certification address at start” is $s_0.\text{vm_csrs.csr_cert_addr} = 0$. “Certified for the supra-cert channel at end” is $s_n.\text{vm_csrs.csr_cert_addr} \neq 0$, the definition of `has_supra_cert`. The trace runs from s_0 to s_n through a finite sequence of steps. The field `csr_cert_addr` starts at 0 and ends nonzero. Somewhere in the trace, it flipped. That step is, by the definition in Section 5, a structure-addition event. The strictly-stronger-predicate hypothesis names what the certification is *for*; it does not affect the existence of the flip. $\square$ $\square$

| Plain reading. Cert address starts at 0, ends nonzero. So somewhere along the trace, it flipped. That flip IS what we defined “structure-addition event” to be in Section 5. The proof is just observing: if the value went from 0 to nonzero across a finite trace, the change happened on some specific step. The mathematics is bookkeeping.

This is the formal answer to “where did the new admissible structure enter?” Arithmetic on its own never answers that, because arithmetic only ever hands back what was already derivable from what you fed it; it closes a question, it does not open the world up to a fact that was not already in the premises. A stronger admissible predicate is exactly such a new fact, so it could not have fallen out of the computation for free. It had to be added, and the theorem says where: not in a flourish at the end, not off the page, but at one specific step you can point to in the trace, a structure-addition event with a timestamp.

Theorem 5.6 (Universal certification cost). *Take any abstract computational system: states, step labels, step rule, cost function, certification predicate. If every one-step move from uncertified to certified costs at least 1, then every trace from uncertified to certified has total cost at least 1.*

Proof. This is the same theorem as the Universal No Free Certification result in Section 8, restated here at the structural-entitlement layer. The proof is the trace-induction argument given there. Empty trace cannot flip the cert predicate. A non-empty trace either flips it at the first step, where single-step A2 gives ≥ 1 , or flips it later, where the

induction hypothesis gives ≥ 1 on the rest. Either case yields total cost ≥ 1 . $\square$ $\square$

| Plain reading. This is the same theorem as the one in the NoFI section, carried up here on purpose. Down there it was proved as a fact about cost. Up here the structural-entitlement layer needs that same floor as one of its own load-bearing parts, not as a footnote pointing back down the page, because the whole argument about who gets to claim a stronger predicate stands on it: certification never comes free wherever A2 is in force. Accept A2 and you have already agreed that flipping certification on costs you something. A2 acceptance is the entry fee.

This theorem matters because it is not about the 51-opcode ISA. The cost floor is forced for any system the moment that system has a certification predicate and a cost rule that charges for certification transitions. Build the predicate so it can flip at zero cost and you have free certification by construction. And a system that refuses to represent certification at all has no way to talk about structural entitlement from the inside. The entitlement gets smuggled in from outside, and somebody signs their name to it. Smuggled entitlement is not entitlement, no matter whose name is on it.

Theorem 5.7 (Structural entitlement representation). *For any explicit witness of structural entitlement consisting of:*

- *a strict feasible-set narrowing $\Omega' \subsetneq \Omega$;*
- *an observation function that distinguishes at least one eliminated witness state from all posterior states;*
- *a certified posterior predicate built from membership in Ω' ;*
- *a decision tree realized by the trace: the tree's depth is bounded by the count of cert-setter instructions the trace executes (`decision_tree_realized_by_trace`, defined as `depth(T) ≤ cert_setter_executions`). This is a payment bound on depth, a scalar comparison, not a claim that the trace's step sequence is mapped node-by-node onto the tree's branches; and*
- *a posterior-representative reduction tying the prior states to posterior representatives,*

the posterior predicate is strictly stronger than the prior predicate, the trace contains a structure-addition event, and the size of the narrowing is bounded by the paid ledger:

$$\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \leq \Delta\mu.$$

Here $|\Omega|$ means the length of the prior feasible list, using the $|\cdot|$ notation from Section 2. Ω is a plain list `VMState`, not deduplicated and not a Coq set; $|\Omega|$ counts list entries, duplicates included, not distinct states. Nothing below assumes Ω is duplicate-free. $|\Omega'|$ is the posterior list's length, on the same convention. $\log_2^\uparrow$ means: take the base-2 logarithm and round up. It counts yes-or-no questions. A 1000-state set needs 10 binary questions because

$2^{10} = 1024$, the first power of two that can cover 1000 items. Both sides of this bound are integer question-counts, not real-valued entropies. The eye wants to read $\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'|$ as the textbook bit-gain $\log_2(|\Omega|/|\Omega'|)$, and that is the one reading to set aside: the two ceilings round in opposite directions, so the difference lands above that real-valued number about as often as below it. What it counts instead is the binary distinctions the narrowing forces the tree to make, and that integer (not its real-valued cousin) is what the proof bounds by the tree's depth, and the depth by $\Delta\mu$. $\Delta\mu$ is final μ minus initial μ across the trace.

Proof. Three conclusions, chained.

(i) *Strictly stronger predicate.* The strict narrowing $\Omega' \subsetneq \Omega$ and the distinguishing observation match the hypotheses of the theorem above. Apply it. The membership predicate of Ω' is strictly stronger than that of Ω .

(ii) *Structure-addition event.* By hypothesis, the posterior certification uses the supra-cert channel. So the trace starts with `csr_cert_addr = 0` and ends with `csr_cert_addr  $\neq$  0`. The Strengthening-requires-structure-addition theorem gives a structure-addition event at some step k^* .

(iii) *The $\log_2^\uparrow$ bound.* The realized decision tree T is binary; write ℓ for its leaf count. Sort the prior states by which posterior state they end at. That is $|\Omega'|$ groups, and the covering hypothesis (`tree_covers_feasible_reduction`, itself discharged from the trace's fibered reduction by `fibered_reduction_implies_tree_cover`) caps every group at ℓ states. So $|\Omega| \leq \ell \cdot |\Omega'|$. Take $\log_2^\uparrow$ of both sides and use that it is subadditive on products, $\log_2^\uparrow(ab) \leq \log_2^\uparrow a + \log_2^\uparrow b$:

$$\log_2^\uparrow |\Omega| \leq \log_2^\uparrow \ell + \log_2^\uparrow |\Omega'|, \quad \text{equivalently} \quad \log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \leq \log_2^\uparrow \ell.$$

A binary tree holds at most 2^d leaves at depth d , so $\log_2^\uparrow \ell \leq d$, and the two combine to

$$\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \leq \log_2^\uparrow \ell \leq d.$$

The realized-by-the-trace hypothesis is exactly the numeric bound $d \leq \text{cert_setter_executions}$: the tree's depth is charged against the count of cert-setter instructions the trace executes, not matched branch-by-branch against the trace's own step sequence. Section 7 charges each cert-setter step at least 1 in μ , so that many cert-setter steps cost at least that much total, and the total is at most $\Delta\mu$, the cost of the whole trace. Chain the inequalities:

$$\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \leq d \leq \text{cert_setter_executions} \leq \Delta\mu. \quad \square$$

□

| Plain reading. Three conclusions, three steps. Conclusion 1 is just the predicate-strengthening theorem, applied to the strict-narrowing and distinguishing-

observation hypotheses. Conclusion 2 is just the structure-addition theorem, applied to the trace’s start-uncertified, end-certified condition. Conclusion 3 is the entropy bound. A binary tree that compresses $|\Omega|$ states into $|\Omega'|$ leaf-classes has depth at least $\lceil \log_2(|\Omega|/|\Omega'|) \rceil$, bounded by $\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'|$ on the underestimate side. Each tree node is a certified observation. Each certified observation pays at least 1 in μ . The tree depth is bounded by $\Delta\mu$. Compression for less than you paid never shows up, because the ledger does not extend that kind of credit. The bound is not a price anyone gets to negotiate down; the universe already settled it.

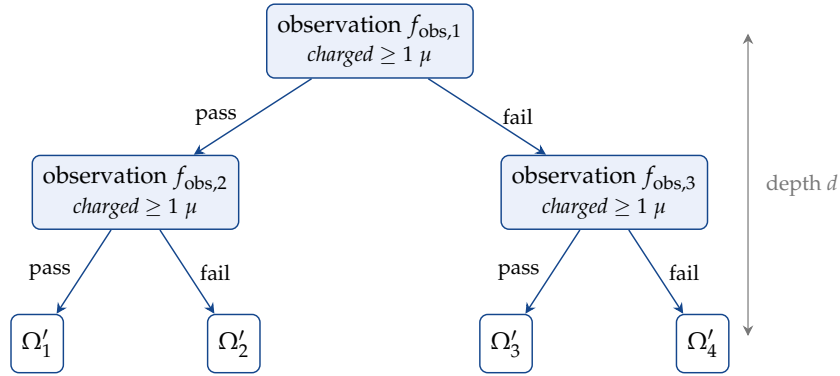


Figure 4: The decision tree witness. Each internal node is a certified observation; each leaf is one posterior class. The realised-by-the-trace condition bounds the tree’s depth by the count of cert-setter steps the trace executes, a numeric payment bound, not a claim that each node is individually matched to a specific step. The tree’s depth is what bounds the entropy gain on the right, and the μ paid for the trace is what bounds the depth on the left.

This theorem does not quantify over the English word “structure.” It defines the witness class: strict narrowing, distinguishability, certification, and an explicit tree/fiber representative for the reduction. For every member of that class, structural entitlement is not metadata and not a slogan. It is predicate strengthening. It requires structure addition. Its quantified narrowing is charged to μ . That is what entitlement means here. Not a feeling, not a story.

Theorem 5.8 (Every sound structural shortcut lands here). *A SoundStructuralShortcut is a package containing exactly the receipts a shortcut must provide to count as sound: a prior feasible set, a posterior feasible set, strict narrowing, a distinguishing observation, a certified posterior predicate, a realized decision tree, and a posterior-representative reduction. For every such package, the structural-entitlement representation theorem applies. The shortcut lands in strictly stronger posterior knowledge, a structure-addition event, and the $\Delta\mu$ lower bound.*

Proof. By construction, a SoundStructuralShortcut Coq record contains: Ω , Ω' with $\Omega' \subsetneq \Omega$, an observation function f_{obs} with a distinguishing-witness proof, a deci-

sion tree T with the numeric tree-payment obligation, a posterior-representative reduction with a representatives proof, and an initial-uncertified proof on the starting state. These fields and obligations are exactly the hypotheses of the Structural-Entitlement-Representation theorem. Project the seven data fields and eight proof obligations from the record into the theorem's hypotheses. Apply the theorem. The three conclusions (strictly stronger predicate, structure-addition event, $\log_2^\uparrow\text{-narrowing} \leq \Delta\mu$) follow. $\square$ $\square$

| Plain reading. The record is exactly what the representation theorem needs as input. Unpack the record. Plug the fields into the theorem. Read off the three conclusions. The class is non-empty because `factored_n1_shortcut` builds a member of the class on the page, by name. Non-empty by exhibition, not by reasoning.

This is the formal version of “it has to.” Once a shortcut stops being a human story and becomes a sound object the substrate may use, it has to carry the data that makes it admissible. In this framework, that data is exactly the structural-entitlement witness. When symmetry, factorization, modularity, independence, or decomposition is named as the source of a shortcut, the next question is no longer mystical. It is a checklist: the prior set, the posterior set, the observation that rules something out, and the representative witness and tree witness. With those in hand the shortcut is admissible. Without them the structure is still sitting outside the computation, however good the story around it sounds.

5.2 What narrowing buys

There's a concrete version of all this you can run, a factored search problem built to make the difference visible. Two programs go after the same answer. The blind one runs with every instruction cost set to 0 and searches the flat space, paying nothing and seeing nothing, which is the point and not an accident: a machine with no ledger gets its blindness for free. The sighted one pays first. Two receipt-visible events, each `EMIT " " 0`, for a total receipt cost of 18, and only then does it split the problem into two factors and search them apart. That 18 is the whole tab. The kernel proves the exact cost formulas, and it proves the arithmetic that makes the trade lopsided: the N^2 versus $2N$ gap doesn't merely win, it eventually dwarfs that constant μ cost by orders of magnitude, so the price you paid up front stops mattering almost immediately. The executable tests run the actual VM at representative sizes and watch the measured numbers land where the proof says they will. The receipt costs 18 once. The exponent saves you forever.

The point of this example is not the N^2 versus $2N$ speedup. Divide-and-conquer is not the new claim. Nobody is selling that. The point is the auditable trace. The receipt-visible `EMIT` events plus the certified suffix make the structural shortcut admissible to the substrate, not just asserted by the programmer who happens to feel good about

it. The $18\ \mu$ is the cost of producing the receipt. The receipt is the evidence that the structural shortcut is honest.

There are two nearby claims here and it matters to keep them separate, because conflating them is exactly how a real theorem turns into a stage trick. The `EMIT-only` $N = 1$ witness closes the observation layer. It proves posterior admissibility as `CertifiedObs` (a Coq type for “a certified observation has been recorded,” meaning the VM has observed and recorded a result but has not yet activated a cert channel). It does not close the stronger bridge. The trace does not end in `has_supra_cert`. In the semantic CSR-transition sense, it does not realize structure addition. A minimally extended witness does close the full theorem honestly. Keep the factorized search exactly as it is, then append `PNEW`, `MORPH_ID`, and `MORPH_ASSERT`. That suffix fires the concrete certification channel. The extended trace lands in the full structure-addition theorem. The fix is three opcodes long. The fix exists.

That example is not a P vs NP result. Anybody who reads it as one is reading their own ambition into the bytes. It is a small, honest witness for what the substrate is meant to track. Certified structure can change which computation is admissible. The cost of acquiring that admissibility is not the same as the time saved by using it. One is the entry fee. The other is the dividend.

What the package looks like in the concrete $n=1$ witness. It pays to lay the whole package open once, at the smallest factored-search case, where the object is small enough to count on your fingers and nothing has to be taken on faith. The record `SoundStructuralShortcut` carries fifteen fields. Seven are data: decoder, observation fn, representative observation fn, observation equality, prior set, posterior set, decision tree. The other eight are proof obligations standing over that data, the receipts that say the data behaves the way the fields promise. The nine semantic ingredients below cover six of the seven data fields, the decoder being the identity map and living in the wiring, plus five of those obligations: strict narrowing, the distinguishing observation, tree realization, the certified posterior, and the initial-uncertified state. The remaining three obligations are discharged inline by the wiring that holds the record together. Fifteen fields, nine ingredients, four pieces of wiring, and that is the whole of it. Nothing here is hand-waved, because there is no room left to wave a hand: every part has a place, and every place is filled.

1. **Prior feasible set Ω .** `sighted_n1_supra_prior := [init_state; sighted_n1_supra_final]`. A literal two-element list of `VMState` values.
2. **Posterior feasible set Ω' .** `sighted_n1_supra_posterior := [sighted_n1_supra_final]`. A literal one-element list.
3. **Strict narrowing.** $|\Omega'| = 1 < 2 = |\Omega|$ by direct inspection of the lists.
4. **Distinguishing observation.** `sighted_n1_supra_obs_fn`. Returns `sighted_n1_supra_trace` when `vm_mu = 19`, else `[]`. The `post-MORPH_ASSERT`

- state has `vm_mu = 19`. The initial state has `vm_mu = 0`. The observation separates them.
5. **Decision tree.** `sighted_n1_tree := dt_branch dt_leaf dt_leaf`. A two-leaf branching tree, one branch per factor.
 6. **Tree realized by the trace.** The trace's actual `EMIT-then-search-then-MORPH_ASSERT` sequence matches a root-to-leaf path of the tree.
 7. **Posterior-representative reduction.** `sighted_n1_supra_repr_obs_fn`. Defined as `if Nat.eqb s.(vm_mu) s.(vm_mu) then [] else [instr_halt 0]`; the tautology `Nat.eqb mu mu = true` always picks the empty-list branch, so this is the empty-observation representative dressed up as a conditional.
 8. **Certified posterior predicate.** Built from `omega_predicaterreceipt_list_eqbsighted_n1_supra_posteriorsighted_n1_supra_obs_fn`. The `MORPH_ASSERT` in the supra-suffix fires the certification.
 9. **Initial uncertified state.** `init_state.(vm_csrs).(csr_cert_addr)=0` by definition of `init_state`.

All of that machinery only earns its keep if something actually satisfies it, and the worry with any definition this elaborate is that you have quietly written down a shape nothing in the world fits. So the lemma `sound_shortcut_from_components` does the unglamorous thing: it takes the seven data fields listed above and eight proof hypotheses, one per obligation, and out the other end comes a `SoundStructuralShortcut` record. Then the closed term `factored_n1_shortcut : SoundStructuralShortcut33sighted_n1_supra_traceinit_state` feeds it the seven fields and clears each of the eight obligations with a named lemma, and the downstream theorem `factored_n1_shortcut_lands_in_representation` runs `every_sound_structural_shortcut_lands_here` over that finished term and reads back exactly what the abstract class promised: a strictly stronger posterior predicate, a structure-addition event, and the $\log_2^\uparrow$ narrowing bounded by $\Delta\mu$. The payoff is small to state and it is the one I actually cared about. The class is not empty, and it is non-empty by construction, not by argument: there is a concrete shortcut built on the page that hits every clause, which means the whole apparatus is about something real and not a definition admiring itself in a mirror.

5.3 The boundary

The hard internal theorem for the explicit class is proven, and so is the boundary in both directions on a concrete witness. An `EMIT-only` factorized-search trace closes only the observation layer. The posterior predicate is certified observationally. But the trace does not reach `has_supra_cert` and does not realize semantic structure addition. A trace with the factorized search unchanged and a post-search suffix `PNEW ; MORPH_ID ; MORPH_ASSERT` closes the whole chain: stronger predicate, structure

addition, and the $\Delta\mu$ lower bound. An unconditional Shannon-style bound on every deterministic single trace is something the code refuses to claim. A single execution path is not enough information. The bound needs the whole tree, or a per-branch preimage, or a distribution over traces. The proof stops where the witness stops. That is the honest place to stop, not a hole to be patched later.

Once entitlement is internalized with explicit witnesses instead of waved at, the picture firms up into three facts that lean on each other. You cannot strengthen a certified claim for free: the receipt costs μ or it does not exist. Strip a trace down to its classical shadow and the distinctions that entitlement turns on stop being visible at all, so the shadow can no longer tell you what was earned and what was merely assumed. And narrowing the feasible set is no longer a vibe; it has a formal path into predicate strengthening with a ledger lower bound underneath it, so the act of knowing more shows up as a number that went up. Then the substrate-level result in Section 19 settles the question you would ask next, which is whether all of this can be put on autopilot: is there a uniform, internally representable, always-terminating procedure that takes a formalized structural argument and hands back a `SoundStructuralS` `hortcut` witness whenever a sound one exists? There is not; Section 19 constructs the reduction. A partial procedure, restricted to some class of arguments or free to decline rather than terminate, is not what that reduction rules out. The structural axis comes with its own undecidable predicate, in that total-procedure sense, which is the unsurprising and slightly bleak news that insight does not reduce to a crank you can turn.

6 What insight is, and why it cannot be free

On this substrate the word *insight* stops being a mood and starts having a meter reading. It comes in three tiers, three genuinely different things that happen when a computation learns, and they sort themselves by price: the first is free, the second never is, and the third is where the price tag has its hand on something no classical account of correlation can reach at all.

6.1 The three tiers of observation

The kernel proves each tier lands where it lands, by what the substrate makes it cost, so the split is the substrate's verdict and not my filing system. Cheapest first.

Tier 0: Raw observation (always free). A raw observation takes something in, writes it down, and stops there. It does not say what the data means, does not lean on it, and does not let any later step lean on it either. The cleanest case in the realisation is a CHSH trial, one round of the two-player game I build from scratch in Section 16: you run the round, a counter ticks up by one, and not a single structural field moves. The certification channels stay cold because nothing has been claimed, and where nothing has been claimed there is nothing to commit to and nothing to pay for. Cost: zero. Looking is free on this substrate the same way looking is free in the world. You can

watch as long as you like and the meter never starts. It starts the instant you turn what you saw into something you are willing to stand behind, and that instant is a different tier.

Tier 1: Certified structural insight (always costs ≥ 1). Two cert channels. `csr_cert_addr` (the formula/certificate address, a control-status register; Section 9 explains the term). `vm_certified` (the top-level flag). Any concrete instruction that turns either channel from off to on costs at least 1. The kernel proves it: `certification_requires_positive_mu`.

Precision first. `LASSERT` reads a constraint package and charges for it. `LASSERT` does not flip either cert channel. Two opcodes do the flipping: `CERTIFY` (sets `vm_certified = true`) and `MORPH_ASSERT` (sets `csr_cert_addr` nonzero when the assertion succeeds). `LASSERT` is the validation that precedes them. The kernel groups a broader cost-positive class: `LASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `READ_PORT`, `CERTIFY`, `MORPH_ASSERT`, and the five-opcode `CHSH_LASSERT` check family (Section 10 builds it). Each costs at least 1 at the step level. `LASSERT` is part of the chain. The cert-channel flip happens at the subsequent `CERTIFY` or `MORPH_ASSERT`, not at `LASSERT` itself.

The chain, written out. Readers collapse two steps into one. Don't. Suppose register `r0` holds the address of a four-byte formula in memory and `r1` holds the address of a dual-witness certificate (a satisfying assignment together with a falsifying assignment, the non-triviality witness from Section 10). Step one: `LASSERT freg=r0 creg=r1 kind=SAT flen=4 delta=0` reads the formula header, walks the formula and the two witnesses through the on-chip Logic Engine, charges $4 \times 8 + 1 = 33$ to μ , and either advances the PC (validation passed) or traps to `LASSERT_TRAP_PC` and latches `vm_err = true` (validation failed). The μ cost is charged either way. Both `csr_cert_addr` and `vm_certified` stay untouched by this step, validation passing or not. Step two, conditional on step one passing: `MORPH_ASSERT mid=mproperty="validated_constraint" cert="" delta=0` charges $S(0) = 1$ to μ (S is successor, $S(n) = n + 1$; Section 7 builds the notation properly) and flips `csr_cert_addr` to the ASCII checksum of "validated_constraint", which is nonzero because the property string carries non-NUL bytes. The supra-cert channel is now active.

Total cost across the two steps: 34μ . `LASSERT` paid for the validation work. `MORPH_ASSERT` paid for the channel flip. The cert channel only crossed from 0 to nonzero on the second step, not the first. The two costs are not redundant. One is the cost of checking the constraint package against its dual witness. The other is the cost of committing to the result of that check by activating the cert channel.

That two-step shape is a protocol the program author chooses to follow, not a dependency the two opcodes enforce on each other. `MORPH_ASSERT` checks only that `mid` names an existing morphism; it does not read, and has no way to read,

whether a `LASSERT` executed earlier in the same trace, on the same formula the property string names. Nothing stops a program from calling `MORPH_ASSERT mid=m property="validated_constraint" cert=""` directly, with no `LASSERT` anywhere in its history, and getting the identical channel-active state and the identical checksum for the identical 34- μ -shaped cost if it pads with an equivalent no-op charge. `csr_cert_addr nonzero` means only that some `MORPH_ASSERT` fired on an existing morphism with a non-NUL-byte property string; it does not mean a specific formula was checked, and the property string is a label, not a proof obligation the opcode verifies against anything. A downstream predicate that references `csr_cert_addr` and relies on “the validated constraint the `LASSERT` step checked” is relying on the trace having actually been built in the two-step shape above, a fact about how the program was written, not a fact `MORPH_ASSERT`’s own semantics establish. Skipping `LASSERT` does not break a chain the kernel checks; it breaks a discipline the surrounding program was supposed to follow.

Tier 2: Certified binary-game witness insight (always costs ≥ 1). The sharpest case, and it is worth slowing down on why it is sharp. Here the trace is certified and its CHSH statistics show S pushed past 2, the classical ceiling (Section 16 builds the game and the ceiling from nothing). The certification on its own is already a Tier 1 event, so it already costs at least one. But look at what is being certified. $S > 2$ on a finite tally rules out any single fixed deterministic strategy replayed identically at every recorded trial (Section 16’s finite-tally remark); a fixed- λ shared-randomness-then-local-choice story of that narrow, unvarying kind cannot produce it. Reading the violation as ruling out local hidden variable models generally, the idealized-correlator claim Section 16 actually proves, needs enough trials that the tally and the population correlation have converged; a short run from a genuinely randomized local model can land past 2 by chance, as the same section’s coin-flip example shows. So this tier certifies a fact that constrains the space of classical explanations, more sharply as the trial count grows, and the price does not flinch for the size of the thing it is pricing. Certified evidence of a game violation costs at least one to come by, on a trace of any length you care to run.

6.2 Why insight cannot be free

The proof:

Suppose a trace starts in an uncertified state and ends in a certified state. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. The two concrete cert-channel instructions writing `vm_certified` or `csr_cert_addr` are: `CERTIFY` (which sets `vm_certified = true`) and `MORPH_ASSERT` when the morphism exists (which sets `csr_cert_addr` to the ASCII checksum of the property string: nonzero for any property string with a non-NUL byte). These are the only instructions that change those fields. `CHSH_LASSERT` sits in the certification cost-discipline class ($S(\delta) \geq 1$) but does not write to `vm_certified` or `csr_cert_addr`. Its success is observable via PC advance and `vm_err` staying

false (the bridge theorem operates on that signature). `MORPH_ASSERT` with an empty property string or a missing morphism does not activate the cert channel, though it still costs $S(\delta) \geq 1$. This is proven by case analysis over all 51 opcodes. Every other instruction leaves those fields unchanged. So: the cert flip happened at a `CERTIFY` or `MORPH_ASSERT` step. Look at that exact step. That step, by the semantics of the 51-opcode VM, has cost $S(\delta) \geq 1$, because both `CERTIFY` and `MORPH_ASSERT` are in the mandatory-floor class. So at that step: $\mu_{\text{after}} = \mu_{\text{before}} + S(\delta) \geq \mu_{\text{before}} + 1$. And because μ never decreases (each instruction adds a non-negative cost), $\mu_{\text{before}} \geq \mu_{\text{initial}}$. Therefore $\mu_{\text{final}} \geq \mu_{\text{after}} \geq \mu_{\text{before}} + 1 \geq \mu_{\text{initial}} + 1$.

This is a property of the specified step and cost rules, proved by case analysis and trace induction. It establishes that a certification transition cannot be free in this model. Semantic truth is a separate obligation: a paid flag alone proves no proposition, and a checker may be sound independently of a charging convention.

But cost alone is not yet semantic depth. A concrete realisation can spend resources checking a claim that turns out to be shallow. `LASSERT` does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend. The witness proves the spend bought an actual narrowing.

7 The μ ledger

μ (mu) is the global structural cost ledger of the Thiele substrate. One counter, carried by every state. Not the whole substrate. The part that keeps receipts.

More observables also do not automatically buy an additive bill. `joint_floor_is_least` proves that the least natural-number charge satisfying a finite family of unit event floors is one whenever any event fires, and zero otherwise. A single unit can satisfy several floors at once. Two independent Boolean coordinates provide a concrete example. If I want their costs to add, I need a resource-accounting condition that actually makes them add. Coordinate independence alone does not do that work.

7.1 What μ is

μ is a single natural number riding along on every substrate state (a natural number being a non-negative integer, $0, 1, 2, \dots$; the notation cheat sheet is in Section 2). It starts at 0 and it never goes down, and that second half is the whole character of the thing. A counter that can fall is an accountant you can lean on to revise the books. A counter that only ever climbs is a fact about history: it says how much has been spent, and it cannot be talked into saying less. μ is the second kind.

In the 51-opcode realisation, every instruction carries a cost parameter (δ) in its encoding. The VM adds that cost to μ when the instruction executes. Here is how δ is used

in practice:

For **pure compute programs**, meaning programs that do arithmetic, load and store values, and jump, every instruction carries $\delta = 0$. The program I use as the concrete zero-cost comparison case, `blind_program`, has this property: every instruction sits at $\delta = 0$. The proof that its total μ -cost is zero is a direct calculation. For these programs, μ stays at 0 unless a positive-cost instruction fires.

For **certification and revelation instructions** (`CERTIFY`, `LASSERT`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `READ_PORT`, and the five-opcode `CHSH_LASSERT` check family): the cost is at least 1, regardless of δ . Even $\delta = 0$ still costs 1. The positive floor is part of this instruction schedule.

`CERTIFY` sets a persistent flag and charges $S(\delta) = \delta + 1$. The first false-to-true transition is the event A2 prices. Subsequent executions still cost money under the VM schedule. Persistence of a flag does not by itself establish physical erasure: an implementation may retain the previous state elsewhere.

One is the least positive natural number. Choosing an additive mandatory floor of one gives $\delta + 1$, but A2 alone also permits larger charges. The substitution theorems identify exact unit pricing relative to the designated flip count, not the entire VM instruction schedule or a unique physical unit.

Throughout this document, $S(n)$ means $n + 1$. The S stands for “successor” in the basic math sense: the successor of n is the next natural number after n . So $S(0) = 1$, $S(1) = 2$, and so on. $S(\delta) = \delta + 1$ is always at least 1. `EMIT`, `REVEAL`, and `READ_PORT` go further: their cost also grows with the information content they carry. An `EMIT` of one ASCII character (which is 8 bits, see Section 2) with $\delta = 0$ costs $8 + 1 = 9$. Ten characters costs $80 + 1 = 81$. Emitting more costs more. The price tracks the bits, every time, and there’s no rate to negotiate.

For **everything outside that positive-cost class**, including `ADD`, `LOAD`, `STORE`, `JUMP`, `PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `COMPOSE`, and the rest, the VM charges exactly δ , which can be 0. Here is why these are allowed to be free.

The schedule permits module operations at $\delta = 0$. This is a design choice, not a derived thermodynamic statement. In particular, a zero-cost `PNEW` can change the graph, so positive cost for every observable structural change would be false for this VM.

Graph construction and assertion have different charges in this schedule. A physical interpretation would need a model of information-bearing states, operation boundaries, retained history, and dissipation; these cannot be inferred from opcode names or from graph changes alone.

μ grows whenever `instruction_cost` is positive. The certification/revelation class is forced to be positive even when its encoded δ is 0. Other instructions can remain

free by carrying $\delta = 0$.

Here is the complete cost schedule:

Instruction class	Cost = <code>instruction_cost</code>
Zero-floor operations (ADD, LOAD, JUMP, PNEW, MORPH, ...)	δ (can be 0)
Certification floor (CERTIFY, MORPH_ASSERT, LJOIN, the CHSH_LASSERT family)	$S(\delta) = \delta + 1$
EMIT (emit payload as receipt-visible trace event)	$ \text{payload} _{\text{bits}} + S(\delta)$
REVEAL (disclose bits from a module)	$\text{bits} + S(\delta)$
READ_PORT (read bits from input port)	$\text{bits} + S(\delta)$
LASSERT (validate formula and witnesses)	$\text{flen} \times 8 + S(\delta)$

The $|\text{payload}|_{\text{bits}}$ notation means “the number of bits in the payload.” A one-byte ASCII character contains 8 bits. A ten-byte string contains 80 bits.

For LASSERT: *flen* is the number of formula units in the encoded formula. Each unit is one byte (8 bits) in the binary encoding used by the VM. This is a design choice in the binary format. The formula header declares how many bytes follow. The VM charges 8 bits per byte. The multiplication by 8 simply converts byte-count to bit-count, matching the information content you are asking the trace to validate.

The proof that μ is always exactly $\mu_{\text{before}} + \text{instruction_cost}$ for each step:

Theorem 7.1 (μ -Conservation). *For any VM state s and VM instruction i : $(\text{vm_apply } s \ i).\mu = s.\mu + \text{instruction_cost } i$.*

Proof. Case analysis over the 51 opcode constructors of `vm_instruction`. For each constructor i , the apply rule has the form

$$(\text{vm_apply } s \ i).\mu = s.\mu + c_i,$$

where c_i is the cost term written in the apply rule. Compare to `instruction_cost`: for zero-floor opcodes $c_i = \delta$; for certification-floor opcodes (CERTIFY, MORPH_ASSERT, LJOIN, and the CHSH_LASSERT family, its four Q_{1+AB} variants included) $c_i = S(\delta)$; for EMIT $c_i = |\text{payload}|_{\text{bits}} + S(\delta)$; for REVEAL and READ_PORT $c_i = \text{bits} + S(\delta)$; for LASSERT $c_i = \text{flen} \times 8 + S(\delta)$. In every case c_i matches the corresponding case of `instruction_cost` by definition. The error sub-cases (e.g.,

MORPH_ASSERT on a missing morphism, CHSH_TRIAL with bad bits, LASSERT length mismatch) follow the same cost rule: the trap or err-latch only changes the PC and the error flag; the μ update is the same. So $(\text{vm_apply } s \ i).\mu = s.\mu + c_i = s.\mu + \text{instruction_cost}(i)$ for every i . $\square$ $\square$

| Plain reading. Look at each of the 51 opcodes one at a time. Every apply rule writes μ as the old μ plus the cost. The cost term is exactly what `instruction_cost` returns. Including the error paths: failing an instruction costs the same as succeeding at it, because the cost is on the attempt, not the outcome. There is no opcode that escapes this. Every one of the 51 lands in the same equation.

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. On this substrate, semantic gain is stricter than spend. It requires a narrowing witness in addition to the ledger entry.

7.2 Why mu is the unique canonical cost measure

Here is the stronger claim, which took a while before the proof compiled. μ is not just a valid cost measure for this 51-opcode realisation. It is the unique one. Any other cost function that starts at zero and tracks the per-instruction cost schedule exactly must equal μ on every reachable VM state. Not “isomorphic” in some abstract sense. Literally equal, value by value, on every reachable state.

Theorem 7.2 (μ -Initiality). *Let M be any function from VM states to natural numbers satisfying: (a) M assigns zero to the initial VM state, $M(\text{init_state}) = 0$; and (b) M tracks the cost schedule exactly: for every VM state s and every VM instruction i , $M(\text{vm_apply } s \ i) = M(s) + \text{instruction_cost}(i)$. Then M equals μ on every reachable VM state. (Monotonicity, the property that $M(s') \geq M(s)$ when s' is reachable from s , follows from (b) because `instruction_cost` is a non-negative natural number. You do not need to state monotonicity separately.)*

Proof. Induction on the reachability witness for s . A reachable VM state is either `init_state` itself or `vm_apply s' i` for some reachable s' and some VM instruction i .

Base case ($s = \text{init_state}$). By hypothesis (a), $M(\text{init_state}) = 0$. By definition of the initial state in the kernel, $\text{init_state}.\mu = 0$. So $M(\text{init_state}) = \text{init_state}.\mu$.

Step case ($s = \text{vm_apply } s' \text{ } i$ for reachable s'). By hypothesis (b),

$$M(s) = M(s') + \text{instruction_cost}(i).$$

By the induction hypothesis on s' , $M(s') = s'.\mu$. By μ -Conservation,

$$s.\mu = (\text{vm_apply } s' \text{ } i).\mu = s'.\mu + \text{instruction_cost}(i).$$

Substituting: $M(s) = s'.\mu + \text{instruction_cost}(i) = s.\mu$. □ □

| Plain reading. M agrees with μ at the start (both zero). The step rule says M and μ both update by the same amount on every step (M by hypothesis (b), μ by μ -Conservation). Induction closes it. There is no alternative accounting that agrees with the cost schedule at every step, starts at zero, and disagrees with μ on a reachable VM state. The cost is forced.

In plain language: given the cost schedule, there is no alternative accounting. If you charge $S(\delta)$ for certification-floor opcodes, $|\text{payload}|_{\text{bits}} + S(\delta)$ for `EMIT`, and δ for everything else, then μ is the only function on VM states that starts at zero and tracks the schedule. What was a design choice is the schedule itself: the decision that `CERTIFY` costs $S(\delta)$ and `ADD` costs δ . But the schedule is not arbitrary either, and that is the part worth slowing down on. Hold the trust fixed and `A2` is the only rule that prices certification exactly, so the only exact, non-overcharging substitute already is `A2` (`a2_equal_trust_substitution_payoff`). And $(\mu, \text{certified})$ is the proved-irredundant extension over $(\text{mem}, \text{regs}, \text{pc})$: `cert` is not a function of the rest, μ is not a function of the rest (`P_full_complete_neither_mu_nor_cert_drop_pable`). A law that lives in the step is not a check you run afterward and hope you got right. Given-the-schedule uniqueness is the accounting; the `A2`-and-irredundancy result underneath it is the priority. No competing count can agree on the schedule, start at zero, and land on a different total, and there's nowhere left for that freedom to hide.

7.3 The LASSERT cost formula

The most important cost formula in the VM is for `LASSERT`, the opcode that validates a logical formula and its witnesses:

$$\Delta\mu_{\text{LASSERT}} = \text{flen} \times 8 + S(\text{mu_delta})$$

flen is the formula-unit count the instruction declares, and the declaration drives the bill: eight concrete bits per declared unit. The per-step mandatory charge is always at least 1, no matter how cheap the unit-count feels. What keeps the declaration honest is the header check below: the VM reads the count the formula's own in-memory header carries, and no validation succeeds unless the two agree.

What makes this the one to watch is where the bill gets keyed. The operand drives the charge, but no successful check ever underpays, because succeeding requires the declared count to equal the count sitting in the formula's own header. Eight bits for every unit, plus the floor of at least 1 you pay for the act of checking at all, and the act of checking is not up for negotiation. So a two-unit formula lands at $16 + 1 = 17$, and there is no posture you can strike to talk it down: the formula's own header gives the true count, and a declaration that disagrees with it never validates anything. Declaring a smaller count does not shrink the formula; it only gets you caught. That is the whole reason you cannot smuggle a larger formula through a smaller bit count.

The minimum is relative to the stated formula-length charging constraints, as formalized by `cost_necessity` and `cost_uniqueness`. It is not a lower bound on all possible formula-validation algorithms or representations.

What this buys you: the trace pays for presenting and checking a constraint package, no more, no less. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true, a tautology, and the ledger would still charge you the full bill. The VM closes this separately via the dual-witness requirement: see the `LASSERT` opcode description in Section 10.

The obvious thing to try, and where it runs out. The programmer declares *flen* in the instruction encoding, so the first thing I wanted to know was: what stops them from writing *flen* = 0 for a huge formula and paying only $S(0) = 1$? It's the cheat I'd reach for first.

The VM catches it. When `LASSERT` runs, the VM reads the formula's actual encoded-unit count from the formula header in memory and compares it to whatever *flen* the instruction claimed. If they do not match, the VM traps (Section 2: routes execution to a fixed error address): the program counter jumps to `LASSERT_TRAP_PC`, which is `0xF00` in hexadecimal, decimal 3840. The error flag is set. The check does not succeed. This trap-to-fixed-address behaviour belongs to `LASSERT` and to the `CHSH_LASSERT` check family that inherits its discipline; every other error in the VM simply sets `vm_err` to true and continues advancing the program counter by 1. The checks are special because a failed check of this kind is a serious integrity violation, not a recoverable error. The μ cost is still charged: that detail matters because failed checks are not free. The Coq lemmas `lassert_honest_cost` and `lassert_honest_mu_cost` prove that if `LASSERT` completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

8 No Free Insight: the first cost law

8.1 The abstract versions

Theorem 8.1 (Universal No Free Certification). *Let $\mathcal{C}$ be any abstract computational system with:*

- a set of states;
- a set of step labels;
- a step rule (given a state and a step label, the rule produces the next state);
- a cost function (each step label has a non-negative integer cost);
- a certification predicate (a yes/no test on states, with at least one “no” state and at least one “yes” state).

Premise (A2, the single-step cost rule): for every state s and step label i , if $\text{cert}(s) = \text{false}$ and $\text{cert}(\text{step}(s, i)) = \text{true}$, then $\text{cost}(i) \geq 1$.

Conclusion: any trace of step labels $t = [i_1, i_2, \dots, i_n]$ that starts at state s_0 with $\text{cert}(s_0) = \text{false}$ and ends at state s_n with $\text{cert}(s_n) = \text{true}$ has total cost $\sum_{k=1}^n \text{cost}(i_k) \geq 1$.

Proof. Induction on the trace t .

Base case ($t = []$). Then $s_n = s_0$, so $\text{cert}(s_n) = \text{cert}(s_0) = \text{false}$. But the hypothesis says $\text{cert}(s_n) = \text{true}$. Contradiction; the empty-trace case is vacuous.

Step case ($t = i :: \text{rest}$). Let $s_1 = \text{step}(s_0, i)$ be the state after the first step label. Case-split on $\text{cert}(s_1)$.

- *Subcase A:* $\text{cert}(s_1) = \text{true}$. The first step label flipped the certification predicate from false to true in one step. By the A2 premise, $\text{cost}(i) \geq 1$. The total cost is $\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 1 + 0 = 1$, since every individual cost is a non-negative natural.
- *Subcase B:* $\text{cert}(s_1) = \text{false}$. Then the rest of the trace must do the flipping: $\text{cert}(\text{run}(\text{rest}, s_1)) = \text{true}$ with $\text{cert}(s_1) = \text{false}$. Apply the induction hypothesis at (rest, s_1) to get $\sum_{j \in \text{rest}} \text{cost}(j) \geq 1$. The total trace cost is $\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 0 + 1 = 1$.

Either subcase gives total trace cost ≥ 1 . □ □

| Plain reading. The cert predicate is off at the start, on at the end. So somewhere a step flipped it. Whichever step did the flipping costs at least 1 by the single-step rule. Every other step costs at least 0 because costs are non-negative naturals. Sum them up: at least 1. Empty trace can't do the flipping at all, so it's vacuous. That's the whole argument. The induction just makes precise the idea that the flip has to happen *somewhere*, and wherever it happens it pays.

This theorem does not know about registers, memory, partitions, morphisms, CHSH trials, or the 51-opcode ISA. It says: once a model has a yes/no certification test and the one-step certification transition is priced, trace-level non-free certification falls out by induction over the trace, no further premises required. If that one-step premise

fails, the model permits free certification by definition. If the model has no certification predicate, it cannot talk about structural entitlement internally.

And don't read "that's the whole argument" as "the premise is the whole content." The single implication is one line from the pricing rule, yes. The content is that this rule is the only one that fits. A trusted erasure-accounting law, the same trust posture, certifies at total cost zero when nothing is erased: a real rival, a different bill (`commitment_cost_not_reducible_to_erasure_cost`). And a substitute that charges any other predicate fails the floor on a one-step trace (`substitution_test_rejects_non_a2_exact_substitute`). A tautology does not get to have a rival that loses.

Theorem 8.2 (Thiele No Free Insight). *For the 51-opcode VM specifically, cert-channel activation is positive-cost, and certified predicate strengthening has to pass through a structure-addition event. The relevant certification/revelation steps all live in the positive-cost class.*

Proof. Two parts.

Cert-channel activation requires positive cost. The Both-channels theorem below establishes that any trace activating either certification channel (`csr_cert_addr` becoming nonzero or `vm_certified` becoming true) has $\Delta\mu \geq 1$. The actual channel-writing cases are covered by the positive-cost cert/revelation class: `CERTIFY`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `LASSERT`, `READ_PORT`, and the `CHSH_LASSERT` family. The class is deliberately broader than literal field writers; every member has `instruction_cost` $\geq S(\delta) \geq 1$, so the lower bound holds.

Certified predicate strengthening requires a structure-addition event. By hypothesis the trace ends certified for a stronger predicate via the supra-cert channel (`csr_cert_addr` nonzero). The Strengthening-requires-structure-addition theorem (Section 5) gives the structure-addition event directly. □ □

| Plain reading. This is just the abstract universal NoFI theorem instantiated for the 51-opcode realisation and read back through the Thiele substrate interface. The two concrete certification channels (`csr_cert_addr` and `vm_certified`) are both cost-positive at the step level, and the predicate-strengthening reading of NoFI falls out of the structure-addition theorem.

This is the VM-level version. The kernel proves the two concrete cert channels cannot be activated for free, by case analysis over all 51 opcodes, then adds the predicate-strengthening layer: once a stronger accepted predicate is tied to `has_supra_cert`, the trace must contain a structure-addition event somewhere in it.

8.2 The specific versions

For the 51-opcode VM concretely, there are several specific versions. The two certification channels I mentioned in Section 6 are the certificate-address register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 9, register by register.

Theorem 8.3 (No Free Certification, single step). *If $s.vm_csrs.csr_cert_addr = 0$ and $(vm_applies\ i).vm_csrs.csr_cert_addr \neq 0$, then $instruction_cost(i) \geq 1$.*

Proof. Case analysis over the 51 opcode constructors. The Coq proof uses `cert_addr_setterb`: if `csr_cert_addr` changes from 0 to nonzero, the instruction must fall inside that cost-positive class. The class includes `instr_morph_assert`, `instr_emit`, `instr_reveal`, `instr_ljoin`, and `instr_lassert`. It is a deliberately broad guardrail class, not a list of literal CSR writers. In the current `vm_apply` semantics, the direct CSR-writing success case is `instr_morph_assert`. For the theorem, the required fact is cost: every member of `cert_addr_setterb` has $instruction_cost \geq 1$, with `instr_emit` and `instr_reveal` higher because they also charge payload bits. $\square$ $\square$

| *Plain reading.* If you see `csr_cert_addr` move from zero to nonzero, the proof catches that step inside a cost-positive guardrail class. That class is broader than the direct CSR writers, but broad is fine here: every member costs at least 1. The field cannot flip for free.

Theorem 8.4 (No Free Certification, trace level). *If a trace starts at a state with $csr_cert_addr = 0$ and ends at a state with $csr_cert_addr \neq 0$, then the trace's total μ cost is ≥ 1 .*

Proof. Instantiate the universal theorem above with $\mathcal{C}$ = the 51-opcode VM; $cert(s) := (s.vm_csrs.csr_cert_addr \neq 0)$; $step = vm_apply$; $cost = instruction_cost$. The A2 premise is exactly the single-step theorem above. $\square$ $\square$

| *Plain reading.* Single-step lifts to trace-level. The universal theorem does the work.

Theorem 8.5 (No Free Certification via CERTIFY). *If a trace starts at a state with $vm_certified = false$ and ends at a state with $vm_certified = true$, then the trace's total μ cost is ≥ 1 .*

Proof. Case analysis over the 51 opcodes shows that `vm_certified` flips from false to true only when i is an `instr_certify` constructor; every other opcode leaves it unchanged. The `CERTIFY` apply rule charges $S(\delta) \geq 1$. So the single-

step instance of A2 holds for this channel. Instantiate the universal theorem with $\text{cert}(s) := s.\text{vm_certified}$, otherwise as above. $\square$ $\square$

| **Plain reading.** Same shape as the other channel. Only one opcode (CERTIFY) can flip the top-level flag. It charges ≥ 1 . The universal theorem lifts that to the trace.

Theorem 8.6 (Both channels). *For ANY transition of either certification channel from absent to present, whether the certificate-address register `csr_cert_addr` or the top-level flag `vm_certified`, μ grows by at least 1 over the trace.*

Proof. Suppose the trace either flips `csr_cert_addr` from 0 to nonzero, or flips `vm_certified` from false to true (or both). In the first case the trace-level theorem (`csr_cert_addr`) applies. In the second case the trace-level theorem (`vm_certified`) applies. The disjunction is exhaustive over “cert-channel activation,” so μ grows by ≥ 1 in either case. $\square$ $\square$

| **Plain reading.** Two channels, one master claim. Whichever channel got activated, the corresponding single-step rule fired and the universal theorem lifted it to the trace. There is no third way.

Theorem 8.6 is the one I actually reach for downstream. When something later in the document needs “and certification was not free,” this is the line it cites, because it has already done the bookkeeping of asking which way the certification came in and answered: doesn’t matter, both are priced. The certificate-address register `csr_cert_addr` and the top-level flag `vm_certified` are not two separate promises I have to keep track of; they collapse into one claim that holds no matter which door a program walks through to call itself certified.

8.3 What this means in practice

Run any program on the kernel. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr nonzero`, then somewhere in the trace, μ went up by at least 1. You can audit the trace and find exactly which opcode paid the bill. There is no program that can reach a certified state from scratch without paying for it.

On a valid execution from the initial state, a certified flag implies a positive accumulated charge. The ledger alone neither authenticates an externally supplied state nor proves an arbitrary proposition.

9 The formal state record

Definition 9.1 (Concrete state record). The kernel’s `VMState` record realises the substrate state in 12 fields:

- `s.vm_graph`: the partition graph. It contains module records, morphism records, and fresh-ID counters.
- `s.vm_csrs`: control-status registers. The hardware split: data registers hold values (like `s.vm_regs`); CSRs hold configuration and status. Four fields:
 - `csr_cert_addr`: 0 until something is asserted. Set to the ASCII checksum of the property string on a successful `MORPH_ASSERT` (nonzero for any property string with a non-NUL byte). The name says address; the contents say fingerprint. Nonzero here means a structural property was certified and paid for.
 - `csr_status`: general status code. Informational. Nothing reads it; nothing branches on it.
 - `csr_err`: error code. Zero means clean. Nonzero means an instruction failed: bad argument, missing morphism, locality violation, etc.
 - `csr_heap_base`: base address for `HEAP_LOAD` and `HEAP_STORE`. Final address = base + register-specified offset. No paging, no translation, just addition.
- `s.vm_regs`: a list of register values. The kernel fixes `REG_COUNT = 16`; reads and writes wrap modulo it. The kernel proofs are parametric in `REG_COUNT`, so 16 is a scaffolding choice, not a theorem.
- `s.vm_mem`: a list of data-memory words. The kernel fixes `MEM_SIZE = 128`; reads and writes wrap modulo it. The proofs are parametric here too.
- `s.vm_pc` $\in \mathbb{N}$: program counter. The integer that says *which instruction next*.
- `s.vm_mu` $\in \mathbb{N}$: the μ ledger, the substrate's accounting field. Starts at 0. Never decreases.
- `s.vm_mu_tensor`: a 4×4 array of cost values (16 entries), indexed by direction pairs (i, j) . `REVEAL` is the one opcode that writes it, stamping the disclosure cost at the direction index encoded in the instruction; the per-module 4×4 grids that `TENSOR_SET` writes and `TENSOR_GET` reads are separate state, carried inside `vm_graph` (Section 10). The kernel tracks *where* costs land, not just how much. In the kernel model these are natural numbers; in the hardware, 32-bit words. If you don't use the tensor tracking, ignore all 16 entries.
- `s.vm_err` $\in \{\text{false}, \text{true}\}$: the error flag. Set when an instruction fails.
- `s.vm_logic_acc`: a natural number with two roles. Role one: the Logic Engine accumulator during `LASSERT` (the on-chip unit that reads formula words and certificate bytes and tracks validation state). Role two: the magic key the FSM matches against. `0xCAFEFACE` (hex; 3,405,703,886 decimal) passes. Anything else, the FSM rejects. The field exists so the Coq kernel and the physical hardware agree on the same accumulator value at every step.

- `s.vm_mstatus`: status word, 0 or 1. 0 = Turing mode (ordinary computation; partition graph and morphism map idle). 1 = Thiele mode (structural state live). The kernel keeps this as a status indicator only. It does not gate the structural opcodes at the step level; those are always available. The field exists so the hardware and the Coq model agree across the bridge proofs.
- `s.vm_witness`: the CHSH witness counters. This is a record of 8 natural-number counters, one for each combination of (measurement setting pair) $\times$ (same/different outcome). The four setting pairs are $(x, y) \in \{(0,0), (0,1), (1,0), (1,1)\}$, where x is Alice's setting and y is Bob's setting. That is the modern Bell-test convention (NPA hierarchy, Brunner et al. 2014 review), and the same one the Coq constructor `instr_chsh_trial (x y a b : nat)` uses. The outcome variables are (a, b) . For each setting pair, there are two counters: same-outcome and different-outcome. Eight counters total: `wc_same_00`, `wc_diff_00`, `wc_same_01`, `wc_diff_01`, `wc_same_10`, `wc_diff_10`, `wc_same_11`, `wc_diff_11`, where the numeric subscript is the value of (x, y) for that bucket. Each `CHSH_TRIAL` instruction increments exactly one counter. (Some Bell-test literature uses the reverse convention with (a, b) for settings; the numeric subscripts 00, 01, 10, 11 are unambiguous either way.) The CHSH score S is computed from the aggregate sums of these counters. For each setting pair (x, y) , define $N_{xy} = \text{wc_same}_{xy} + \text{wc_diff}_{xy}$ (total trials with that setting) and $E_{xy} = (\text{wc_same}_{xy} - \text{wc_diff}_{xy}) / N_{xy}$ (the correlation: fraction same minus fraction different, ranging from -1 to $+1$). Then $S = E_{00} + E_{01} + E_{10} - E_{11}$. If no trials have been run for a setting ($N_{xy} = 0$), the correlator for that setting defaults to 0 by convention. Classically $|S| \leq 2$; the NPA algebraic model allows $|S| \leq 2\sqrt{2}$ (see Section 16). The concrete Coq definition is `chsh_stat_from_wc`.
- `s.vm_certified` $\in \{\text{false}, \text{true}\}$: the top-level certification flag. Only `CERTIFY` sets it. Nothing else can.

The reference kernel and the physical hardware are not the same shape at every bit, on purpose. The kernel is where I pin the step rule down precisely enough that nothing can quietly drift; it is the gap-catcher, not the source of the truth. The truth is the cost argument itself, the thing you could re-run with a pencil. The kernel stores register and memory values as natural numbers, but truncates writes through 64-bit word helpers to keep the math clean and the bit-width honest. The hardware layer (explained in Section 14) is finite: 16 general-purpose registers, 128 memory words, bounded tables. The bridge proofs in the hardware section state exactly where those finite hardware representations match the kernel step; the size constants are parametric, so the same proofs apply at any bound, and the 16 and 128 are scaffolding choices, not theorems.

Definition 9.2 (Partition Graph). The partition graph `s.vm_graph` is a record:

- `pg_next_id`: the next fresh module identifier.
- `pg_modules`: a list of module-ID/module-record pairs. Each `ModuleState` holds region data, axiom data, and the module's μ -tensor.
- `pg_next_morph_id`: the next fresh morphism identifier.
- `pg_morphisms`: a list of morphism-ID/morphism-record pairs. Each `MorphismState` holds a source module, target module, coupling data, and an identity flag.

Remark 9.1. There is no separate `vm_heap`, `vm_output`, or `vm_imem` field in the Coq kernel's `VMState`. Heap operations are heap-relative accesses into `vm_mem`. Output and instruction transport live one layer up: the runner, the trace, the RTL harness. None of that is kernel state. The distinction matters because NoFI is a theorem about the record above, nothing more.

Remark 9.2. The partition graph splits in two. The *module layer*: how state is decomposed into named pieces. The *morphism layer*: the typed categorical relationships between those pieces. The two layers are independent. Two states can agree on every module and still differ on every morphism. The Categorical Separation Theorem (Section 12) lives on that fact.

10 The instruction set: all 51 opcodes

The kernel realises the Thiele substrate as a 51-opcode VM (the term “opcode” is in Section 2: the numeric tag the hardware decodes to choose which behaviour to run). Every one carries an explicit `mu_delta` parameter. Cost is part of the instruction, not bolted on after. The VM reads it, the VM charges it.

One reminder before the machinery. This is scaffolding. These 51 opcodes are one way to build the substrate, they are not the substrate. Burn every one of them and the machine is exactly as real, because the machine is the law, not the instruction set. What follows shows the machine can be built and can't be waved away. It is not what makes it real.

10.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the kernel builds structural knowledge: one module, one partition edit, one accounting entry at a time.

PNEW(R, δ): allocate a new module.

Creates a fresh module and charges δ to μ . The instruction carries a region list R describing which memory addresses belong to this module. The current step rule normalises this to a standard form (the math sense of “canonical”: a unique representative for a class of equivalent inputs): it keeps the size (how many addresses) and stores it as the range $0, 1, \dots, \text{sz}-1$. So the checked step preserves the module

size, not arbitrary input geometry. The bounded RTL has its own partition-table and active-module machinery for locality enforcement (locality, in this VM, is the constraint that an instruction may only access memory inside its own active module's region; Section 14 covers the table). The `PNEW` opcode itself is the graph allocation step, nothing fancier.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the VM “this region of computation is a named unit I’m tracking.” The cost δ is programmer-declared and can be 0; the VM does not force you to pay for allocation itself. If building this partition cost something because you had to figure out the right boundary, run a search, or derive the structure, you record that here. If it was bookkeeping, $\delta = 0$ is fine, no fee for shuffling paper.

PSPLIT(m, L, R, δ): split one module into two.

Splits module m and charges δ to μ , no more. The constructor still carries left and right region payloads, but the current hardware-aligned step ignores those payloads and calls `graph_hw_psplitt`: the left side gets half the module size, and the right side gets the remainder. This is the refinement operation in the finite hardware model. If discovering the split deserves a cost, encode a positive δ ; the canonical reversible-accounting policy keeps it at 0.

PMERGE(m_1, m_2, δ): merge two modules into one.

Merges modules m_1 and m_2 and charges δ . The hardware-aligned step concatenates the module sizes through `graph_hw_pmerge`; module IDs wrap modulo 64. It does not perform a rich invariant-compatibility check in this step. Like `PNEW` and `PSPLIT`, the VM does not enforce a minimum cost here. Trust comes later, at the certification step, when the structure has to pay for being believed.

PDISCOVER($m, evidence, \delta$): query partition structure.

The evidence payload is carried by the instruction, but the hardware-aligned relation does not attach axioms or mutate the graph. It advances the PC and charges δ . This is a placeholder for discovery accounting, not a register query in the current kernel step.

10.2 Group 2: Logic and certification (3 opcodes)

This group carries the Logic Engine’s checked validation step and its ledger side. `LASSERT` is the one that talks to the Logic Engine, the on-chip finite-state machine that checks formulas and certificates; `LJOIN` and `MDLACC` are ledger entries standing beside it. No outside solver gets smuggled in.

LASSERT($freg, creg, kind, flen, cost$): validate a formula and witnesses.

Reads a formula whose declared encoded-word count is $flen$ from memory starting at the address in register $freg$. The $kind$ boolean flag selects which validation path the Logic Engine should follow. “SAT” (satisfiable) means: I’m claiming this formula has at least one satisfying assignment, some combination of variable values that makes it

true. “UNSAT” (unsatisfiable) means: I’m claiming it has no satisfying assignment, it is always false. In the on-chip semantics, *kind* = *true* is the only success path: it requires a dual-witness certificate, one satisfying assignment and one falsifying assignment, so the validation proves the formula is satisfiable and not a tautology. *kind* = *false* is the UNSAT-tag path: `lassert_check_ok` returns false unconditionally when *kind* = *false*, so any `LASSERT` issued with *kind* = *false* fails the check, traps to `LASSERT_TRAP_PC`, and sets the error flag. The μ -cost is still charged. The SAT-only validation path is the kernel’s structural-proof shape; the UNSAT tag is a reserved opcode bit that the kernel routes straight into the trap. This matches the RTL FSM, which has phases 0 (idle), 1 (header read), and 2 (scan). The kernel and hardware both validate that the declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. `LASSERT` reads a certificate block from memory starting at the address in register *creg*. That block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine FSM processes the formula and both witnesses in-place. No external solver gets a hidden vote. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula excludes at least one valuation. `LASSERT` itself is a checked validation step. Certification channels are activated by concrete state-changing paths such as `CERTIFY` and `MORPH_ASSERT`. The abstract cert-address theorem uses a broader guardrail predicate and puts `LASSERT` in the cost-positive certification/revelation class. That is not a claim that `LASSERT` writes the certificate-address register. It is a claim that this class cannot move for free. Charges $flen \times 8 + S(cost)$ to μ , even when the check fails.

This is the most important logic opcode in the VM. `LASSERT` is where a constraint package is checked before any later certification step can lean on it. There’s no faking the pricing anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the VM traps. And no faking semantic narrowing with a tautology either, which would otherwise be the slick move: the dual-witness requirement closes the tautology-inflation hole, because a tautology has no falsifying witness to show the formula excluded any valuation, and the VM wants to see that witness or it walks. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object, not to a thing the programmer wished they had checked.

LJOIN(*c1reg*, *c2reg*, δ): reserve certificate-join cost.

Reserves the cost of joining two memory-resident certificates. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. In the current hardware-aligned semantics, `LJOIN` does not compare certificate strings, write a composite certificate, touch CSR state, or mutate the graph. It advances the PC and pays the positive floor. The join proof lives at the abstraction/verifier boundary, not inside this kernel step.

Opcode	What it does	Typical cost
LOAD_IMM(r, v, δ)	Write immediate value v into register r	0
XFER(r_{dst}, r_{src}, δ)	Copy register r_{src} to r_{dst}	0
ADD(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
SUB(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
MUL(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a \times r_b$ (modular 64-bit)	0
AND(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ AND r_b (bitwise)	0
OR(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ OR r_b (bitwise)	0
SHL(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted left by r_b bits	0
SHR(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted right by r_b bits	0
LUI(r_d, v, δ)	Load upper immediate: $r_d \leftarrow v \ll 8$	0
XOR_LOAD($r_d, addr, \delta$)	Load from absolute memory address $addr$ into r_d	0
XOR_ADD(r_d, r_s, δ)	$r_d \leftarrow r_d$ XOR r_s (XOR accumulate)	0
XOR_SWAP(r_a, r_b, δ)	Swap registers r_a and r_b (reversible, direct two-write exchange)	0 by convention
XOR_RANK(r_d, r_s, δ)	$r_d \leftarrow \text{popcount}(r_s)$ (Hamming weight)	0

MDLACC(m, δ): MDL cost accounting.

Charges δ as a module-discovery/model-description cost for module m . It does not read a register, and it does not mutate the graph. It just enters the bill.

Suppose you have two models that both fit the same data. One is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which bill do you want to pay? Minimum description length says the total bill is the model description plus the cost of encoding the data once the model is known. Description length is how many bits it takes to write the model down. A simple formula takes few bits. A massive network takes many. The short formula may cost more to encode the remaining data, but almost nothing to describe. The massive network may compress the training set perfectly, but its own description is enormous. MDL picks the smaller total: bits for the model plus bits for the data given the model.

In the Thiele vocabulary: a module structure that you built can carry a description-length cost. Not because someone decreed it. Because constructing a specific partition of state space may require information that belongs on the bill. MDLACC is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and charge the encoded δ . The MDL connection is just this: the substrate's μ -ledger can track the model-description bill when the program chooses to enter it.

10.3 Group 3: Compute and data transfer (14 opcodes)

Ordinary arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural entitlement. They just compute.

Why does the ISA have all these? Because the 51-opcode realisation has to run programs, not just keep receipts. Programs need to compute intermediate values, manage data, and control flow. The compute group gives them that ordinary machinery. The key property: none of these opcodes touch the certification channels. Arithmetic can move bits all day. It cannot mint certified structural knowledge.

XOR_SWAP is a reversible register swap. The kernel implements it as a direct two-write exchange. The name suggests the classic XOR-swap sequence that some assembly programmers use to swap two values without a temporary register (it works by writing one register with the XOR of both, then the other, then the first again), but the kernel does not actually run that sequence. It writes both registers at once. In the table it carries $\delta = 0$ by convention because a reversible operation, in the Bennett/Landauer sense (Section 17), does not destroy any information. If a program supplies a nonzero δ , the ledger still charges it.

10.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control. This is the group where a machine could quietly cheat if you let it: load from wherever it pleases, jump wherever it pleases, and the partition stops meaning anything. So this is exactly where the locality story has to hold. The opcodes here move data and steer the program counter and nothing more, and the wall that keeps a module's reach inside its own region lives one layer down, in the bounded hardware table that the partition-wall note right below makes concrete.

Opcode	What it does	Note
LOAD(r_d, r_a, δ)	Load <code>vm_mem[r_a]</code> into r_d	Kernel register-indirect load
STORE(r_a, r_s, δ)	Store register r_s into <code>vm_mem[r_a]</code>	Kernel register-indirect store
JUMP(addr, δ)	Set PC to addr unconditionally	none
JNEZ(r , addr, δ)	Jump to addr if $r \neq 0$	none
CALL(addr, δ)	Push return address via stack pointer r15, jump to addr	Qed coverage
RET(δ)	Pop return address via stack pointer r15, jump there	Qed coverage
HALT(δ)	Advances PC and charges δ at the kernel step, like any other instruction. The runner stops at HALT (and when PC goes past the program), stamping a halt status; the hardware latches halted.	runner-level
CHECKPOINT(ℓ , δ)	Accept label ℓ , advance PC	label outside kernel

At the kernel level, LOAD and STORE are register-indirect memory operations over

vm_mem. At the RTL/cosimulation layer, the partition wall adds active-module locality checks. INIT_PT and INIT_ACTIVE_MODULE are harness setup commands for that finite hardware table, not VM opcodes and not fields of VMState. This is the honest split: the abstract ISA defines memory semantics; the bounded hardware layer enforces locality through its own table state.

10.5 Group 5: Revelation, I/O, heap, and tensor (7 opcodes)

This group mixes two fundamentally different kinds of operations. EMIT and READ_PORT are in the *certification/revelation cost class*: they cost ≥ 1 unconditionally and are subject to the same mandatory-floor rule as CERTIFY and MORPH_ASSERT. WRITE_PORT, HEAP_LOAD, and HEAP_STORE are ordinary I/O and storage, no mandatory floor. TENSOR_SET and TENSOR_GET manage per-module cost tensors. They share this group only by physical-channel adjacency; their cost semantics are distinct.

Opcode	What it does	Cost
READ_PORT($r, c, v, bits, \delta$)	Write value v into register r . The value v is embedded in the instruction encoding at assembly time (the kernel step is deterministic: it writes the declared v , not a runtime port read). Channel c and the $bits$ count are used by the runner/environment layer for I/O orchestration and cost accountability.	$bits + S(\delta) \geq 1$
WRITE_PORT(c, r_s, δ)	Send register r_s to channel c outside kernel state	δ
EMIT($m, payload, \delta$)	Emit payload as a receipt-visible trace event	$ payload _{\text{bits}} + S(\delta) \geq 1$
HEAP_LOAD(r_d, r_a, δ)	Load from <code>csr_heap_base + regs[r_a]</code> into r_d	δ
HEAP_STORE(r_a, r_s, δ)	Store r_s to <code>csr_heap_base + regs[r_a]</code>	δ
TENSOR_SET(m, i, j, v, δ)	Set per-module tensor entry (i, j)	δ
TENSOR_GET(r_d, m, i, j, δ)	Read per-module tensor entry (i, j) into r_d	δ

EMIT charges the encoded payload length in bits plus $S(\delta)$. READ_PORT charges the declared $bits$ operand plus $S(\delta)$. Its kernel semantics does not prove that this declaration equals a physical transfer length. REVEAL likewise needs an external evidence contract to represent a binding commitment: an unchecked certificate operand alone supplies no such guarantee. The positive floor holds independently of those external interpretations.

TENSOR_SET and TENSOR_GET access a flattened 4×4 tensor in the kernel and dedicated tensor state in the hardware model. Their intermediate correspondence requires the source theorem's index and module conditions. Physical RTL agreement must also

satisfy the finite representation and retirement contract in Section 14.

10.6 Group 6: Witness and certification (8 opcodes)

These opcodes accumulate binary game outcome statistics into hardware witness counters, enforce the NPA-PSD boundary on those counters, and manage the top-level certification flag.

CHSH_TRIAL(x, y, a, b, δ): record one CHSH trial.

Records one trial of the CHSH game. $x \in \{0,1\}$ is Alice’s measurement setting. $y \in \{0,1\}$ is Bob’s measurement setting. $a \in \{0,1\}$ is Alice’s outcome. $b \in \{0,1\}$ is Bob’s outcome. The Coq constructor order is settings first (x, y), then outcomes (a, b). The instruction increments exactly one of the 8 `WitnessCount` counters: if $a = b$ (same outcome), it increments `wc_same_xy`; if $a \neq b$ (different outcome), it increments `wc_diff_xy`. There are 4 setting pairs $\times$ 2 same/different buckets = 8 counters. The CHSH score S is computed later from these counters. Charges δ to μ .

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that `CHSH_TRIAL` does not and cannot trigger a cert-insight event (proven, `chsh_trial_is_tier0`). One guardrail: if any of x, y, a, b is not in $\{0,1\}$, for example passing $x = 5$, the VM fires a bad-bits error step. The error flag latches, no counter is incremented, but the cost is still charged. The VM does not silently count invalid inputs as valid trials, and there is no path through the kernel that lets you pretend it did.

CERTIFY(δ): set the top-level certification flag.

Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ , advances PC. This is the simplest cert-channel activation: you are saying “I, the program, certify that this execution has reached an attested state.” The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1 .

REVEAL($m, bits, cert, \delta$): disclose a value.

Reveals *bits* bits from module m with certificate string *cert*. In the current kernel relation, the certificate string is carried but not checked; the priced event is the disclosure itself. A bit-commitment protocol is a two-phase scheme: first you commit to a value (seal it in an envelope), later you reveal it. The reveal step proves you didn’t change your answer after seeing the other side’s response. `REVEAL` is the disclosure half of that scheme. `REVEAL` charges $bits + S(\delta)$: the number of bits being disclosed, plus at least 1. The disclosed bit count is also recorded in the global μ -tensor at flat index $(m \bmod 16)$, where the module operand is read as the tensor direction index (the μ -tensor is indexed by direction pairs, not module slots; Section 9), marking which direction the disclosure landed in; the $S(\delta)$ part of the charge goes to the ledger alone. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. Disclosing more costs more, and there’s no plan where you reveal the whole module for the price of

one bit. This is what makes the revelation requirement concrete: getting receipt-visible cross-correlations from binary game trials requires an explicit disclosure instruction, and that disclosure is priced by how much you reveal.

CHSH_LASSERT(δ): enforce the level-1 NPA-PSD boundary on the witness counters. Reads the 8 `vm_witness` buckets, runs the integer-arithmetic check `column_contractive_check_witness` ($\mathbb{Z}$ arithmetic only, Q_1 column-contractivity), and advances PC on pass or traps to `LASSERT_TRAP_PC` and latches `vm_err` on fail. Charges $S(\delta) \geq 1$ either way. A non-trapping step entails the witness-derived 5×5 NPA moment matrix is PSD (`chsh_lassert_no_trap_implies_quantum_realizable`). The four `CHSH_LASSERT_1AB` variants below extend this base check from the 5×5 level-1 matrix to the 9×9 level-1+AB matrix; the full treatment of the family is in the “CHSH check family” note after the opcode tally and in Section 16.

CHSH_LASSERT_1AB(δ): Q_{1+AB} -aware witness check at $\gamma = 0$.

Extends `CHSH_LASSERT` with the level-1+AB NPA condition. Reads the 8 `vm_witness` buckets, runs an integer-arithmetic check (Q_1 column-contractivity plus the SOS sum-of-squares condition $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$ at $\gamma_1 = \gamma_2 = \gamma_3 = \gamma_4 = \gamma_5 = 0$), advances PC on pass with cert state intact or traps to `LASSERT_TRAP_PC` on fail. Charges $S(\delta) \geq 1$ either way. The bridge theorem `chsh_lassert_1ab_no_trap_implies_quantum_realizable_q1ab` proves a non-trapping step entails `quantum_realizable_q1ab = symmetric9  $\wedge$  PSD9` of the 9×9 NPA Q_{1+AB} moment matrix at the witness-derived rationals.

CHSH_LASSERT_1AB_G5($\delta, same_g5, diff_g5$): Q_{1+AB} check with the four-body moment γ_5 free.

Carries one γ -bucket pair (`same_g5`, `diff_g5`) for $\gamma_5 = (same_g5 - diff_g5) / (same_g5 + diff_g5)$. Runs the γ_5 SOS check (a weighted 4-D Cauchy-Schwarz identity in cleared integer form), enforces $-Dg5 < Ng5 < Dg5$ as part of the bool decider. On pass, the bridge gives PSD9 at $(E, 0, 0, 0, 0, \gamma_5)$.

CHSH_LASSERT_1AB_G345($\delta, same_g3, diff_g3, same_g4, diff_g4, same_g5, diff_g5$): Q_{1+AB} check with three-body and four-body moments $\gamma_3, \gamma_4, \gamma_5$ free.

Carries three γ -bucket pairs. Runs the γ_{345} check: an augmented 2×2 Schur-slack lemma plus a 4×4 Sylvester PD condition on the difference matrix $H_{\gamma_{345}} = \det_M \cdot M_M - M_N$. On pass, the bridge gives PSD9 at $(E, 0, 0, \gamma_3, \gamma_4, \gamma_5)$.

CHSH_LASSERT_1AB_G12345($\delta, same_g1, diff_g1, \dots, same_g5, diff_g5$): full Q_{1+AB} check with all five γ moments free.

Carries five γ -bucket pairs. Runs the `sym6  $\rightarrow$  sym5  $\rightarrow$  sym4` Schur-cascade check on the supplied rational moments. A passing check implies PSD9 of the specified moment matrix. The four Q_{1+AB} opcodes are represented in the Kami model and its conditional kernel correspondence. The synthesis target contains the 47-opcode subset, including

the base `CHSH_LASSERT`; the four higher-moment variants are outside that synthesis target.

MORPH_ASSERT (see Group 7): on success, it activates the cert-address channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID.

10.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that operate on the categorical morphism layer of the partition graph. Six of them (`MORPH`, `COMPOSE`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_ASSERT`, `MORPH_TENSOR`) modify graph state and `is_classical_opcode` returns false on them. `MORPH_GET` only reads graph state into a register and is classified as classical by that predicate, even though a Turing machine still has to simulate the graph on its tape to evaluate it. A Turing machine can simulate programs using these opcodes by encoding the full Thiele state on its tape. The simulation can be honest (maintaining the μ -ledger cost semantics in the encoding) or dishonest (omitting that accounting). The TM substrate's transition relation is indifferent to which, which is exactly the substrate-versus-program distinction Section 3 names: A2 lives in the simulator program when the substrate runs the simulation, not in the substrate itself.

A *morphism* is a record saying: “module *A* is related to module *B* in this specific, named way.” Not just “*A* and *B* exist.” Not just “*A* and *B* are connected by an edge.” A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling field's operations are actual relational content, proven as such: `COMPOSE` computes the relational composition of two morphisms' coupling lists, pair (a, c) included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second, and this composition is proven correct against the standard relational-composition definition; `MORPH_TENSOR` concatenates two coupling lists. Neither operation is a programmer tag; both are data the kernel builds and proves properties about, at the level of the coupling type.

What is reachable at runtime is narrower than that type permits. `MORPH` always constructs its morphism with an empty coupling list, regardless of the `coupling_idx` argument the opcode is given; `MORPH_ID` also starts empty. Composing or concatenating empty coupling lists returns an empty coupling list. So in the current reference semantics, every coupling list reachable from `init_state` through the seven opcodes of this group is empty, and `MORPH_GET` selector 2 reads back zero on any reachable state; the kernel does contain a decoder, `load_coupling_from_mem`, meant to populate a morphism's coupling from a serialized block in VM memory, but nothing in the current opcode semantics calls it, so it is unused code rather than an active loading path. The relational-composition and concatenation proofs above hold at the level of the `CouplingData` type for arbitrary coupling lists, including states a caller could

construct directly in Coq; they are not, at present, proofs about data any reachable VM trace produces. The graph-presence separating witness used elsewhere in this section (morphism existence, not coupling content) does not depend on nonempty coupling data and is unaffected by this gap.

Two states can agree on the selected classical projection and ledger while differing in their morphism maps. An observer restricted to that projection cannot distinguish them. The VM can query its graph via `MORPH_GET`; a conventional encoding that retained the graph could also support such a query.

Opcode	Effect	Cost
<code>MORPH(r_d, s, t, c, δ)</code>	Create morphism $s \rightarrow t$, write ID into r_d . Empty coupling. (c carried but unused.)	δ
<code>COMPOSE(r_d, m_1, m_2, δ)</code>	Compose $m_1 : A \rightarrow B$ with $m_2 : B \rightarrow C$ (requires target of m_1 = source of m_2). Coupling = relational composition.	δ
<code>MORPH_ID(r_d, m, δ)</code>	Identity morphism on module m .	δ
<code>MORPH_DELETE(id, δ)</code>	Delete morphism id . Error if not found.	δ
<code>MORPH_ASSERT($id, prop, cert, \delta$)</code>	Assert morphism id exists with property $prop$. Sets <code>csr_cert_addr</code> = <code>checksum(prop)</code> on success. Always charges $S(\delta) \geq 1$.	$S(\delta)$
<code>MORPH_TENSOR(r_d, m_1, m_2, δ)</code>	Tensor $m_1 \otimes m_2$: requires disjoint source/target regions. New morphism from union-source to union-target. Coupling = concatenation.	δ
<code>MORPH_GET(r, id, sel, δ)</code>	Read field (0=source, 1=target, 2=#coupling-pairs, 3=is-identity) of morphism id into r .	δ

`MORPH_ASSERT` deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the 51-opcode realisation it is enforced by the step semantics and tied back to the abstract theorem.

10.8 Why 51 opcodes?

The number 51 comes from what the kernel needs to do:

- Compute arithmetic and data: 14 ops (`LOAD_IMM`, `XFER`, `ADD`, `SUB`, `MUL`, `AND`, `OR`, `SHL`, `SHR`, `LUI`, `XOR_LOAD`, `XOR_ADD`, `XOR_SWAP`, `XOR_RANK`)
- Memory and control flow: 8 ops (`LOAD`, `STORE`, `JUMP`, `JNEZ`, `CALL`, `RET`, `HALT`, `CHECKPOINT`)
- I/O, heap, and tensor: 7 ops (`READ_PORT`, `WRITE_PORT`, `EMIT`, `HEAP_LOAD`,

HEAP_STORE, TENSOR_SET, TENSOR_GET)

- Partition structure: 4 ops (PNEW, PSPLIT, PMERGE, PDISCOVER)
- Logic and certification: 3 ops (LASSERT, LJOIN, MDLACC)
- Witness and certification (general plus the full CHSH check family): 8 ops (CHSH_TRIAL, CERTIFY, REVEAL, CHSH_LASSERT, CHSH_LASSERT_1AB, CHSH_LASSERT_1AB_G5, CHSH_LASSERT_1AB_G345, CHSH_LASSERT_1AB_G12345)
- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)

$14 + 8 + 7 + 4 + 3 + 8 + 7 = 51$. Of the 51, 47 are synthesized into the FPGA bitstream; the four Q_{1+AB} check opcodes (CHSH_LASSERT_1AB and its three γ -extended variants) live in the Kami HW abstraction with kernel-equivalence proven, but not in the synthesized bitstream (see the “CHSH check family” note just below for the silicon-budget reason).

The CHSH check family: CHSH_LASSERT plus four Q_{1+AB} extensions. The base opcode CHSH_LASSERT μ_delta reads the eight buckets of `vm_witness` directly, computes the integer-arithmetic check `column_contractive_check_witness` (defined using only $\mathbb{Z}$ arithmetic on the same/diff counts), and either advances the program counter on success or traps to LASSERT_TRAP_PC and latches `vm_err` on failure. The cost is $S(\mu_delta) \geq 1$ regardless of outcome. Mandatory-floor discipline: you pay to check, even to fail. The bridge theorem `chsh_lassert_no_trap_implies_quantum_realizable` proves that any no-trap, no-err-flip execution of CHSH_LASSERT implies the witness-derived 5×5 NPA moment matrix is PSD (i.e., quantum-realizable at NPA level 1). The chain in plain order: the integer check on the witness counters passes; that fact lifts to real-valued column-contractivity by `column_contractive_check_witness_sound`; column-contractivity is biconditional with NPA-PSD by `column_contractive_iff_quantum_realizable`. Three theorems. No physics input. The opcode is the place in the kernel where the algebra of Section 16 becomes something the VM actually refuses to do, not just declines to recommend.

The four Q_{1+AB} extensions CHSH_LASSERT_1AB (tag 0x2F, $\gamma = 0$), CHSH_LASSERT_1AB_G5 (tag 0x30), CHSH_LASSERT_1AB_G345 (tag 0x31), and CHSH_LASSERT_1AB_G12345 (tag 0x32) lift the same discipline from the level-1 5×5 matrix to the level-1+AB 9×9 NPA moment matrix. Each runs its own integer-arithmetic Schur-cascade check on `vm_witness` plus the supplied γ -bucket pairs, advances on pass or traps on fail, charges $S(\mu_delta) \geq 1$. The four bridge theorems (one per slice, named `chsh_lassert_1ab_*_no_trap_implies_quantum_realizable_q1ab`) deliver `quantum_realizable_q1ab = symmetric9  $\wedge$  PSD9` on the witness-derived rationals. The mathematical detail of the four-slice Schur cascade is in §16’s Q_{1+AB} subsection

below.

The instruction set separates ordinary register and memory operations from graph operations, witness recording, and designated charged checks. `CERTIFY` activates `vm_certified`; `MORPH` constructs morphism relationships; `CHSH_TRIAL` records binary-game outcomes; and the `CHSH` assertion family tests the corresponding arithmetic constraints. Their costs and effects are specified in `VMStep.v`.

11 What a categorical morphism is: the category theory, from scratch

Category theory from scratch. I didn't know what it was when I started this project.

11.1 The idea of a category

A category is the part of mathematics that takes the relationships between things as seriously as the things, and then quietly decides the relationships matter more. It is built from two ingredients. There are objects, the things. And there are arrows between them, the relationships. That is the entire inventory, and it looks thin until you notice what got promoted. In most of mathematics the objects are the citizens and the maps between them are second-class traffic. A category turns that around. The arrows are first-class, and nearly everything you can say inside a category you say by pointing at arrows, not by reaching inside objects.

The objects can be anything at all, and that is at once why category theory reaches as far as it does and why it feels slippery the first time through. Sets can be the objects. So can vector spaces, or topological spaces, or any kind of mathematical thing that has structure-respecting maps between its members. The theory does not care what the objects are made of. It cares only how they connect. In the realisation here the objects are modules, the named pieces a partition cut the state into, and the arrows are the relationships between those pieces.

Arrows go by a second name, morphisms, and the two words are interchangeable; I use both. An arrow runs from one object to another, written $f : A \rightarrow B$ and read " f , from A to B ." The thing everyone imports from school and has to set down here is the assumption that an arrow is a function. It does not have to be. An arrow is whatever the category accepts as a structure-respecting way to get from A to B , and in the realisation here it is something more pointed than a function. A morphism is a claim: the assertion that these two modules are related, and here is the type of the relationship.

Two laws must hold:

1. **Composition:** If $f : A \rightarrow B$ and $g : B \rightarrow C$, there must exist a composite $g \circ f : A \rightarrow C$. This is what `COMPOSE` implements.
2. **Identity:** For every object A , there must exist an identity arrow $\text{id}_A : A \rightarrow A$. This is what `MORPH_ID` implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because relationships have to chain. A to B , B to C , therefore A to C . Without that, every multi-hop path has to be named individually, and the structure collapses into a pile of unconnected pairs. A category without composition is a phone book without a search function.

Why identity? Every object needs a self-loop, even the boring ones. A module with no connections to anything else still has to exist somewhere in the framework. `MORPH_ID` is the instruction for that: it builds an $A \rightarrow A$ morphism, stores an empty coupling, and flags it `is_identity`. That flag is load-bearing, and here is why it has to be. An empty coupling would *annihilate* under relational composition (compose anything with the empty relation and you get the empty relation back, not the thing you started with), so the identity cannot be carried by the coupling data. It is carried by the flag, which `COMPOSE` honors: composing an identity-flagged arrow with f returns f 's coupling unchanged, so $\text{id}_A ; f = f$ and $f ; \text{id}_B = f$ hold at the coupling level (`graph_compose_morphisms_coupling`; the matching hardware path is `driven_step_compose`). Drop it and isolated objects fall outside the framework entirely. The category does not see them.

Why associativity? Parenthesization should not matter. $(h \circ g) \circ f$ and $h \circ (g \circ f)$ both spell the same three-hop path $A \rightarrow B \rightarrow C \rightarrow D$. If they came out different, two programs that built the same chain in different orders would produce two different morphisms. That is a bug. The chain is the thing. How you assembled it is not.

These laws are proven in Coq. Concretely: morphisms carry coupling data, a list of (source-cell, target-cell) pairs plus a label. A “cell” is a natural number indexing into a module’s memory region. A coupling pair (a, c) says: cell a in the source module is related to cell c in the target module. The coupling is the *relational content* of the morphism, the actual evidence of which parts of one module correspond to which parts of another. `COMPOSE` produces a new morphism whose coupling pairs are the relational composition of the two inputs: (a, c) is included when there exists an intermediate cell b such that (a, b) is in the first coupling and (b, c) is in the second. Standard relational composition. Matrix multiplication for sets of pairs. The one exception is the identity short-circuit from the previous subsection: if either input is flagged `is_identity`, `COMPOSE` returns the other input’s coupling unchanged, which is what makes the identity law hold. This is proven (`graph_compose_morphisms_coupling`). The categorical layer is not just called a category. It satisfies the actual category axioms on the actual coupling data. I checked that in Coq, because the identity law holding “obviously” is exactly the kind of thing my eye waves through right before it turns out to be false.

11.2 Why modules as objects

In the concrete kernel, the objects of the category are the modules of the partition graph, and that choice is less arbitrary than it looks. A module is a named piece of the computation, the place where a partition cut the state and put a fence around it. Drawing that fence is the act that creates an address. Once a piece has a name it owns a region of memory or state, it may carry certified axioms, it carries a local 4×4 μ -tensor, and, the part that earns it the job, it becomes a thing the rest of the structure can point at. That is the whole reason modules are the bookkeeping unit: a relationship has to attach to something, and the only somethings on offer are the named pieces. An unnamed piece, as far as the category is concerned, is not there at all. There is nothing to relate it to and nothing to relate it from. So everything structural hangs off a module not by convention but because a module is the only kind of thing the category can see.

A morphism between two modules is a claim with its evidence stapled on: “there is a named relationship between A and B , and here is the explicit coupling data that says which parts line up.” That is the move that turns the relationship into a first-class object instead of a remark in a comment somewhere. `COMPOSE` chains two of these by the relational composition already spelled out above, and an identity-flagged input still falls to the short-circuit from before; I will not re-derive either one here. What is worth stopping on is that the relationship does not dissolve into the modules it connects once it is built. It stays in the graph as a thing you can hold and ask questions of: via `MORPH_GET` (selector 2) you read back the coupling-pair count of any morphism, after the fact, straight off the state. The relationship is not lost in the act of relating. It is its own object, carrying a count you can go and demand back.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which transitions/outputs correspond)

The kernel enforces the relational composition correctly. The interpretation is yours.

11.3 Why this is not just “extra state”

Here is the question I chewed on for a long time. If you can just bolt extra registers onto a classical machine and stash morphism IDs in them, why does the categorical layer matter at all?

The answer is in the `MORPH_ASSERT` opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the VM checks: does morphism 42 actually exist in the graph, put there by a `MORPH` or `COMPOSE` instruction

that genuinely ran? `MORPH_ASSERT` looks the morphism up by ID and fails (setting `vm_err`) if no such morphism is there. Existence is all it verifies; the source and target are not arguments it checks against, though you can read them back separately with `MORPH_GET` (selectors 0 and 1). And `MORPH_ASSERT` costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain by calling `MORPH`, `COMPOSE`, and `MORPH_ID` is free when those instructions carry $\delta = 0$. Those ops don't have a mandatory positive floor. What costs is `MORPH_ASSERT`: the act of asserting that the morphism exists and activating the cert-address channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot assert it into the cert-address channel for free.

The graph is explicit structural state. A trusted trace records its updates and the cost of an assertion attempt. `MORPH_ASSERT` checks existence and stores a checksum label; it does not validate arbitrary property text or the supplied certificate. The separation theorem identifies the information the selected shadow discards.

12 Categorical separation: what the classical shadow drops

Section 3 separates the transition law from the observer's projection. This section proves a concrete graph distinction invisible in a selected six-field observation. The result concerns this map, regardless of whether a different representation could preserve the graph.

Theorem 12.1 (Categorical Separation). *There exist states s_1 and s_2 such that:*

- $s_1.vm_regs = s_2.vm_regs$ (identical registers)
- $s_1.vm_mem = s_2.vm_mem$ (identical memory)
- $s_1.vm_mu = s_2.vm_mu$ (identical μ)
- $s_1.vm_err = s_2.vm_err$ (identical error flag)
- $s_1.vm_pc = s_2.vm_pc$ (identical program counter)
- $s_1.vm_certified = s_2.vm_certified$ (identical top-level certification flag)

yet their morphism graphs differ. In the concrete witness, one state has an identity morphism in `pg_morphisms`; the other has no morphisms.

Construction. Build s_1 with `pg_morphisms = [(0, id)]`. Build s_2 with `pg_morphisms = []`. Set every shadow field the same: registers, memory, μ , PC, error flag, and `vm_certified`. The equality proof is by reflexivity on those fields. The graph distinction is by list constructor mismatch: a one-element morphism list is not the empty list. $\square$

Two witness states, side by side. What the six-field shadow sees. What the graph

state actually holds.

	State s_1	State s_2
vm_regs	[]	[]
vm_mem	[]	[]
vm_mu	0	0
vm_pc	0	0
vm_err	false	false
vm_certified	false	false
pg_morphisms	[(0, id)]	[]

What this witness shows. The two states above are not contrived. They are the simplest possible six-field shadow collision under the Thiele projection: identical registers, identical memory, identical PC, identical error flag, identical μ , identical vm_certified. Every observer restricted to that projection says they are the same state. The morphism graph says they are not. This is a witness inside the model that the chosen projection drops information. It is not, on its own, a substrate-comparison theorem: an analogous projection on any other substrate reproduces the same shape. What anchors this in the rest of the argument is the cost-floor theorem plus the irredundancy and law-versus-checker results Section 3 lays out. The uniqueness/initiality theorem in that company gives the richer extension; substitution-adequacy and irredundancy, with the fact that A2 can be a step-rule law instead of an after-the-fact check, give the priority.

What the witness does not prove on its own. The obvious move is to reach for more room: bolt on a register, a tape region, any auxiliary slot, and surely the Turing machine wriggles out. It doesn't. It rebuilds the same witness one size up. The augmented machine still has a non-identity projection (drop the added slot from the projected fields) and the same collision shape on the augmented states. The collapse the shadow lemma exhibits happens no matter how much room you give it, because it is baked into the construction.

The deeper question (whether you can extend a Turing machine with a cost-tracking instruction so the substrate itself enforces A2) is the substrate-versus-program distinction. Section 3's subsection "The structural axis is canonical, not orthogonal" makes this precise once and explicitly. The short version: a Turing machine with extra tape and a longer instruction set is still a Turing machine. A buggy program on the augmented TM can still flip the cert bit without writing the cost increment. The kernel theorems require the cost-on-certification rule to live in the substrate's step function, not in the running program. The Turing substrate does not satisfy that requirement, no matter how the program above it is written.

Connecting to the universal NoFI. Combine this within-model witness with `universal_nfi_any_substrate` and `thiele_represents_simulating_cert_`

system (both) and you get the substrate-level statement. Take a certification system that satisfies A2 *and* supplies a structure-preserving translation into the Thiele VM, packaged as a `SimulatingCertificationSystem`, the record that carries the translation together with a proof it commutes with steps. Its certifications then transport: an uncertified-to-certified trace costs at least 1 in the source system, and the embedded execution lands in a Thiele-certified state. The translation is a hypothesis the record supplies, not something the theorem conjures for an arbitrary A2 system; the only embedding the kernel builds outright is the Turing one (`thiele_simulate_s_turing`), and the cost floor for an A2 system with no embedding at all is the separate, unconditional content of `universal_nfi_any_substrate`. Unconditional here means the axiom closure is empty, nothing named or assumed; the theorem's own two premises are just the trace endpoints, given a run that starts uncertified and ends certified. The witness theorem shows what the Thiele projection drops within the model. The substrate-independent theorem shows that any honest substrate inherits the μ -price on certification. Together they say: the cost floor is forced wherever A2 is accepted. Thiele is the canonical place where the dropped information sits as primitive state.

What would falsify this section. The whole section rides on one separating function existing nowhere. A function on the six shadow fields (`vm_regs`, `vm_mem`, `vm_mu`, `vm_pc`, `vm_err`, `vm_certified`) that returns different values on s_1 and s_2 above would make `shadow_strictly_lossy` false, and the Coq proof would stop compiling. The witness pair is small enough to check by hand. That is the seam this section breaks at, and the place to go looking for it is Section 22.

Theorem 12.2 (Shadow Separation). *There exist two states with the same classical shadow but different morphism graphs, and a legitimate graph-preserving probe after which the morphism-level distinction remains.*

Proof. Take s_1, s_2 to be the Categorical-Separation witness pair: s_1 has `pg_morphisms` = `[(0, id)]`, s_2 has `pg_morphisms` = `[]`, and the six classical-shadow fields agree. So `shadow_proj(s1)` = `shadow_proj(s2)` but `s1.vm_graph` $\neq$ `s2.vm_graph`.

The probe is the single-instruction program `[instr_add0000]` (ADD with $\delta = 0$). This instruction is classical: `is_classical_opcode(instr_add0000)` = `true`. By D3 conservativity, the probe leaves `vm_graph` untouched on both s_1 and s_2 . After the probe, the six classical-shadow fields agree (because they agreed before and the probe is deterministic on the shadow) and the morphism graphs still differ. $\square$ $\square$

| Plain reading. Same witness pair as Categorical Separation. The probe is a classical opcode that touches only the classical shadow. D3 says it leaves the morphism graph alone. So the graph-level difference survives the probe. The point: the

difference between s_1 and s_2 is not an artifact of the initial state. It persists through ordinary classical computation.

The probe used is deliberately boring: `ADD 0 0 0 0`. It is not a morphism operation. That is the point. The theorem is not relying on a failing probe to manufacture a difference. The difference is already in the structural state, and ordinary classical projection cannot see it.

Corollary 12.3 (Classical observer is blind). *No function of (registers, memory, μ , PC, error, certified) can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 12.4 (Thiele strictly extends classical). *Every program in the classical fragment, meaning every program restricted to `is_classical_opcode`, leaves `pg_next_id` unchanged. There exist Thiele programs that change `pg_next_id` via `PNEW`. Therefore Thiele strictly extends the classical fragment: it can represent and probe states that no program in that fragment can reach. A universal Turing machine can simulate the full Thiele state by encoding it on tape, but a faithful simulation has to preserve the μ -ledger cost. Drop the accounting and it is no longer simulating the substrate honestly.*

The embedding and projection results specify which observations are preserved and which structural distinctions are omitted. A projection that identifies different graphs cannot recover the original graph on every state. This does not rule out an encoding that preserves the full state, nor does it make the named concrete VM opcodes necessary in every model.

13 The knowledge receipt: accounting and structural witnesses

The example distinguishes a graph entry from a register containing its ID, and shows how the trace records the charge for an assertion attempt. It does not authenticate arbitrary external receipts or establish the truth of arbitrary property text.

Act 1: The attempt costs, even when it fails.

Run `MORPH_ASSERT(k, "test", "", 0)` where morphism k does not exist. What happens:

- `vm_err` becomes true (the assertion failed, morphism k not found).
- `csr_cert_addr` stays 0, the cert-address channel is NOT activated on failure.
- μ increases by $S(0) = 1$ regardless (the attempt cost something).

The VM charges the failed existence check. The guarantee is specific to `MORPH_ASSERT`: morphism existence is checked, property text supplies a checksum label, and the certificate string is not validated.

Act 2: Build the chain.

Run this sequence:

$$\begin{aligned} & \text{PNEW}(A), \text{PNEW}(B), \text{PNEW}(C) \\ & \text{MORPH}(f, A, B), \text{MORPH}(g, B, C) \\ & \text{COMPOSE}(h, f, g) \end{aligned}$$

Then $\text{MORPH_GET}(r_0, h, \text{source})$ and $\text{MORPH_GET}(r_1, h, \text{target})$. The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \rightarrow C$ exists in the morphism map. With $\delta = 0$ on each build step, $\mu = 0$. The morphism exists, but no cert-address commitment has been paid for yet.

Act 3: Assert.

Run $\text{MORPH_ASSERT}(h, \text{"composed"}, \text{"", } 0)$. Morphism h exists and the property string "composed" has a non-NUL byte. The assertion succeeds. `csr_cert_addr` goes to the ASCII checksum of "composed" (nonzero). Cert-address channel activated. μ goes up by $S(0) = 1$.

(Why does the property string need a non-NUL byte? Because `csr_cert_addr` stores the ASCII checksum of the property string, a plain byte-sum. The empty string sums to 0, and so would an all-NUL string, leaving the channel at 0. Not activated. Any string with a non-NUL byte gives a nonzero checksum and works. "composed" is just an example label.)

The trace records a successful existence check and its cost. The ledger cannot decrease under the VM transition rule. The total alone does not determine which assertion occurred or where; those facts require the trace. This accounting invariant is distinct from authentication against an adversary supplying a different history.

Act 4: Equal observations can hide different graphs.

Snapshot Program A at the end of Act 2, before Act 3's MORPH_ASSERT . Its six-field classical shadow is fixed: MORPH and COMPOSE wrote the new morphism IDs into their destination registers and MORPH_GET read A and C back into r_0 and r_1 ; the program counter sits at 8; $\mu = 0$; error clear; uncertified.

Program B forges that shadow without building anything. It never runs MORPH or COMPOSE . It just LOAD_IMMs the same values into the same registers (the morphism IDs are deterministic, handed out in order (1, 2, 3) by `pg_next_morph_id`, which boots at 1, so there is nothing secret to reproduce) and pads with $\delta = 0$ CHECKPOINT no-ops until its program counter also reads 8. Now every field the shadow keeps matches Program A's, $\mu = 0$ on both sides. A classical observer cannot tell them apart. That two states can share the entire shadow while differing in the graph is the

Categorical Separation Theorem (Section 12); Program B is the forged half of exactly such a pair.

The whole difference lives in the graph. Program A built morphism h with `MORPH` and `COMPOSE`, so h sits in its `pg_morphisms`. Program B's register h holds the number 3, but its graph is empty: the value is a forged token, not a graph entry. Apply `MORPH_DELETE(h)`: Program A succeeds, because there is a morphism to delete. Program B fails and sets `err`, because there is not. `MORPH_DELETE` reads the graph, and the graph is the one thing `LOAD_IMM` cannot forge.

The ledger is the sum of the instruction schedule. The graph carries structural distinctions that the selected shadow omits. The trace supplies provenance when the verifier's interface provides a trustworthy execution history.

14 The substrate, instantiated three times

Three scaffolding layers wrap the substrate: a Coq kernel, an OCaml runner extracted from it, and a Kami RTL design tied back to it. None of them IS the substrate. They are how the substrate gets to live on a chip, on a laptop, and inside a proof checker. The kernel-to-RTL boundary is closed by formal bisimulation proofs (Section 2: same input, same output, field by field). The kernel-to-OCaml boundary is closed by audited parity tests (Section 2: run both on a battery of inputs, verify they agree). Proof on the hardware side. Tested correspondence on the runner side. Both boundaries are named in the source, out in the open, so you can walk to either one and see what it actually rests on. I would rather hand you the exact seam where proof stops and tested correspondence starts than dress the whole stack up as one seamless theorem it is not.

14.1 Layer 1: The Coq kernel

Coq is where the substrate gets pinned down. The kernel defines:

- The `VMState` record (all 12 fields, as in Section 9).
- The `vm_instruction` type: an inductive type (Section 2) with 51 constructors, one for each opcode, each carrying the right argument types.
- The `vm_apply` function: takes a state and an instruction, returns a state. It is total (Section 2): every input pair produces a defined output, including the error case.
- All the theorems: No Free Insight, μ -monotonicity, μ -initiality, categorical separation, Turing strictness, and the rest.

The Coq kernel operates mostly over natural numbers, but `VMState` is not an unbounded mathematical fantasy. `REG_COUNT = 16`. `MEM_SIZE = 128`. Both are matched against the synthesised RTL. The accessors wrap indices through those sizes (modular arithmetic, Section 2). Both constants are parametric in the proofs (Section 2:

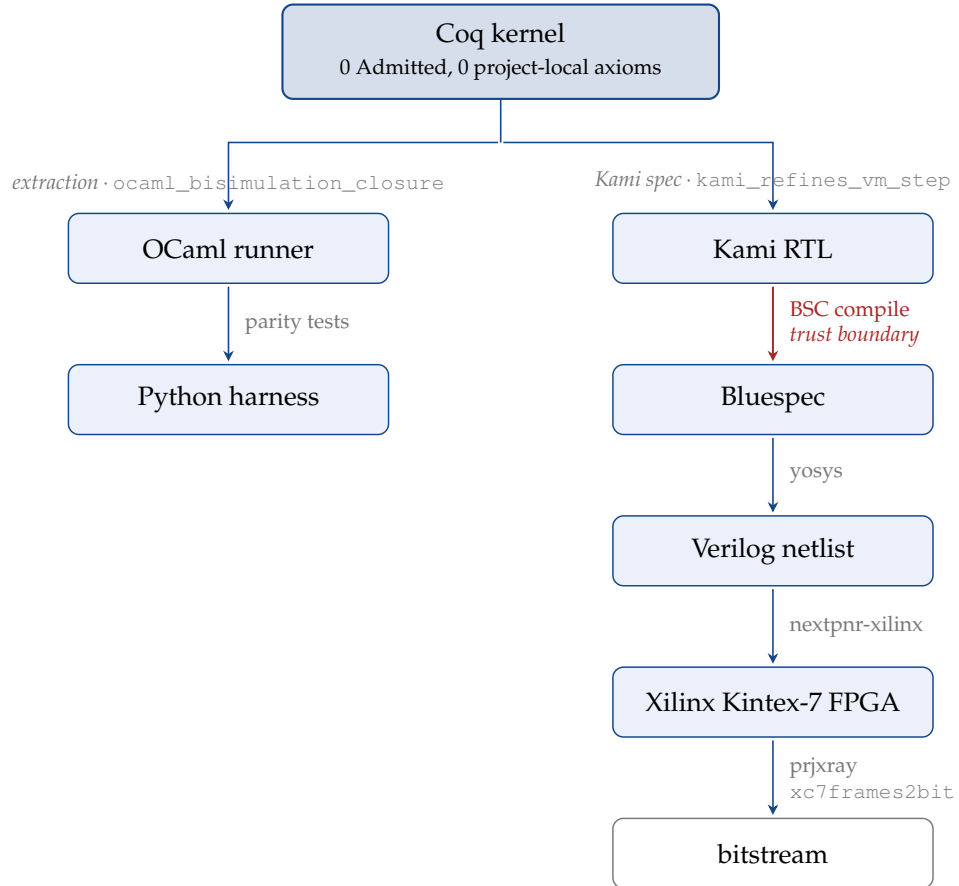


Figure 5: The three-layer realisation. The Coq kernel is the source of truth; the left branch is the runnable-binary path, the right branch is the silicon path. Blue arrows are either formally bisimulated (with the Coq theorem named on the edge) or empirically parity-tested. The red BSC-compile arrow is the single residual trust boundary on the silicon path, named `bsc_kami_compilation_trusted`.

same proofs, any value), so nothing in the formal chain depends on the specific numbers. Register and memory writes are truncated through the kernel’s 64-bit word helpers to keep the math clean. The μ counter is a plain natural number with no fixed upper bound. The RTL instantiates a finite 32-bit datapath (the part of the chip that moves and operates on data, not the control logic) and bounded tables.

Arithmetic near a word boundary is where a bridge claim can quietly stop meaning what it says, so state it concretely. `ADD` on the register values $2^{32} - 1$ and 1: 32-bit modular arithmetic wraps the sum to 0; the kernel’s 64-bit word truncation instead gives 2^{32} , a different value. This is not a hypothetical; it is exactly the gap between the RTL’s 32-bit datapath and the kernel’s 64-bit word helpers named in the paragraph above. Values whose sum stays under 2^{32} agree on both sides; values whose sum crosses that boundary do not, and the bridge domain has to say which side is authoritative for such a value or exclude it. The μ counter’s own range needs the same treatment rather than an unqualified “unbounded natural on both sides” reading: the kernel’s μ is a plain unbounded natural number, but the RTL ledger is a fixed-width register, so a program that accumulates enough μ to exceed that width is outside what the current hardware correspondence establishes, the same way an oversized `ADD` is.

Zero Admitted means: no step in any theorem proof was asserted without derivation. Coq checked everything. I did not.

14.2 Layer 2: The OCaml extracted runner

Here is the trick that makes the OCaml layer worth having at all. A Coq definition is two things wearing one coat: a recipe for computing an answer, and a proof that the recipe behaves. Extraction takes the coat off. It keeps the recipe, in this case rendered into OCaml, an ordinary programming language you can compile and run like any other, and it drops the proof terms on the floor, because a CPU has no use for them. So `vm_apply` (the function that defines what every opcode does) walks out the door as an OCaml function that computes the same answer on the same inputs. The proofs do not come along. The algorithm does. That is the whole point: I get a binary I can actually run, descended from the exact definition the theorems were proved about, and the descent is mechanical rather than me retyping the semantics into a second language and praying. The extracted runner executes all 51 opcodes from the Coq semantics, morphism operations and `CHSH_LASSERT` included.

The trust boundary at this layer is named explicitly. The kernel proves formal facts about the Coq-side observable (`shadow_to_eo (vm_apply s i)` strips a state down to its observable fields after one instruction): exact μ accounting, μ never decreases, the function is total. It also names the external runner boundary. The Coq lemma `ocaml_runner_agrees` is a Coq-side reflexive witness (a proof that one Coq expression equals itself, useful as a placeholder), not an unproved `Hypothesis` (Section 2). The cross-language claim, that the built OCaml binary returns the same observable as the Coq semantics on every input, is checked by the parity test suite

(Section 2: run both on the same battery, verify they agree). I do not count that as a kernel theorem. It is an audited boundary, backed by tests, named in the source.

14.3 Layer 3: The Kami RTL hardware

The whole reason this layer exists is to refuse to retype the design in a second language. Kami lets you write a hardware specification inside Coq itself and then extract it, the same move the OCaml runner used, except the target is not software, it is a chip. The first step is Bluespec SystemVerilog, a hardware description language pitched high enough that you say what the hardware should compute rather than which wire goes where: the registers it holds, the rules it runs, and the conditions under which each state transition is allowed to fire. The Bluespec compiler then drops a level and emits standard Verilog RTL, the gate-and-register description that synthesis tools chew into actual logic, either on an FPGA, a chip you can reconfigure after it leaves the factory, or an ASIC, a chip burned for one design and never reprogrammable. So a Kami module is the same idea descending one more flight of stairs: the finite hardware realisation, pinned down in Coq, walks all the way to gates without anyone hand-translating it on the way, and the Kami extractor runs it through the pipeline above.

The hardware has:

- 16 registers, each 32 bits wide (`RegCount = 16`; kernel proofs parametric)
- 128-bit instruction encoding (`InstrSz = 128`)
- 128 words of data memory and 128 instruction words (`MemSize = 128`; kernel proofs parametric in `MEM_SIZE`)
- bounded instruction transport and execution state
- 64 partition module slots (`PTableSz = 64`)
- 16 slots in each of the morphism, coupling, formula, certificate, descriptor-metadata, and coupling-pair tables
- An on-chip `LASSERT` FSM for formula verification (with 64-entry inlined backing buffers)
- The μ ledger updated atomically with each executed instruction (no separate parallel datapath; the cost arithmetic happens inside the same `step` rule)

The Verilog is the hardware artifact. The FPGA is whatever I could put it on for free. The current target is the Xilinx Kintex-7 `xc7k325tffg900-2` on the Digilent Genesys 2: the smallest fully-open-toolchain part the design still fits comfortably after the multi-cycle `CHSH_LASSERT` witness checker landed. Section 2 explains the synthesis pipeline I use: `yosys` does the gate-level synthesis, `nextpnr-xilinx` does the place-and-route, and `prjxray`'s `xc7frames2bit` produces the bitstream the chip reads to configure itself.

The CI (Full) workflow defines synthesis, simulation, and FPGA-generation jobs. His-

torical resource, timing, and bitstream reports describe their recorded RTL and tool versions. They do not measure a changed Kami source merely because its Coq proofs compile. For this revision, regenerated RTL, synthesis resources, timing, and bitstream claims require fresh pipeline artifacts linked to the source identity in Section 20.1.

The kernel proofs are parametric in `REG_COUNT` and `MEM_SIZE`, so the same proofs apply at any size constants the design picks. This part was a proof of existence on hardware, not a design constraint. The size constants (16 registers, 128 data words, 128 instruction words) were chosen small enough to fit on a free-toolchain FPGA. The design scales to whatever size you actually want.

47 of the 51 opcodes are implemented in the synthesized RTL: the Kami core has hardware logic for those 47, and the synthesised Verilog carries them through to the FPGA bitstream. The remaining four (the Q_{1+AB} check-opcode family `instr_chsh_lassert_lab`, `instr_chsh_lassert_lab_g5`, `instr_chsh_lassert_lab_g345`, `instr_chsh_lassert_lab_g12345`) are defined in the Kami HW abstraction with kernel-equivalence proven, and they execute in the OCaml runner and Python harness, but they are not synthesized into the FPGA bitstream because the silicon budget for additional wide-arithmetic FSMs is exhausted by the base `CHSH_LASSERT` check. The OCaml/RTL opcode-set parity test tolerates exactly these four. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof*.

The theorem `KamiHW.GraphReconstructionBridge.driven_step_wf` compares the Gallina function `kami_step` with `Kernel.SimulationProof.vm_apply` through `abs_full_snapshot`. Its premise is `WFDDrivenPrecondition ks i`. The trace theorem `driven_trace_commutes` requires `WFDDrivenRun fuel trace ks`, which checks that premise at every visited state. This is the intermediate Coq model surface, separate from physical clock transitions in `ThieleCPUCore.v`.

- The default branch of `WFDDrivenPrecondition` is `True`. It adds no instruction-specific premise to this intermediate model equality.
- `CALL` and `RET` require `WellFormedSnapshot`, with a PC bound for `CALL`. `CHSH_TRIAL` requires Boolean operands. `LASSERT` requires agreement with its memory header. Tensor access requires valid indices, and `TENSOR_GET` also requires module existence.
- `MORPH` requires `extended_hw_invariant` and existing source and target modules. `COMPOSE`, `MORPH_TENSOR`, and `MORPH_GET` require the extended invariant. `MORPH_ID` uses the empty-descriptor invariant and module existence. `MORPH_ASSERT` and `MORPH_DELETE` require morph-table well-formedness.
- `PNEW` requires a nonempty normalized region and fresh tensor state. `PSPLIT`

and PMERGE carry the partition, capacity, freshness, and arithmetic conditions in their source branches. These per-operation conditions must be established along the actual run; reset alone does not provide unlimited table capacity.

The actual CPU rules also have executable Kami semantics. `CoreRules` places all twelve rules in the supported linear-action fragment; `CoreExecution` proves that finite traces of its first-enabled selector are actual Kami `MultiStep` executions, with a firing bound and an early-stop characterization by absence of enabled rules. `Reset` has an enabled selected rule. `CoreTyping` proves preservation of the complete register-name/kind schema from reset, while `DispatchReset` states thirteen typed initialization facts. These results establish selected execution and register types. They do not establish fairness, reachable allocation/resource invariants, the full abstract observation relation, or semantic preservation through the RTL toolchain.

Finite RTL transfer additionally requires compatible word ranges, memory addressing, allocation capacity, reset state, and observation at instruction retirement. For example, adding $2^{32} - 1$ and 1 yields zero in a 32-bit register and 2^{32} in the kernel's 64-bit arithmetic. Exact numerical equality therefore needs a range restriction or an explicitly proved modular abstraction. The finite ledger similarly needs a no-overflow bound for exact natural-number equality. Multi-cycle operations require a retirement relation that accounts for intermediate FSM states.

The coupling FSM reads serialized MORPH pairs and implements pair copying and relational joins for the other two operations. Descriptor zero remains reserved for empty/identity couplings. The finite pair table holds 16 entries; insufficient space raises an error, and empty results need no pair entries. The RTL regression tests inspect the backing pair tables through the testbench. They do not establish full-state refinement: hardware lacks the software coupling-label field and module-membership filter, and the full endpoint/region representation contract remains an obligation for a broader physical correspondence claim. The RTL normalizes the newly allocated pair range, retaining each pair's last occurrence as the software's `nodup` does. Allocation first requires room for the raw intermediate list; a normalized result fitting the table does not guarantee admission when the raw list exceeds it.

The inventory arithmetic `rtl_coverage_partition` is not a proof of these representation obligations. The 47 synthesized / four nonsynthesized distinction describes the implementation inventory; it does not turn a conditional model theorem into unrestricted RTL equality.

And the four `Q1+AB` check opcodes are not stranded by sitting outside that partition: their kernel-equivalence proof makes them sound at the substrate level (see §16), and they run in the OCaml runner and Python harness. What they lack is a synthesized-Verilog counterpart to bisimulate against, which is why they fall outside the 47 rather than inside it as a gap. That is honest proof accounting, not a hole papered over.

The substrate is the math. The Coq kernel pins it down. The hardware is a finite physical instantiation with explicit table sizes and word widths. The proofs state the bridge between them under the finite representation invariants where those invariants are needed. This is the only honest relationship physical hardware can have with an abstract mathematical substrate.

Substrate-level vs program-level enforcement. In the Coq kernel, A2 is part of the step relation. A2 says: flipping certification false-to-true costs at least 1, at the step level. Look at the cases for `CERTIFY` and `MORPH_ASSERT` in `vm_apply`: the cost is added before the flag flips; the step that mutates the certification channel and the step that charges μ are the same step. That is what A2 means in the kernel.

A correct conventional simulator can preserve the VM invariant on its encoded states. A different or buggy simulator may fail to do so; that does not establish an incapacity theorem about conventional computation. The VM theorem constrains its own transition relation. Hardware transfer additionally requires the representation conditions of `WFDrun` and the documented extraction and compiler trust boundaries.

Enforcement, representation, and observation must be assessed separately. The abstract A2 theorem allows arbitrary state types and designated predicates. Exact unit pricing is a stronger condition relative to certification-flip count; A2 itself permits larger charges. Neither result authenticates an external receipt without an evidence interface.

15 Five minimal models inspired by other disciplines

The abstract records leave state and instruction types open. The following five examples instantiate them with deliberately minimal models inspired by independently developed disciplines. Coq checks the models and their consequences; correspondence with deployed protocols remains a modeling claim requiring separate evidence.

15.1 Proof-of-stake finality

The formal finality example uses a Boolean finalized flag and a nonnegative stake-at-risk charge on a finalizing transition. A zero-charge transition from unfinalized to finalized violates its A2 obligation. A schedule charging at least one unit on that transition satisfies the obligation. This model records exposure to loss. It does not assert that collateral was actually destroyed, that successive exposures are distinct spent units, or that every deployed consensus protocol uses this mechanism.

In this model, `nothing_at_stake_is_free_forgery` excludes an A2 instance for the supplied free-finalization witness. Under the slashing-schedule premise, `slashing_finality_floor` bounds the sum of modeled charges along an initially unfinalized, finally finalized run by at least one. Applying this result to a deployed protocol requires a correspondence between its transitions and the model.

Falsifier: a transition system satisfying the stated slashing-schedule and execution premises with an initially unfinalized, finally finalized trace whose modeled charge sum is zero.

15.2 Gas metering

The gas example defines a unit commitment counter. `gas_schedule_exactness` characterizes exact pricing relative to its designated commitment predicate: floor and no-overcharge together identify the charging predicate and unit price. A deployed gas schedule also prices resources such as computation and storage. Its full fee is not identified with this counter.

The model separates missing mandatory charges from overcharging the reference counter. `undercharged_opcode_admits_free_commitment` concerns a committing step charged zero. `overcharge_breaks_exactness` concerns the no-overcharge condition. The VM has an exact state-dependent commitment pricer, `thiele_vm_commit_pricing_is_exact`, bounded above by its full ledger schedule via `thiele_unit_price_lower_bounds_mu`. Repeated CERTIFY instructions illustrate the distinction: each pays the instruction floor, while only a false-to-true transition contributes to the flip counter.

Falsifier: a schedule satisfying floor and no-overcharge whose charging predicate differs from the commitment predicate on some reachable step.

15.3 Trusted-execution attestation

The attestation model enriches a report with a measurement field. Under the stated coverage and evidence assumptions, `attestation_cannot_factor_through_bare_transcript` excludes a sound and complete verifier using only the specified bare transcript. `replay_is_a_two_preimage_witness` supplies the model's collision, and `measurement_enriched_attestation_succeeds` supplies a verifier for its enriched reports. This construction requires distinguishing evidence. It does not force a particular physical register or authenticate an adversarially supplied measurement.

Falsifier: a sound and complete attestation scheme for a μ -dependent claim that reads only the bare transcript.

15.4 Certificate transparency

The transparency example supplies a log-like evidence field within the hardness escape model. `transparency_log_escape` inherits that model's acceptance guarantee and trust assumptions. `log_free_verifier_impossible` excludes the specified bare verifier under the collision and coverage premises. `split_view_witness` is a pair in this model. Identifying these fields with a deployed certificate-transparency protocol requires a separate implementation and security argument.

Falsifier: a sound and complete log-free verifier with the same soundness target.

15.5 Proof-carrying verification

The proof-carrying example supplies evidence in a transcript field called `rounds`. `proof_of_rounds_escape` verifies that field under its evidence contract; `bare_pcc_impossible` concerns removing the distinguishing evidence. A noninteractive certificate does not become an interactive protocol because of this field name. Separately, `level_k_verification_floor` bounds the modeled cost of k charged certification events, with a tightness witness in `level_k_verification_floor_tight`. This event count differs from the single-instruction charge levels in Section 17. Neither count measures prover time, circuit size, or proof length.

Falsifier: a level- k certified trace with total μ below k , or a sound and complete zero-round bare verifier.

15.6 The same table, five ways

System	Escape used	Kernel result	Model consequence		What refutes it
PoS finality	none, the cost records directly	<code>universal_nfi_any_substrate</code>	modeled finalization charge ≥ 1	final-charge	a zero-stake gadget admitting A2
Gas metering	none, a pricing biconditional	<code>exact_commitment_pricing_characterization</code>	charge $\equiv$ commit, unit price		a conforming non-commitment schedule
TEE attestation	substrate (enrich the state)	<code>V_does_not_factor_through_classical</code>	distinguishing evidence		sound+complete bare-transcript attestation
Transparency logs	hardness	<code>hardness_escape_succeeds</code>	the supplied evidence		sound+complete log-free verifier
Proof-carrying / ZK	interaction	<code>interactive_escape_succeeds</code> , <code>level_k_certification_cost_floor</code>	evidence, and k units for k charged events		a level- k trace under $k\mu$

These five constructions establish consequences of their explicit records. Deployment interpretations require independent evidence. The examples do not establish that certification is the unique event every model must price; that broader selection question remains open.

16 Binary Game Statistics: where the μ -Ledger Meets the NPA Boundary

The substrate has a CHSH step. The VM realises it with two opcodes. `CHSH_TRIAL` records outcomes of a 2-player binary game into hardware witness counters. `CHSH_LASSERT` refuses to advance normally unless those counters satisfy a Z-arithmetic check.

The enforcement doesn't care who built the wall. `CHSH_LASSERT` refuses a super-quantum correlation by trapping to `LASSERT_TRAP_PC`; `LASSERT` refuses a formula

whose dual witness fails to exclude a single valuation by trapping to the same address, the same way. One wall is the Tsirelson bound, which I did not author and could not have. The other is a logical constraint I wrote down myself. The step traps the same on both. So μ isn't pricing contact with external structure as some special case. It prices the certified narrowing as such, and the trap fires the same whether the state it throws out is a PR box or a falsifying assignment I handed it. Insight costs, and the price never asks whether you found the wall or built it.

And the claim is about that step, not the chip. Some of these cert-opcodes never reach the synthesised FPGA. That's a silicon budget, not a hole in the argument: the trap is the cost-on-certification law, and the law lives in the transition, not the gate count.

Everything below earns that. The matrix algebra is the discharge: it shows the wall `CHSH_LASSERT` traps against is real and externally fixed, the Tsirelson boundary, not a bar I set myself. I came at it sideways, stress-testing the μ -ledger's cost algebra against a known algebraic boundary, the NPA moment matrix conditions for what correlations are achievable under polynomial constraints.

I did not set out to find what I found. The chain is concrete. The Z-arithmetic check the kernel runs on the witness counters (decidable, integer-only) is sound with respect to the real-valued column-contractive predicate by `column_contractive_check_witness_sound` (188 lines, closing with `nra` after IZR distribution and field simplification). That real-valued predicate is biconditional with PSD of the specified zero-marginal, zero-within-party-moment matrix by `column_contractive_iff_quantum_realizable`. End-to-end: a successful `CHSH_LASSERT` step entails NPA-PSD on the witness-derived correlators, by `chsh_lassert_no_trap_implies_quantum_realizable`. No quantum-mechanical premises live in the Coq dependency tree: no Hilbert space, no operators, no observables. The identification of NPA-PSD with quantum-realizable correlations lives in the NPA framework itself, cited externally (Navascués–Pironio–Acín 2007, 2008) as standard. The kernel's content is the algebraic bridge between an integer-arithmetic check inside an opcode and PSD of a specific matrix; calling that condition "quantum-realizable" uses NPA's own terminology, not a physics derivation inside the kernel.

Direction of derivation: substrate first, NPA second. Two passes, in this order. First pass, substrate-side: the `CHSH_TRIAL` opcode, its witness-counter cost schedule, the integer Z-arithmetic predicate `column_contractive_check` that `CHSH_LASSERT` runs on those counters, and the soundness of that integer check against its real-valued lift. None of this references NPA. The coherence condition the μ -ledger enforces at `CHSH_LASSERT` is: a successful witness check requires the counters to satisfy three explicit polynomial inequalities on cleared integer ratios. That predicate is defined inside the cost algebra and verified by the kernel without consulting the quantum-mechanics literature. Second pass: structural agreement. The substrate-native predicate, lifted to the reals via IZR and the witness-count denominators, is the column-contractive

predicate; the column-contractive predicate is biconditional with PSD of the specified zero-marginal, zero-within-party-moment matrix. The biconditional is an agreement between two independently-defined objects (the cost algebra’s gate condition and NPA’s quantum-realizability condition), proven in pure polynomial arithmetic with no project-local axioms. The load-bearing result is §16’s “Algebraic characterization of the μ -ledger gate condition” subsection. The earlier subsections (Bell test setup, classical bound, quantum bound) are the walk there. If you have time for one subsection, that is the one. The CHSH game is built from scratch in the next subsection so the result makes sense to readers who have not seen Bell tests before; readers who have can skip ahead.

16.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($x \in \{0,1\}$). The referee sends Bob one of two questions ($y \in \{0,1\}$). Alice answers $a \in \{0,1\}$. Bob answers $b \in \{0,1\}$. (Settings (x,y) , outcomes (a,b) : same convention as Section 9 and the Coq.)

The game rule: they want $a \oplus b = x \wedge y$. The symbol $\oplus$ is XOR (exclusive OR): $a \oplus b = 1$ when $a \neq b$, and 0 when $a = b$. The symbol $\wedge$ is AND: $x \wedge y = 1$ only when both $x = 1$ and $y = 1$, otherwise 0. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations (both 0, or one each), they should match.

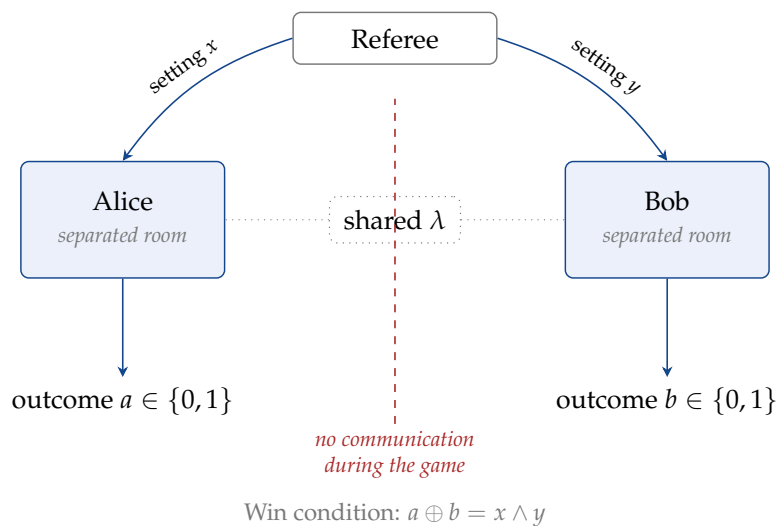


Figure 6: The CHSH setup. The referee distributes settings x, y ; Alice and Bob each have a separated room and may share an agreed-on λ from before the game started. No communication during the game. Each produces a binary outcome (0/1 on the wire, recoded as ± 1 for the correlators). The four correlators $E_{xy} = \langle ab \rangle$, read in the ± 1 encoding, are what the kernel’s CHSH_LASSERT instruction checks against the column-contractivity / NPA condition.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any

deterministic strategy: a function $a(x, \lambda)$ for Alice and $b(y, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy and freeze λ . Then Alice has two predetermined answers, a_0 and a_1 , and Bob has two predetermined answers, b_0 and b_1 . Winning all four question combinations would require:

$$a_0 \oplus b_0 = 0, \quad a_0 \oplus b_1 = 0, \quad a_1 \oplus b_0 = 0, \quad a_1 \oplus b_1 = 1.$$

Now XOR the four left sides. Every variable appears twice, so the total is 0. XOR the four right sides and the total is 1. That is impossible. So every deterministic local strategy loses at least one of the four cases, and the best deterministic strategy wins at most 3 out of 4 question combinations.

And you cannot buy your way past it with a coin flip. A randomized strategy is nothing but a bag of deterministic ones with a die deciding which to pull, and every strategy in the bag already tops out at 75%. Average a pile of things that each sit at 75% or below and the average sits at 75% or below; there is no arrangement of losers that sums to a winner. That is linearity of expectation, the most boring theorem in the building, quietly closing the last door.

So the classical ceiling is $3/4$, seventy-five percent, and not a hair more. And I do not want you to take that on the strength of an XOR trick on a page; the reference kernel pins it down both ways and leaves nothing Admitted. One executable trace, built from nothing but `PNEW`, `PSPLIT`, and `CHSH_TRIAL` and never spending a cent of the cost ledger ($\mu = 0$), walks right up to 75% and touches it. Then `local_box_CSHSH_bound` closes the lid from the other side: no local strategy, however randomized, no factorizable box anywhere, gets over the line. A trace can of course just *write* any tallies it likes into the counters at $\mu = 0$, the way anyone can write a fake scoreboard; the theorem is about what local play can earn, and the certification step that would stand behind a scoreboard is the priced one. The achievable and the unbeatable land on the same number, which is the only kind of ceiling I trust.

16.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ , Alice's output $A_x(\lambda) \in \{-1, +1\}$ (I'm using ± 1 encoding now, not 0/1, indexed by Alice's setting x) and Bob's $B_y(\lambda) \in \{-1, +1\}$ (indexed by Bob's setting y). Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1 , either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ .

Average over λ :

$$S = \langle A_0 B_0 \rangle + \langle A_0 B_1 \rangle + \langle A_1 B_0 \rangle - \langle A_1 B_1 \rangle$$

where $\langle A_x B_y \rangle$ is the correlation (expected value of the product) when settings are x and y . Since the expression before averaging was bounded by 2 for every λ , the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2 , forcing the whole thing below 2. The inequality $|S| \leq 2$ is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

None of that lives only in my head. The argument I just walked you through is the English proof; the formal check that I did not fool myself anywhere is `local_box_CH SH_bound`, which says the plain thing in symbols: for any factorizable box B , $|S(B)| \leq 2$, with nothing left Admitted. And a bound from above is only half an honest claim. `classical_bound_achieved` hands over the other half, a constructive witness, an executable trace at $\mu = 0$ that actually reaches $|S| = 2$ rather than just respecting it. Put the two together and the ceiling is not loose: $\max\{|S| : B \text{ factorizable}\} = 2$, exactly, no slack to argue about, and a $\mu = 0$ trace reaches it.

16.3 The quantum bound: why $2\sqrt{2}$?

Now hand the same game to quantum mechanics and the ceiling lifts. Not to anything you please, and not all the way to a perfect 4; it lifts to $2\sqrt{2} \approx 2.828$ and stops there as hard as the classical 2 stopped. That number is the Tsirelson bound, named for Boris Tsirelson, who was the first to write down $|S| \leq 2\sqrt{2}$ and pin quantum correlations under it. The physics story of *why* it is that exact number, and not some other irrational nobody would have guessed, runs through Hilbert-space operators, anticommutators, the Pauli matrices, the Bloch sphere, the whole machinery. It is a real and beautiful derivation and I am going to walk straight past it, because none of the formal content here leans on it. I do not need to retell quantum mechanics to show you the kernel respects the wall; I only need the algebra, and that is what I am keeping.

What the formal content does depend on, and what I will derive in the next subsections, is the algebra. The Coq theorem about quantum realizability takes “the correlation is quantum-realizable in the zero-marginal NPA framework” as a premise (Section 2 builds the term) and concludes $|S| \leq 2\sqrt{2}$. The premise is the physics input; the conclusion is pure algebra. The “What the cost algebra proves” subsection below then builds the same bound from polynomial inequalities on the NPA moment matrix without any quantum-physics premise at all. That is the path I follow, and that is the part I am claiming.

And that gap between 2 and $2\sqrt{2}$ is not bookkeeping. It is the most stubborn fact in physics: people have gone into labs, fired entangled photons through polarizers, counted coincidences, and watched the number sit up above 2 where no local classical story is allowed to put it. Whatever the world is doing, it is not Alice and Bob with a shared cheat sheet agreed beforehand. The physics of *why* the wall sits at $2\sqrt{2}$ is somebody else’s chapter and a good one. My question is narrower and, to me, stranger: why does the kernel’s CHSH check respect a wall it was never told about? The polynomial arithmetic below answers it. Feed the cost ledger nothing but its own algebra, no quantum mechanics anywhere in the room, and the same $2\sqrt{2}$ falls out the bottom on its own.

A note on the rational approximation in the kernel. The kernel names the rational threshold $5657/2000 \approx 2.8285$ for exact rational comparisons (you can verify $(5657/2000)^2 = 32,001,649/4,000,000 > 8$, so $5657/2000 > \sqrt{8} = 2\sqrt{2}$). The real-valued quantum-model bridge uses $\sqrt{8}$ directly. Those are two different proof surfaces, and I keep them apart on purpose: the rational one so a machine can check it in exact arithmetic, the real one so the quantum bridge has something to bite on.

16.4 What the cost algebra proves

Theorem 16.1 (Classical CHSH Bound). *For population correlations with a setting-independent shared-variable distribution, Alice’s local response depending on x, λ and Bob’s on y, λ , $|S| \leq 2$. The constant strategy attains equality. Its four-setting VM witness has zero charge when every trial uses $\delta = 0$.*

Proof. $|S| \leq 2$: the XOR-factoring argument from Section 16’s “Why 75% is the classical ceiling.” Fix any local deterministic strategy and any λ . Alice produces $A_x(\lambda) \in \{-1, +1\}$; Bob produces $B_y(\lambda) \in \{-1, +1\}$. The quantity

$$\begin{aligned} A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda) \\ = A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda)) \end{aligned}$$

satisfies $|\cdot| \leq |B_0 + B_1| + |B_0 - B_1| = 2$ (because for $B_0, B_1 \in \{\pm 1\}$, one of $|B_0 + B_1|$ and $|B_0 - B_1|$ is 2 and the other is 0, and $|A_0|, |A_1| \leq 1$). Average over λ to get $|S| \leq 2$. Mixtures of deterministic strategies are convex combinations, so the same bound holds

by linearity of expectation.

Tightness: the strategy $A_x \equiv 1, B_y \equiv 1$ gives $\langle A_0 B_0 \rangle = \langle A_0 B_1 \rangle = \langle A_1 B_0 \rangle = \langle A_1 B_1 \rangle = 1$, so $S = 1 + 1 + 1 - 1 = 2$. This is achievable at $\mu = 0$ by a program with all $\delta = 0$ instructions accumulating one trial per setting pair in `vm_witness`. The Coq witness is `classical_bound_achieved`. $\square$ $\square$

| Plain reading. Factor the expression. Two terms. One has $B_0 + B_1$, the other has $B_0 - B_1$. With $B_0, B_1 \in \{\pm 1\}$, only one can be nonzero at a time, and the nonzero one is ± 2 . Multiply by $|A| \leq 1$. The whole expression caps at 2. Average over λ , you get $|S| \leq 2$. To make it 2, just have both players always answer 1.

Theorem 16.2 (Non-locality of an exact CHSH violation). *An exact correlation $\mathbf{E} = (E_{00}, E_{01}, E_{10}, E_{11})$, with $S = E_{00} + E_{01} + E_{10} - E_{11}$ and $S > 2$, cannot arise as the population-level correlation of any local hidden variable model (the term is built in Section 2: a classical-physics-style theory where outcomes come from shared randomness λ plus local settings). This is a claim about the idealized correlator, the value an unbounded run of trials at each setting converges to. It is not yet a claim about any one finite tally of recorded trials; that distinction is spelled out below.*

Proof. Contrapositive of the Classical CHSH Bound. Any LHV model produces a locally factorizable correlation by definition: each player’s outcome is a function of their setting and the shared λ . The theorem above says all such correlations satisfy $|S| \leq 2$. So an idealized correlation with $S > 2$ cannot have arisen from any LHV model. $\square$ $\square$

| Plain reading. Direct contrapositive. If $|S| \leq 2$ for the population-level correlation of every LHV, then a population-level $S > 2$ rules out every LHV. The Bell-test contradiction is in this form, and it is a claim about the number an unbounded run converges to, not the number on any one finite scoreboard.

Remark 16.1. A finite scoreboard is not the population correlation. A `WitnessCounts` tally from a finite run of `CHSH_TRIAL` instructions is a sample statistic, `chsh_stat_from_wc`, not the idealized correlator the theorem above bounds. A local hidden variable model can draw a fresh λ per trial, and a short run can land away from its own population mean by chance. Run one trial at each of the four settings, with Alice and Bob each answering an independent fair coin flip, an honest LHV strategy with population correlation zero at every setting: the event “same at (0,0), same at (0,1), same at (1,0), different at (1,1)” has probability 1/16 and produces the finite tally $S = 4$, the algebraic maximum, with no communication and no quantum content anywhere in the construction. This is the same finite-sample distinction Zhang, Glancy, and Knill work out in full for Bell tests (*Efficient quantification of experimental evidence against local realism*, arXiv:1303.7464): a population bound of $|S| \leq 2$ is not, by

itself, a claim that every finite sample drawn from a local model also satisfies it.

What the kernel proves about a finite tally is narrower, and it does hold: `chsh_violation_rules_out_locally_factorizable_coupling` shows that a tally with $S > 2$ is inconsistent with any single fixed deterministic strategy (a_0, a_1, b_0, b_1) replayed identically at every recorded trial (`locally_consistent_classical_bound` is the converse direction: a tally consistent with one fixed deterministic strategy has $|S| \leq 2$). That rules out one specific, narrow explanation of the tally. It does not rule out a randomized local model that redraws λ trial by trial, which is exactly the kind of model the coin-flip example above uses to reach $S = 4$ honestly. Reading a certified witness violation as physically informative rests on running enough trials to make the fixed-strategy count and the population count converge, not on an identity between one finite tally and the exact-correlation theorem above.

Theorem 16.3 (Tsirelson upper bound for the fixed completion). *If the correlators satisfy PSD of the specified zero-marginal, zero-within-party-moment completion, their CHSH value satisfies $|S| \leq \sqrt{8} = 2\sqrt{2}$.*

Proof. The fixed-matrix biconditional gives column contractivity. It bounds each row norm of E by one. Apply Cauchy–Schwarz to $S = (E_{00} + E_{01}) + (E_{10} - E_{11})$: each parenthesis has magnitude at most $\sqrt{2}$ times its row norm. Thus $|S| \leq 2\sqrt{2}$. This proof uses the specified matrix, without an identification with arbitrary zero-marginal operator models. □ □

| Plain reading. The formal fixed-completion predicate is equivalent to column contractivity. The resulting row bounds give the CHSH inequality by Cauchy–Schwarz. Arbitrary quantum behaviors and empirical local samples have separate definitions.

Theorem 16.4 (General algebraic Tsirelson bound). *For every algebraically coherent correlator (one satisfying $|E_{xy}| \leq 1$ along with four constraints derived from 3×3 submatrices of the NPA moment matrix; the submatrix determinants, called minors, must be non-negative, which is part of the requirement that the full matrix be positive-semidefinite; both terms are built in full a few pages below, in the “Algebraic characterization” subsection), and given some witness parameters t and s that the kernel builds explicitly, the CHSH value satisfies $S^2 \leq 8$, that is, $|S| \leq 2\sqrt{2}$. This is the general case: every algebraically coherent correlator, not just the symmetric configuration. The Coq proof uses `psatz` (described in Section 2: a Coq tactic that proves polynomial inequalities by exhibiting a sum-of-squares decomposition). The rational approximation corollary uses $5657/2000$ as an exact rational overestimate of $2\sqrt{2}$ (you can verify $(5657/2000)^2 = 32,001,649/4,000,000 > 8$), allowing the bound to be checked in exact rational arithmetic without floating-point. No physics premises. Pure polynomial arithmetic. The Tsirelson bound $2\sqrt{2}$ is derived from algebra, not assumed.*

Proof. The four minor-positivity conditions on 3×3 submatrices of the NPA moment matrix are: each principal 3×3 minor's determinant is non-negative. Working through the determinant formulas with the moment-matrix structure of Section 16, the four constraints become four polynomial inequalities of the form $1 - E_{xy}^2 - E_{x'y'}^2 \geq 0$ for the appropriate index pairs (these reduce to the column-contractivity conditions 1 and 2 plus their transposes).

Given these constraints, $S^2 \leq 8$ is a polynomial inequality in $(E_{00}, E_{01}, E_{10}, E_{11})$ subject to four polynomial constraints. The Positivstellensatz (the algebraic certificate for polynomial-positivity) gives a sum-of-squares decomposition: the polynomial $8 - S^2$ minus a non-negative combination of the constraint polynomials is itself a sum of squares. Coq's `psatz` tactic finds and verifies such a decomposition; the explicit witness parameters t, s are coefficients in the SOS combination, computed once and stored in the kernel.

The rational approximation corollary uses $5657/2000$ in place of $\sqrt{8}$ to keep the comparison in $\mathbb{Q}$: $(5657/2000)^2 = 32,001,649/4,000,000 > 8 = 32,000,000/4,000,000$, so $5657/2000 > \sqrt{8}$. Combined with $|S|^2 \leq 8$, this gives $|S| < 5657/2000$, a strict rational bound checkable without floating-point evaluation. $\square$ $\square$

| Plain reading. Four polynomial constraints on the correlators (the 3×3 minor positivity conditions). Goal: show $S^2 \leq 8$ subject to those constraints. Positivstellensatz says: if a polynomial is non-negative on the feasible set, you can write it as a non-negative combination of the constraint polynomials plus a sum of squares. `psatz` finds the combination. The Coq file stores the witness coefficients. The Tsirelson bound is the consequence; no physics input goes into the proof.

Bell's theorem has another face, and it is worth seeing both. The statistical face is the one everyone draws: counts going into a number S and the number refusing to stay under 2. But the same wall shows up one floor down, in the language of couplings, where you are no longer talking about averages at all but about whether two parties can each answer locally and have their answers fit together. A WitnessCount-consistent locally deterministic strategy here is an honest, almost insultingly simple thing: four bits, Alice's answer for each of her two settings and Bob's for each of his, fixed in advance with no peeking. Pin that down and you get a separable coupling, by which I mean one that hands each setting exactly one outcome type, no hedging, no mixing. Any locally factorizable morphism coupling that respects the WCLocallyConsistent predicate is stuck below the bound: $|S| \leq 2$. So read it the other way, which is the way that bites. If a trace ever shows $S > 2$, there is no local strategy of four bits that could have produced it, full stop. The wall is not a fact about statistics that we then dress up in structure; it is already there in the structure, and the statistics are just where you happen to bump into it first.

What does any of this have to do with computation? Everything, and it comes in

through the witness counters and the certification layer. Picture a trace that sits there racking up CHSH trial results through CHSH_TRIAL, tally after tally, costing nothing, because counting is cheap and the universe does not send you a bill for watching it. Then the trace does the one expensive thing: it turns around and certifies that $S > 2$ via CERTIFY or LASSERT. That is the cert-flip, and A2 forces $\mu \geq 1$ on it, no exceptions. So the statistics are free and the claim about the statistics is not, and that gap is the whole point. You can stare at the data all day for nothing; the moment you stand up and say you know what it means, somewhere a counter ticks. Throughout, the witness counters carry Alice's setting x and Bob's setting y , the same indexing the formal layer uses, so this is not a side dialect (Section 9). It is the witness insight taxonomy of Section 6, viewed from the place where physics hands you a free supply of trials and then charges you for the sentence you write about them.

16.5 The bit-commitment requirement

If you want cross-correlations above the classical bound to come out attested, not just observed, the reference kernel makes you spend a disclosure instruction to get them. That falls straight out of how cost is priced: A2 charges the cert-flip, so the only way to reach an attested claim is to take the step that flips the cert, and that step costs. The strictness is downstream of the accounting, not the other way around, and the next few lines walk through exactly where it gets forced.

So here is where that gets concrete. A single CHSH_TRIAL drops a few setting bits and outcome bits into the witness counters, and the cross-correlations $\langle A_x B_y \rangle$ are nothing but those tallies divided through. They are a record of what happened, and the ledger has no opinion about a record. The accounting only wakes up when you point at one of those numbers and say it backs a particular claim. That is not a count any more. It is a stance, and a stance is the one thing the ledger knows how to charge for, which is why the cost lands precisely on the sentence and never on the arithmetic underneath it.

And this part is not a hope, it is settled. There is exactly one instruction in the reference kernel that can push `csr_cert_addr` off zero, and that is MORPH_ASSERT, and even then only when the morphism genuinely exists and the property string it carries is more than an empty NUL byte. So the door to attestation has a single key. Run that backward and it pins the trace: anything ending in `supra_cert` had to have walked through a valid MORPH_ASSERT step on the way, there is no other path that sets the register, and the kernel will not let you fake one. Which is the same wall from a closer angle. CHSH_TRIAL counts pile up for free, all the raw data you like, but the instruction that stands up and certifies what the pile means is the one that costs, and it is the only one that even can.

16.6 Algebraic characterization of the μ -ledger gate condition

If you only have the patience for one subsection of Section 16, make it this one. Everything before it is me showing my work, the honest grind of getting from the cost

algebra to the physics boundary. What lands here is the thing the grind was for: the place where the bookkeeping the machine does and the wall the universe puts up turn out to be the same statement read two ways. The rest of the section earns it. This is what it earns.

Here is the result of stress-testing the cost algebra against the NPA boundary. The column-contractivity conditions arise from the natural polynomial conditions that make the NPA moment matrix well-formed; they were not imported from physics, and the biconditional with NPA-PSD is proved in pure polynomial arithmetic. The following biconditional holds.

Positive semidefinite (PSD). A matrix M is positive semidefinite if for every vector v you can multiply with it, the quantity $v^T M v$ is at least 0. (Here v^T is the transpose of v from Section 2: turning a column of numbers on its side; $v^T M v$ is the matrix product of v^T , M , and v , which is just one number.) The intuition: PSD means the matrix never maps a vector to something pointing “backwards” relative to the vector. For a 2×2 matrix $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$, PSD reduces to three simple conditions: $a \geq 0$, $c \geq 0$, and $ac - b^2 \geq 0$. The first two say the diagonal entries are non-negative. The third says the determinant (Section 2) is non-negative. For larger matrices, the PSD condition generalises: every principal submatrix (every square subblock you get by picking some rows-and-columns at the same indices, see Section 2) must also be PSD.

PSD is the mathematical condition that a matrix can represent genuine inner products, covariances, or correlations from a quantum state. The NPA framework uses PSD of a specific 5×5 matrix (the “moment matrix”) as its level-1 condition for quantum realizability. You build that 5×5 matrix from the four CHSH correlators, and if it is PSD, the correlators are consistent with some quantum state measured with orthogonal observables and zero marginals. Two scope words in that sentence are load-bearing: for this pinned zero-marginal matrix the PSD test is exact for its slice, while the general level-1 NPA test, marginals free, is a relaxation of the quantum set that the hierarchy tightens at higher levels (Section 16.7).

16.6.1 The matrix, written out

Here is the actual 5×5 NPA moment matrix for the zero-marginal CHSH scenario. The five operators are: I (identity), A_0 (Alice’s measurement for setting 0), A_1 (Alice’s measurement for setting 1), B_0 (Bob’s for setting 0), B_1 (Bob’s for setting 1). The (i, j) entry is the expected value of $O_i \cdot O_j$. Zero-marginal means we set $\langle A_0 \rangle = \langle A_1 \rangle = \langle B_0 \rangle = \langle B_1 \rangle = 0$ and the cross-correlation terms $\langle A_0 A_1 \rangle = \langle B_0 B_1 \rangle = 0$.

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix}$$

Rows and columns are indexed 0 through 4, in the order $\{I, A_0, A_1, B_0, B_1\}$. The diagonal is all 1s because each operator O_i satisfies $O_i^2 = I$ when the outcomes are ± 1 -valued: squaring a ± 1 -valued operator gives the identity, so the diagonal entries are all $\langle I \rangle = 1$. The off-diagonal entries E_{xy} are the CHSH correlators with Alice's setting x and Bob's setting y (Section 2). Everything else is zero because the marginals vanish (Section 2: a marginal is the average of one variable with the others integrated out; zero-marginal means $\langle A_x \rangle = \langle B_y \rangle = 0$).

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{bmatrix}$$

Condition 1

 $1 - E_{00}^2 - E_{10}^2 \geq 0$

rows/cols $\{1, 2, 3\}$ (A_0, A_1, B_0)

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{bmatrix}$$

Condition 2

 $1 - E_{01}^2 - E_{11}^2 \geq 0$

rows/cols $\{1, 2, 4\}$ (A_0, A_1, B_1)

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{bmatrix}$$

Condition 3

 $AC \geq B^2, \text{ Schur on } 4 \times 4$

rows/cols $\{1, 2, 3, 4\}$

Figure 7: The 5×5 specified zero-marginal, zero-within-party-moment matrix shown three times, each copy highlighting one principal submatrix whose PSD condition gives one column-contractivity condition. Two come from 3×3 principal-submatrix determinants; the third comes from the Schur complement of the 4×4 block. All three follow from PSD of the parent matrix. Panel 2's $\{1, 2, 4\}$ submatrix is non-contiguous, skipping row and column 3, so its highlight marks the band of rows the condition constrains rather than the exact cells; panels 1 and 3 are contiguous and highlighted exactly.

16.6.2 Where the three conditions come from

The three column-contractivity conditions come directly from PSD of this matrix. Here is the derivation.

Condition 1: $1 - E_{00}^2 - E_{10}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 3\}$ of the 5×5 moment matrix (the rows and columns indexed by A_0, A_1, B_0):

$$\begin{pmatrix} 1 & 0 & E_{00} \\ 0 & 1 & E_{10} \\ E_{00} & E_{10} & 1 \end{pmatrix}$$

Compute the determinant of this 3×3 submatrix by the cofactor expansion along the

first row (Section 2: pick a row, multiply each entry by the determinant of the 2×2 matrix you get by deleting that entry's row and column, with alternating signs, then sum):

$$1 \cdot (1 \cdot 1 - E_{10}^2) - 0 \cdot (0 \cdot 1 - E_{10} \cdot E_{00}) + E_{00} \cdot (0 \cdot E_{10} - 1 \cdot E_{00}) = 1 - E_{10}^2 - E_{00}^2.$$

The principal-submatrix rule for PSD says this determinant must be ≥ 0 . That is condition 1.

Condition 2: $1 - E_{01}^2 - E_{11}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 4\}$ (the A_0, A_1, B_1 block):

$$\begin{pmatrix} 1 & 0 & E_{01} \\ 0 & 1 & E_{11} \\ E_{01} & E_{11} & 1 \end{pmatrix}$$

Same calculation: determinant = $1 - E_{01}^2 - E_{11}^2 \geq 0$. That is condition 2.

Condition 3: $(1 - E_{00}^2 - E_{10}^2)(1 - E_{01}^2 - E_{11}^2) \geq (E_{00}E_{01} + E_{10}E_{11})^2$

This comes from the 4×4 submatrix at rows and columns $\{1, 2, 3, 4\}$. That submatrix has a block structure (an arrangement where the entries are themselves 2×2 matrices):

$$\begin{pmatrix} I_2 & E \\ E^T & I_2 \end{pmatrix}$$

where I_2 is the 2×2 identity matrix (Section 2) and E is the 2×2 correlator matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix}.$$

E^T is the transpose of E from Section 2. A standard linear-algebra fact (the Schur complement criterion, also Section 2) says that a block matrix of this shape is positive-semidefinite if and only if the Schur complement of the upper-left I_2 is positive-semidefinite. The Schur complement here works out to $I_2 - E^T E$. The order of the transpose matters: $E^T E$ puts the squared norms (sums of squares) of the columns of E on its diagonal, while EE^T would put the squared norms of the rows. The right one for this argument is $E^T E$.

Computing $E^T E$ entry by entry:

$$E^T E = \begin{pmatrix} E_{00} & E_{10} \\ E_{01} & E_{11} \end{pmatrix} \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix} = \begin{pmatrix} E_{00}^2 + E_{10}^2 & E_{00}E_{01} + E_{10}E_{11} \\ E_{00}E_{01} + E_{10}E_{11} & E_{01}^2 + E_{11}^2 \end{pmatrix}.$$

The Schur complement is therefore

$$I_2 - E^T E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix} = \begin{pmatrix} A & -B \\ -B & C \end{pmatrix}$$

where $A = 1 - E_{00}^2 - E_{10}^2$ (the left side of condition 1), $B = E_{00}E_{01} + E_{10}E_{11}$, and $C = 1 - E_{01}^2 - E_{11}^2$ (the left side of condition 2). For this 2×2 Schur complement to be PSD, two things must hold: the diagonal entries must be non-negative (which are conditions 1 and 2 again) and the determinant must be non-negative, $AC - B^2 \geq 0$. That last inequality is condition 3.

That is where the three conditions come from: two from 3×3 submatrices, one from the Schur complement of the 4×4 block, each one a principal minor forced non-negative by M being PSD. That derivation runs necessity, PSD of M implies the three conditions. It is not yet sufficiency, and checking these particular minors does not by itself certify every principal minor of the full 5×5 ; a direct argument for the other direction follows.

The forward direction, directly. Write M in block form as $1 \oplus \begin{pmatrix} I_2 & E \\ E^T & I_2 \end{pmatrix}$: the $(0,0)$ entry is 1, the rest of row and column 0 are 0 because the marginals vanish, and the remaining 4×4 block is exactly the one just computed. For $v = (v_0, x, y)$ with $v_0 \in \mathbb{R}$ and $x, y \in \mathbb{R}^2$,

$$v^T M v = v_0^2 + x^T x + 2x^T E y + y^T y = v_0^2 + \|x + E y\|^2 + y^T (I_2 - E^T E) y,$$

the last step by expanding $\|x + E y\|^2 = x^T x + 2x^T E y + y^T E^T E y$ and cancelling the $y^T E^T E y$ term against its copy in $y^T (I_2 - E^T E) y$. The first two terms are non-negative for every v_0 and every x , with no condition on E at all; x is free to range over all of $\mathbb{R}^2$, and $\|x + E y\|^2 \geq 0$ regardless of what value it takes. The entire content is in the third term: $v^T M v \geq 0$ for every choice of v_0, x, y if and only if $y^T (I_2 - E^T E) y \geq 0$ for every $y \in \mathbb{R}^2$, which is exactly PSD of the 2×2 matrix $I_2 - E^T E$ computed above. A 2×2 symmetric matrix is PSD iff its diagonal entries are non-negative and its determinant is non-negative, which is conditions 1, 2, and 3 together. So column-contractivity is equivalent to PSD of $I_2 - E^T E$, which is equivalent to $v^T M v \geq 0$ for every v , which is PSD of M . This is the complete forward direction: three conditions checked once, as the PSD criterion for one 2×2 matrix, not as a claim that a hand-picked list of minors stands in for all $2^5 - 1$ principal minors of the 5×5 .

Why this equals quantum realizability. The reverse direction (proved as `npa_psd_implies_column_contractive`) is that if M is PSD, the correlators must be column-contractive. Here is the argument. Take any PSD matrix M and feed it the specific 5-dimensional vector $v = (0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$, where y_0 and y_1 are free real numbers. The PSD condition $v^T M v \geq 0$, after the matrix algebra, gives a quadratic expression in y_0 and y_1 : $Ay_0^2 - 2By_0y_1 + Cy_1^2 \geq 0$ for all real

y_0 and y_1 . If a quadratic of the form $ay^2 + 2by + c$ is non-negative for every real y , then $a \geq 0$ and $ac \geq b^2$ (the non-negative-discriminant rule from intermediate algebra: a parabola that never dips below the axis has non-positive discriminant, which after sign-flipping reads $b^2 \leq ac$; that one direction is all the argument uses). That closes condition 3. The diagonal conditions (1 and 2) come from setting $y_1 = 0$ and $y_0 = 0$ separately.

A correlation $(E_{00}, E_{01}, E_{10}, E_{11})$ is *slice-coherent* if and only if it is realizable by the specified zero-marginal, zero-within-party-moment completion. Exactly. Not approximately. Not “consistent with some physics axiom.” The name describes the test: membership in the fixed orthogonal completion used by `zero_marginal_npa`. Four ratios can satisfy that test whether their counters came from trials or were written directly. A passing gate earns the algebraic membership claim. It does not establish the history of those counters. The biconditional is pure polynomial arithmetic with no project-local axioms (Print Assumptions on `column_contractive_iff_quantum_realizable` reports only Coq’s standard classical-reals and functional-extensionality `stdlib` axioms, namely `sig_not_dec`, `sig_forall_dec`, and `functional_extensionality_dep`, the same `stdlib` families Section 16.7 discloses for the Q_{1+AB} theorems; “zero axioms” in this section means zero project-local, in the sense of Section 22):

Theorem 16.5 (Slice coherence $\iff$ NPA-PSD).

$$\text{coherent}(\mathbf{E}) \iff \text{psd-npa}(\mathbf{E})$$

where *coherent* means the three column-contractivity conditions hold, and *psd-npa* means the 5×5 specified zero-marginal, zero-within-party-moment matrix is positive semidefinite.

Proof. The full math is in the two derivations of Section 16 preceding this theorem. Recapitulated for the proof block:

Forward direction (column-contractive $\Rightarrow$ PSD). Assume the three column-contractivity conditions hold. The Section 16 derivation showed: condition 1 is exactly the principal-submatrix determinant on rows/columns $\{1, 2, 3\}$; condition 2 is exactly the principal-submatrix determinant on rows/columns $\{1, 2, 4\}$; condition 3 is exactly the Schur-complement determinant of the 4×4 block. Combined with the $\{0\}$ principal submatrix (just the entry $1 \geq 0$) and the diagonal entries (all $1 \geq 0$), every principal submatrix has non-negative determinant and non-negative diagonal, so by the principal-submatrix PSD criterion (Section 2), the full 5×5 matrix is PSD. The Coq witness is `zero_marginal_npa_column_contractive_implies_psd`.

Reverse direction (PSD $\Rightarrow$ column-contractive). Assume the moment matrix M is PSD.

Specialize $v^T M v \geq 0$ at the vector

$$v(y_0, y_1) := (0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$$

for free $y_0, y_1 \in \mathbb{R}$. Computing $v^T M v$ using the explicit moment-matrix entries and collecting terms gives the quadratic form

$$Ay_0^2 - 2By_0y_1 + Cy_1^2 \geq 0 \quad \text{for all } y_0, y_1 \in \mathbb{R},$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. By the quadratic-non-negative-discriminant lemma (`quadratic_nonneg_discriminant`), this is equivalent to: $A \geq 0$, $C \geq 0$, and $B^2 \leq AC$. The three conditions are exactly the three column-contractivity conditions. Specializing further at $(y_0, y_1) = (1, 0)$ and $(0, 1)$ recovers conditions 1 and 2 directly; the general (y_0, y_1) case yields condition 3. The Coq witness is `npa_psd_implies_column_contractive`. $\square$ $\square$

| Plain reading. Both directions are pure algebra on the 5×5 matrix. Forward: each of the three column-contractivity conditions is a principal-submatrix or Schur-complement determinant being non-negative. PSD reduces to all principal-submatrix determinants ≥ 0 , which is exactly what column-contractivity gives you. Reverse: feed the PSD matrix a specific test vector parameterized by (y_0, y_1) . The result $v^T M v$ comes out as a quadratic in (y_0, y_1) . Non-negative-for-all- (y_0, y_1) means the quadratic's discriminant is non-positive, i.e., $b^2 \leq ac$. That's condition 3. The diagonal conditions fall out at $(1, 0)$ and $(0, 1)$. No physics anywhere; both directions are polynomial-arithmetic identities.

The left side is three polynomial inequalities (column contractivity of the specified zero-marginal, zero-within-party-moment matrix):

$$1 - E_{00}^2 - E_{10}^2 \geq 0 \quad (\text{condition 1})$$

$$1 - E_{01}^2 - E_{11}^2 \geq 0 \quad (\text{condition 2})$$

$$AC \geq B^2 \quad (\text{condition 3})$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. These are the three conditions of `zero_marginal_column_contractive` (defined): column 0 norm ≤ 1 , column 1 norm ≤ 1 , determinant $AC - B^2 \geq 0$. The right side says the NPA moment matrix is positive semidefinite. One scope note so this sentence cannot be quoted into a bigger claim than it makes: for this pinned 5×5 matrix, PSD characterizes the orthogonal-observables zero-marginal slice exactly (that is what the biconditional proves, and a Gram decomposition of the PSD matrix hands back the realizing observables), but level-1 PSD in general, with marginals in play, is a relaxation, an outer approximation the NPA hierarchy tightens level by level. Section 16.7 climbs to the level that is exact for CHSH.

The forward direction (`zero_marginal_npa_column_contractive_implies_psd`) was proved first. The reverse direction: given a PSD NPA matrix, specialize it at the vector $(0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$. The PSD condition gives a quadratic-in- y_0/y_1 that is everywhere non-negative. Then `quadratic_nonneg_discriminant` ($\forall t, a + 2bt + ct^2 \geq 0 \Rightarrow b^2 \leq ac$) closes the determinant condition. The diagonal conditions come from $y_0 = 1, y_1 = 0$ and $y_0 = 0, y_1 = 1$. Both directions are packaged as the biconditional `column_contractive_iff_quantum_realizable`.

What this means for the hierarchy of correlations:

- Classical local hidden variables: $|S| \leq 2$.
- Slice-coherent = realizable by the specified zero-marginal, zero-within-party-moment completion (biconditional, both directions, no project-local axioms). Every slice-coherent correlation satisfies $|S| \leq 2\sqrt{2}$, but NOT every correlation with $|S| \leq 2\sqrt{2}$ is slice-coherent: $(1, 0, 1, 0)$ has $|S| = 2$ but fails condition 1.
- No-signaling: no constraint beyond $|E_{ab}| \leq 1$.
- PR box ($E_{00} = E_{01} = E_{10} = 1, E_{11} = -1$): condition 1 gives $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Fails immediately by direct arithmetic.
- Tsirelson bound: `state_column_contractive_implies_tsirelson` proves slice-coherent $\Rightarrow |S|^2 \leq 8$ (i.e., $|S| \leq 2\sqrt{2}$) directly from column-contractivity. The chain factors through PSD: column-contractive $\Rightarrow$ NPA-PSD $\Rightarrow$ row bounds $\Rightarrow$ Tsirelson, and the row-bounds step is `tsirelson_from_row_bounds`. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not slice-coherent) follows immediately. The converse is false and not proved.

The gate checks membership in a specified algebraic set. That is the claim it has to earn. Run the sixteen deterministic local strategies through it: every one of them has all four correlators at ± 1 , so condition 1 reads $1 - 1 - 1 = -1 < 0$ and `CHSH_LASSERT` traps every single one of them, including the all-ones strategy that this section's own tightness theorem (`classical_bound_achieved`) runs at $\mu = 0$ to touch the classical bound. Those tallies are as true as tallies get, the trace really did accumulate those counts, and the gate rejects them anyway, because their correlators fail the fixed orthogonal completion. Marginals alone do not explain this: mixing each deterministic strategy with its simultaneous sign reversal preserves those correlators and gives zero marginals. In the other direction, a program can run a sequence of `CHSH_TRIAL` calls chosen to land the tally inside the slice, rather than earning it by playing the game honestly against an unpredictable referee, and the gate waves it through the same as an honest run; `LOAD_IMM` itself only writes registers, not the `vm_witness` buckets `CHSH_LASSERT` reads, so it cannot forge a tally directly, but a program is free to choose its own `CHSH_TRIAL` inputs and drive the buckets to any reachable value it likes. Whether a tally was earned by play against a genuine referee or walked

there on purpose is the kind of fact the receipt discipline of Section 13 meters, not this opcode. So the correct reading of the boundary is set-theoretic, not moral. The classical polytope and the slice-coherent set intersect, uniform noise $(0,0,0,0)$ sits comfortably in both, but neither contains the other: $(1,0,1,0)$, reachable by a classical mixture and carrying a legal $|S| = 2$, fails condition 1 ($1 - 1^2 - 1^2 = -1 < 0$, closed by `lra` once `column_contractive_iff_quantum_realizable` is destructured), while every point satisfying the fixed orthogonal completion passes. What a successful `CHSH_LASSERT` certifies is that the counters cohere with the zero-marginal slice, no more and no less, and the Tsirelson corollary is inherited from that membership, not from any power to sniff out forged data.

Kernel-level enforcement (the `CHSH_LASSERT` opcode). Here is the thing about a certification bit: by itself it is just a bit. A generic `CERTIFY` can set `vm_certified := true` and never once glance at the witness counters that are supposed to earn it. That is a rubber stamp with the ink already on it, and a rubber stamp is exactly what the whole discipline is built to forbid. So the certification channel cannot lean on `CERTIFY` for this discipline; it needs an opcode that does the looking, that reads the evidence before it is allowed to say the word. That opcode is `CHSH_LASSERT`, and it ties slice coherence as column-contractivity to the certification channel by refusing to take the claim on faith. It reads the `WitnessCounts` buckets, the counts of how the two parties actually came out together and apart, and it computes the three integer-arithmetic conditions `column_contractive_check_witness` in $\mathbb{Z}$ -arithmetic, no real and no rational ever evaluated at runtime, because the moment you let floating point near a soundness check you have smuggled in a place for the answer to drift. If the counts satisfy the three conditions, the program counter advances and the run goes on. If they do not, there is no negotiating: the step traps to `LASSERT_TRAP_PC` and latches `vm_err`, and the incoherent certification never happens. Two theorems close the chain to NPA-PSD; both are stated and proved below.

Theorem 16.6 (Integer-check soundness, `column_contractive_check_witness_sound`). *Let w be the `WitnessCount` record with eight non-negative-integer fields $wc_same_{xy}, wc_diff_{xy}$ for $(x,y) \in \{0,1\}^2$. Define $N_{xy} = wc_same_{xy} + wc_diff_{xy}$. The integer-only check `column_contractive_check_witness` is the conjunction of two parts: the four strict-positivity conjuncts $N_{00} > 0, N_{01} > 0, N_{10} > 0, N_{11} > 0$, and three $\mathbb{Z}$ -arithmetic inequalities on the counts, all written with no division. If the integer check returns `true`, then every N_{xy} is a positive integer, so*

$$E_{xy} = \frac{wc_same_{xy} - wc_diff_{xy}}{N_{xy}}$$

is an ordinary quotient with a positive denominator, and $\mathbf{E} = (E_{00}, E_{01}, E_{10}, E_{11}) \in [-1, 1]^4$ satisfies the three column-contractivity conditions of Section 16. This is a different fact from the general zero-trial convention of Section 2, which sets $E_{xy} = 0$ when $N_{xy} = 0$ so that the

CHSH score S is defined on every witness record. That convention lets S be computed at all; it plays no role here, because a passing check already guarantees no setting pair is empty.

Proof. The four positivity conjuncts are checked first, independently of the three polynomial conditions: the boolean function returns `false` the moment any $N_{xy} = 0$, before the polynomial terms are even compared. A `true` result therefore rules out every empty setting pair by construction. Take the reviewer counterexample $wc_same_{00} = wc_same_{01} = 1$ with the other six counters zero: $N_{10} = N_{11} = 0$, so the positivity conjuncts for settings $(1, 0)$ and $(1, 1)$ fail and the check returns `false` outright, without ever reaching the polynomial comparison. No witness record with an empty setting pair passes.

For the remaining three conditions, multiplying through by N_{xy}^2 to clear denominators gives polynomial inequalities in the non-negative integer counts:

- Condition 1 ($1 - E_{00}^2 - E_{10}^2 \geq 0$) becomes the inequality $N_{00}^2 \cdot N_{10}^2 - (wc_same_{00} - wc_diff_{00})^2 N_{10}^2 - (wc_same_{10} - wc_diff_{10})^2 N_{00}^2 \geq 0$ after substituting E_{xy} and clearing denominators.
- Condition 2 is the analogous inequality with the E_{01}, E_{11} correlators.
- Condition 3 ($AC \geq B^2$ in the notation of Section 16) becomes a polynomial inequality of degree 8 in the integer counts: A 's numerator and C 's numerator are each degree 4, their product is degree 8, and B 's numerator squared is likewise degree 8, after clearing the common denominator $N_{00}^2 N_{01}^2 N_{10}^2 N_{11}^2$.

To prove the conclusion: assume the check returns `true`. The positivity conjuncts give $N_{xy} > 0$ for all four settings, so no case split on $N_{xy} = 0$ is needed; that case cannot arise. Cast the integer counts to $\mathbb{R}$ by $\text{IZR} : \mathbb{Z} \rightarrow \mathbb{R}$, which preserves the inequalities. Divide through by N_{xy}^2 (or N_{xy}^4 , for condition 3), licensed by $N_{xy} > 0$. After dividing, the three inequalities are exactly conditions 1, 2, 3 of column-contractivity in $\mathbb{R}$. The Coq proof closes the post-division algebra by `nra` (nonlinear real arithmetic) acting on a sum-of-squares-shaped goal. $\square$ $\square$

| Plain reading. Integer arithmetic upstairs, real arithmetic downstairs, `IZR` is the elevator. The check throws out an empty setting bucket before it does anything else: $N_{00}, N_{01}, N_{10}, N_{11} > 0$ are load-bearing conjuncts of the boolean function, not a fallback applied after a division that never happens. A witness record with two settings never exercised, all trials piled onto the other two, fails on those two positivity conjuncts and never reaches the polynomial conditions. With positivity in hand, write down the three column-contractivity conditions in terms of the counts, cleared of denominators, and you have polynomial inequalities in $\mathbb{Z}$. Cast to $\mathbb{R}$ via `IZR`; this preserves ≥ 0 . Divide back through by the (strictly positive) total-trial counts; this gives the real-valued column-contractivity conditions. The algebra after the division is just a sum-of-squares-shaped polynomial inequality;

nra closes it. The whole route avoids any floating-point evaluation at any step: integers in, reals out, division at the boundary, and the boundary is never zero.

Theorem 16.7 ($\text{CHSH_LASSERT} \Rightarrow \text{NPA-PSD}$, `chsh_lassert_no_trap_implies_quantum_realizable`). *Let s be a state and let $s' = \text{vm_apply } s \text{ (instr_chsh_lassert } \delta)$. If $s'.\text{vm_err} = \text{false}$ and $s'.\text{vm_pc} = s.\text{vm_pc} + 1$ (i.e., the CHSH_LASSERT step advanced normally rather than trapping), then the specified zero-marginal, zero-within-party-moment matrix M built from the witness-derived correlators E_{xy} of $s.\text{vm_witness}$ is positive semidefinite.*

Proof. By the apply rule for `instr_chsh_lassert`: the step branches on `column_contractive_check_witness` run on $s.\text{vm_witness}$. If the check returns `false`, the step traps to `LASSERT_TRAP_PC` and sets $\text{vm_err} = \text{true}$. If the check returns `true`, the step advances PC by 1 and leaves $\text{vm_err} = \text{false}$. By hypothesis, s' exhibits the second behavior, so the integer check returned `true` on $s.\text{vm_witness}$.

Compose with the preceding theorem (integer-check soundness): the correlators E_{xy} derived from $s.\text{vm_witness}$ satisfy the three real-valued column-contractivity conditions.

Compose with the forward direction of `column_contractive_iff_quantum_realizable` (the biconditional proved in Section 16): column-contractivity implies the NPA moment matrix M is positive semidefinite. $\square$ $\square$

| Plain reading. Three theorems compose: the kernel step rule says no-trap means the integer check passed; the soundness lemma says the integer check passing implies real-valued column-contractivity; the biconditional says column-contractivity implies NPA-PSD. Stack them. A successful CHSH_LASSERT step is a proof that the witness-derived correlators land inside the NPA-PSD set. No physics input anywhere in the stack. The (1,0,1,0) correlation fails the integer check (condition 1 fails immediately: $1 - 1 - 1 = -1 < 0$), so any program that accumulated those statistics traps on CHSH_LASSERT.

Run the (1,0,1,0) correlation through CHSH_LASSERT and watch what the step does with it: the integer check fails on the very first condition, the step refuses to advance, and the state comes out the far side wearing $\text{vm_err} = \text{true}$. Nothing in the kernel was ever told where (1,0,1,0) sits relative to any physics, and the point itself is not unphysical: a classical mixture reaches it with room to spare, marginals and all. The kernel was told to count, clear denominators, and test three inequalities, and for this point the arithmetic simply would not close, because the point lies outside the zero-marginal slice the three inequalities carve out. That is the only gate there is. A trace whose correlators are column-contractive walks straight through; one whose correlators are not gets stopped by its own numbers, and the kernel never has to know

why.

To falsify: exhibit $E_{00}, E_{01}, E_{10}, E_{11}$ satisfying the three polynomial inequalities but with a non-PSD NPA matrix. No such example exists; the theorem proves it impossible. Or exhibit a quantum experiment producing a non-PSD NPA matrix. That would break quantum mechanics, not this theorem.

16.7 Lifting the bridge to level 1+AB: the four-slice Schur cascade

Section 16 establishes the runtime bridge: CHSH_LASSERT implies PSD of a fixed 5×5 moment matrix with zero marginals and zero within-party cross moments. This fixed slice excludes some classical correlators. Allowing the within-party moments to vary gives the full correlator elliptope developed in Section 16.8. The Q_{1+AB} checks instead validate a specified 9×9 moment matrix on the basis $\{I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1\}$ with supplied higher moments. Their soundness theorem establishes PSD for that matrix. For complete behaviors, including marginals, finite-level NPA feasibility is a relaxation: Chaturvedi (2026, arXiv:2607.14569) proves that no fixed finite level describes the whole quantum behavior set in this scenario.

And the two levels are not two separate tricks. The 5×5 and the 9×9 checks are one dimension-polymorphic PSD predicate evaluated at two sizes: top index 4 for the 5×5 , top index 8 for the 9×9 , the same fold either way. The level-1 CHSH biconditional from Section 16 is the dimension-5 instance of it, recovered as a corollary that imports the original 5×5 result as a black box and adds no axioms over it (`column_contractive_iff_general_realizable`). So the lift in this subsection is not a 9-shaped coincidence dressed up as a method: it is the same realizability projection run one dimension up, and the dimension-5 corollary reproducing the bridge you already have is exactly the check that it is one predicate and not two.

16.7.1 The 9×9 moment matrix and its five γ parameters

The level-1+AB moment matrix is parametric in nine reals: the four CHSH correlators ($E_{00}, E_{01}, E_{10}, E_{11}$) and five additional moments

$$\begin{aligned} \gamma_1 &= \langle A_0 A_1 B_0 \rangle, & \gamma_2 &= \langle A_0 A_1 B_1 \rangle, & \gamma_3 &= \langle A_0 B_0 B_1 \rangle, \\ \gamma_4 &= \langle A_1 B_0 B_1 \rangle, & \gamma_5 &= \langle A_0 A_1 B_0 B_1 \rangle. \end{aligned}$$

γ_1, γ_2 are the two three-body A - A - B moments. γ_3, γ_4 are the two three-body A - B - B moments. γ_5 is the four-body moment, and it enters the matrix in two cells that are *not* equal: the $(A_0 B_0, A_1 B_1)$ cell carries $\gamma_5 = \langle A_0 A_1 B_0 B_1 \rangle$, while the conjugate $(A_0 B_1, A_1 B_0)$ cell carries $\langle A_0 A_1 B_1 B_0 \rangle = -\gamma_5$, the same four operators in the opposite B -order, equal in magnitude and opposite in sign because the orthogonal measurements anticommute (the slice models the orthogonal-measurement case $\{B_0, B_1\} = 0$, which also forces the $\langle B_0 B_1 \rangle$ entry to 0, and under it $\langle A_0 A_1 B_1 B_0 \rangle = -\langle A_0 A_1 B_0 B_1 \rangle$). That minus sign is the operator anticommutator a CHSH violation runs on. Writing

the same γ_5 into both cells would assert $B_0B_1 = B_1B_0$, a classical commutativity that forces the certifiable score down to the classical bound 2; the conjugate sign is what lets the slice reach Tsirelson's $2\sqrt{2}$. Each γ_k lives in $[-1, 1]$; on a quantum state with zero marginals, each γ_k is the corresponding expectation. The 9×9 matrix entries are polynomials of degree ≤ 2 in (E_{ij}, γ_k) , given explicitly by the products of basis operators (e.g. entry $(I, A_0B_0) = E_{00}$, entry $(A_0, A_1B_0) = \gamma_1$, entry $(A_0B_0, A_0B_0) = 1$, etc.). PSD of this matrix is the level-1+AB NPA condition.

16.7.2 Why four slices

The substrate certifies four nested γ -slices, each via its own cert-opcode and its own integer-arithmetic check:

- **Slice A** ($\gamma = 0$, tag 0x2F): all five γ_k pinned to zero. The bool decider checks Q_1 column-contractivity (the three conditions of §16) plus the SOS sum-of-squares condition $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$. Closed by an explicit 4-D Cauchy-Schwarz SOS identity.
- **Slice B** (γ_5 free, tag 0x30): the four-body moment γ_5 is admitted as a bucket-pair argument; the other four γ are zero. The check adds a weighted 4-D Cauchy-Schwarz identity that handles the $2\gamma_5(v_{11}v_{22} - v_{12}v_{21})$ cross term arising in the residual (the v_{ij} are the coordinates of the paired 4-D vectors the identity runs over) (the relative minus is the conjugate-cell sign above: γ_5 on the $v_{11}v_{22}$ pairing, $-\gamma_5$ on the $v_{12}v_{21}$ pairing).
- **Slice C** ($\gamma_3, \gamma_4, \gamma_5$ free, tag 0x31): the two A - B - B moments and the four-body moment are admitted; $\gamma_1 = \gamma_2 = 0$. The check is a 4×4 Sylvester PD test on a difference matrix $H_{\gamma_{345}} = \det_M \cdot M_M - M_N$, where M_M and M_N are the two denominator-cleared moment blocks the slice compares and $\det_M$ is the cleared determinant scaling them, derived via an augmented 2×2 Schur-slack lemma.
- **Slice D** (full $\gamma_1, \dots, \gamma_5$ free, tag 0x32): all five γ parameters are admitted. The check is a compositional $\text{sym}6 \rightarrow \text{sym}5 \rightarrow \text{sym}4$ Schur cascade: the only check stiff enough to handle the bilinear A - A - B cross terms γ_1, γ_2 introduce in the residual.

These four are not four independent gadgets; they nest. Each slice admits everything the slice before it admitted and then opens a little more room, so the admitted sets climb $A \subset B \subset C \subset D$, each one strictly larger than the last because each one stops pinning a moment the previous slice held at zero. The reason to build them in exactly this order is that every newly freed moment drags a harder cross term into the residual, and you would rather meet those one at a time than all at once on the first try. Slice D, with nothing pinned, is the whole of the Q_{1+AB} cone, and the three slices below it are not lesser objects: they are that same cone seen with progressively more of the parameters frozen.

16.7.3 The compositional Schur cascade (Slice D)

The mathematical core of Slice D is the chain

$$\begin{aligned}
 \text{PSD}_9(M) &\Leftarrow \text{sym6 PD-interior on the 21 entries of } M_{6 \times 6} \\
 &\Leftarrow \text{sym5 PD-interior on the } 5 \times 5 \text{ Schur complement of row 1} \\
 &\Leftarrow \text{sym4 PD-interior on the } 4 \times 4 \text{ Schur complement of row 2}
 \end{aligned}$$

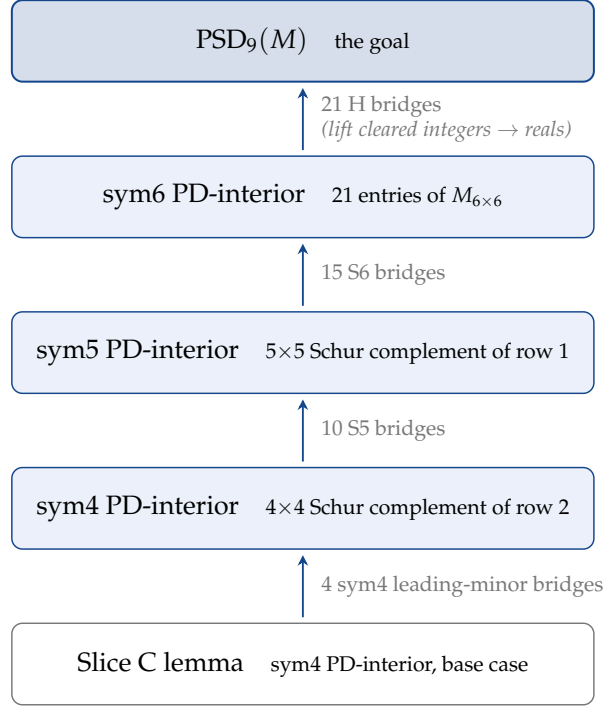


Figure 8: The Schur cascade lifts the base-case sym4 PD-interior fact from Slice C all the way up to $\text{PSD}_9(M)$. Each arrow is one Schur-complement reduction, a polynomial identity in the correlators and moments. The counts beside each arrow are how many such identities the integer check grinds through at that step, one per matrix entry.

where $M_{6 \times 6}$ is what the 9×9 collapses to once the Q_{1+AB} subsumption identities pin its dependent entries: the projection onto the basis $\{v_{B1}, v_{B2}, v_{11}, v_{12}, v_{21}, v_{22}\}$, after which the 9×9 lands in the PSD cone whenever the 6×6 residual does. From there it is Schur complements all the way down, and it is the same move you already watched at level 1, run on a bigger matrix. The Schur-complement criterion (Section 2), the one the level-1 bridge used on its 4×4 block, turns a positive-definiteness question about the 6×6 into one about a 5×5 Schur complement; run it again and the 5×5 becomes a 4×4 ; and the 4×4 closes by Sylvester’s criterion, all four leading principal minors positive. That is the whole soundness argument: a base case and three reductions, each one a Schur step you can carry out by hand, stacked from the 4×4 back up to PSD_9 .

Every reduction is a single polynomial identity in the correlators E_{ij} and the moments γ_k . To keep the opcode’s check decidable and integer-only, each identity is multiplied

through by the common denominator $KZ = (N_{00}N_{01}N_{10}N_{11} \cdot D_{g_1}D_{g_2}D_{g_3}D_{g_4}D_{g_5})^2$, where the N_{ij} are the per-setting trial counts and the D_{g_k} the γ -bucket denominators, and verified on integers, the same clear-the-denominators move as level 1. It is a lot of arithmetic: one identity per matrix entry, twenty-one across the 6×6 , fifteen for the first Schur step, ten for the second, four leading minors at the base. The kernel grinds every one. But the grinding isn't the argument; the argument is the iterated Schur reduction above, and a reader who wants to break it breaks the reduction, not the fifty identities. A passing integer check is those reductions holding, and those reductions holding is M in the PSD cone. The check leans on nothing past the two background facts level 1 already leaned on: the decidability of the reals and functional extensionality.

16.7.4 What this delivers and what it does not

Soundness. For each of the four Q_{1+AB} assertion opcodes, the successful-step condition in its theorem implies symmetry and positive semidefiniteness of the specified 9×9 moment matrix at the witness-derived correlators and supplied γ values. This is an arithmetic certificate about those entries. The theorem does not construct a physical experiment.

Scope. These theorems establish the four integer-check soundness implications and the associated matrix identities. Physical realization, completeness of the checks, and identification of a moment relaxation with a quantum behavior set require their own arguments. The full correlator ellipsope is treated separately below.

Accounting. Each assertion instruction has declared cost $S(\mu_delta) \geq 1$, charged on both success and failure. The integer test establishes its matrix property; the cost schedule records payment for executing the test. These are separate guarantees. The assertion opcodes preserve `vm_certified`; their success is observed through the stated trap and error conditions.

Falsification. Exhibit witness counters and γ bucket pairs for which one of the four cert-opcodes does not trap but the resulting 9×9 NPA moment matrix is *not* PSD, OR show that the Schur-cascade soundness theorem at any slice is incorrect (an SOS identity wrong, a minors-witness step that does not imply PSD9, a bool decider unsound), OR show that the soundness chain depends on a project-local axiom beyond the two standard-library axioms cited above. The `Coq Print Assumptions` report is the audit surface: every theorem either lists no new axiom or it does not pass closeout.

16.8 The full correlator set: the ellipsope completion

The fixed orthogonal completion excludes some classical correlators, including every deterministic local strategy. The full ellipsope allows the within-party cross moments to vary. `EllipsopeCompletion.v` defines membership by existence of a PSD completion and proves inclusion of classical correlators, the Tsirelson bound, and exclusion of the PR box. That is the mathematical object this subsection examines.

The characterization is the one Tsirelson’s theorem forces. Take the same 5×5 moment matrix, marginals still zero, but stop pinning the two cross moments: let x stand for $\langle A_0 A_1 \rangle$ and y for $\langle B_0 B_1 \rangle$, both free. Call a correlator tuple $(E_{00}, E_{01}, E_{10}, E_{11})$ *elliptope-realizable* when *some* completion (x, y) makes the matrix PSD (`elliptope_realizable`, defined in `ElliptopeCompletion.v`). The existential over (x, y) is the entire difference between the slice and the set: the slice asks whether the point is coherent with one fixed completion, the zero one; the set asks whether any completion works.

Five theorems pin the characterization down, all mechanised, all with `Print Assumptions` reporting only the same `stdlib` families as the rest of the CHSH algebra (the classical-reals construction and functional extensionality; zero project-local axioms):

- `deterministic_strategy_elliptope`: every deterministic local strategy is in the set. The witness is the honest one, $x = a_0 a_1$ and $y = b_0 b_1$, the cross moments the strategy actually has, and the completed matrix becomes an explicit Gram matrix whose quadratic form is a sum of two squares. All sixteen sign patterns are instances of one parametric theorem. `classical_tightness_witness_elliptope` names the sharpest instance: the all-ones strategy, the $\mu = 0$ witness of `classical_bound_achieved` that the slice gate traps, is elliptope-realizable.
- `turing_point_elliptope`: $(1, 0, 1, 0)$, this document’s running example of a classically reachable point the slice rejects, is in the set, witnessed by $x = 1$, $y = 0$ and the sum-of-squares decomposition $v_0^2 + (v_1 + v_2 + v_3)^2 + v_4^2$ of its completed quadratic form.
- `elliptope_convex`: the set is closed under binary convex combination; `elliptope_finite_mixture` extends that to arbitrary finite sub-convex mixtures by a mechanised renormalization induction (the weight deficit absorbed by the origin, `elliptope_zero`); and `lhv_mixture_elliptope` composes it with the deterministic theorem: every finite mixture of deterministic sign strategies, which is to say every local-hidden-variable correlator, lands inside. Not by hand-waving from the binary case; the n -ary theorem is in the kernel.
- `elliptope_tsirelson`: everything in the set satisfies $S^2 \leq 8$, i.e. $|S| \leq 2\sqrt{2}$. The proof is worth a sentence, because the fixed-slice row-norm bound does not hold for arbitrary completions. At $y \neq 0$ the row-norm bound fails, and honestly so: the point $(1, 0, 1, 0)$ itself has a row of norm $\sqrt{2}$. What replaces it is Cauchy–Schwarz run inside the PSD form itself (`psd_cauchy_schwarz`, derived from the same discriminant lemma the reverse bridge uses): the two budgets $\|B_0 + B_1\|^2 = 2 + 2y$ and $\|B_0 - B_1\|^2 = 2 - 2y$ are coupled through the completion, their product is at most 4, and $S^2 \leq 8$ falls out of that coupling. The bound holds at every point of the set, not because each row is short, but because the completion cannot make both budgets long at once.

- `pr_box_not_elliptope`: the PR box has no completion; its S is 4 and the Tsirelson theorem refuses it. And `beyond_classical_elliptope` exhibits a rational point with $S = 12/5 > 2$ inside the set, so the containment of the classical polytope is strict: $\text{classical} \subset \text{elliptope} \subset \text{no-signaling}$, with both inclusions proper, from inside the formalization.

The correlator geometry has three distinct sets: the classical polytope and the zero-marginal slice intersect without containment, and both sit inside the elliptope, which admits every honest classical point the slice refuses while still refusing everything past $2\sqrt{2}$.

The completion gate is executable. `ElliptopeGate.v` builds the decidable check an opcode can branch on (`elliptope_check_full`), in the same integer-only discipline as `column_contractive_check_witness`: no real and no rational evaluated at runtime, every denominator cleared into $\mathbb{Z}$. The check has two branches. The PD branch is the cheap path for a strictly-interior point: it takes completion buckets for (x, y) , exactly the bucket-pair idiom the 1+AB opcodes use for their γ moments, and runs the fraction-free Sylvester test on the denominator-cleared completed matrix, whose soundness is the same `sym4_qf_nonneg_from_pd` LDLT lemma the Slice-C development proves. The LDL branch checks certificates that can include singular boundary completions: it takes a rational LDL^T certificate, six subdiagonal entries and four pivots over common positive denominators, and verifies the ten entry equations of $M = LDL^T$ in cross-multiplied $\mathbb{Z}$ -arithmetic. Its soundness (`elliptope_ldl_check_sound`) is a sum-of-weighted-squares identity: given the equations, the quadratic form is $d_1P_1^2 + d_2P_2^2 + d_3P_3^2 + d_4P_4^2$. The file proves soundness of this certificate format and supplies concrete accepted boundary points. The load-bearing theorem is `elliptope_check_full_sound`: a passing check entails elliptope membership of the witness-derived correlators, with the check's own passing as its only hypothesis. And the gate computes, not just proves, each by reflexivity: `gate_accepts_all_ones` runs the $\mu = 0$ tightness witness through the LDL branch (its all-ones completion is rank-one singular) and it passes; `gate_accepts_turing_point` does the same for $(1, 0, 1, 0)$; `gate_accepts_beyond_classical` passes the interior $S = 12/5$ point through the PD branch; `gate_accepts_pythagorean_boundary` certifies $(3/5, 4/5, 4/5, -3/5)$, whose arcsine sum is exactly π by the 3-4-5 triangle so it sits exactly *on* the Tsirelson curve with $S = 14/5 > 2$, a point invisible to the strict PD branch because its completion is singular, reached by the LDL branch with pivots $(1, 1, 0, 0)$ at $x = y = 0$; and `elliptope_full_gate_never_accepts_pr_box` proves no completion buckets and no certificate ever get the PR box through, not by enumeration but by soundness plus the Tsirelson theorem closing both branches at once. This certificate format requires rational completion data. Soundness and the accepted examples do not establish completeness for every elliptope point, or an impossibility theorem about other exact certificate formats.

The runtime `CHSH_LASSERT` opcode checks the fixed orthogonal slice. Connecting `e1`

liptope_check_full to an opcode requires its instruction semantics, cost rule, and hardware implementation. The completion results concern correlators; they do not characterize the full behavior set with marginals. Identifying the completion model with physical quantum correlators uses external mathematics. Producing entanglement in hardware requires physical evidence. To falsify: a deterministic sign pattern whose completed Gram matrix has a negative quadratic form kills deterministic_strategy_elliptope; an elliptope-realizable tuple with $S^2 > 8$ kills elliptope_tsirelson; a PSD completion for the PR box kills pr_box_not_elliptope and Tsirelson with it.

17 Irreversibility accounting and the μ -hierarchy

17.1 The Landauer association is motivation only

Same shape as Landauer's 1961 argument: erasing one bit must release at least $kT \ln 2$ of heat into the environment (k Boltzmann, T absolute temperature, $\ln 2$ the natural log of 2). The Coq proofs below do not depend on Landauer being right about the physics. They formalise irreversibility at the instruction level only. A cost-positive instruction is tagged irreversible. The count of such firings is bounded by total μ . Shape borrowed, content paid for separately.

Theorem 17.1 (μ -Landauer step bound). *Let $irreversible_bits(i)$ be the indicator function that returns 1 if $instruction_cost(i) > 0$ and 0 otherwise. Let $total_irreversible_bits(trace) := \sum_{i \in trace} irreversible_bits(i)$. Then for every trace,*

$$total_irreversible_bits(trace) \leq trace_total_cost(trace).$$

The Coq witness is `total_irreversible_bits_le_cost`.

Proof. Per-instruction inequality: for every i , $irreversible_bits(i) \leq instruction_cost(i)$. Two cases.

Case $instruction_cost(i) = 0$. Then $irreversible_bits(i) = 0$ by definition, and $0 \leq 0$ holds.

Case $instruction_cost(i) > 0$. Then $irreversible_bits(i) = 1$, and the cost schedule (Section 7) gives $instruction_cost(i) \geq S(\delta) \geq 1$ for cert-setters or $instruction_cost(i) \geq \delta \geq 1$ otherwise. So $1 \leq instruction_cost(i)$.

Summing the per-instruction inequality over the trace:

$$\begin{aligned} \sum_{i \in trace} irreversible_bits(i) &\leq \sum_{i \in trace} instruction_cost(i) \\ &= trace_total_cost(trace). \end{aligned}$$

The right equality is by definition of `trace_total_cost` (or via the Latent- μ theorem applied to the trace from $\mu_0 = 0$). □ □

| Plain reading. Each cost-positive instruction contributes at least 1 to total cost and at most 1 to the irreversible-bit count. Summing across the trace, the count is bounded by the cost. Shape-Landauer in the sense that irreversible operations are charged. No physics in the proof. A 0/1 indicator bounded by what it indicates. No microscopic erasure model. No physical entropy. No Boltzmann constant. The conversion of μ counts to joules is the calibration hypothesis (`mu_landauer_unruh_calibrated`, Section 24). No kernel theorem uses it.

The physical claim has to pass a consistency test. `calibrated_model_exists_if_f_positive_cost` characterizes when the abstract positive-dissipation and upper-calibration premises can hold together. The explicit Boolean reset model has an instance. But retaining the previous state in a history list makes every fixed-instruction step injective while preserving its observed transition. `zero_cost_vm_jump_has_injective_history_lift` applies this construction to the zero-cost F1 counterexample. The PC can collapse to one while the larger state retains its past. I cannot identify physical erasure by watching that PC alone.

The physical question is still open. I want to know whether the formal charge has a defensible physical interpretation. Counting positively charged instructions does not answer that question. Neither does the finite-map Python demonstration: it checks its combinatorial model, not heat released by a device. The F1 counterexample makes the next requirement concrete. Build a physical model whose premises can hold together, account for retained history, and then justify the connection to an experiment. Until that is done, the VM ledger gives no measured energy price for certification.

17.2 The μ -hierarchy

The proof:

Borrow the silhouette of the classical time-hierarchy theorem, and then say honestly how much smaller the thing inside it is. The shape is the part everyone knows: more of the resource buys you something you provably could not have had with less. The resource here is μ , and the something is the cost level a certified trace lands on. Every level $k \geq 1$ is reachable, meaning you can build a trace that pays exactly that much and ends certified; and no level k comes for less than k units of μ , because the ledger conserves and conservation does not run backward. That pairing, reachability on one side and a conservation floor on the other, is the whole content, and it is a statement about how cost stacks up at the ledger, not a statement that one problem class outruns another. Whether more μ ever buys you a genuinely harder problem is a different and much heavier question. It is open, I have it on the list in Section 24, and I am not going to quietly borrow its weight here while it is still unpaid.

Theorem 17.2 (μ -Hierarchy (`mu_hierarchy_theorem`)). *Call a trace level- k certified when it ends with `vm_certified = true` and its executed-instruction log contains a `CERTIFY` whose charge $S(\delta)$ is at least k . For every $k \geq 1$:*

1. *There exists a level- k certified trace with total cost exactly k .*
2. *Any level- k certified trace has `trace_total_cost` $\geq k$.*

Proof. Part (1): the level- k witness. The one-instruction trace `[instr_certify( $k - 1$ )]` does it: the single `CERTIFY` charges $S(k - 1) = k$, which certifies at level k by definition, sets `vm_certified` to `true`, and is the whole bill. Total cost exactly k .

Part (2): cost lower bound. The level- k hypothesis hands over an executed `CERTIFY` with $S(\delta) \geq k$; the kernel proof finds it by scanning the executed-instruction log. That one step charged at least k to the ledger, every other step charged at least 0, and by Latent μ the total cost is the sum of the step charges, so the total is at least k . The scan earns the floor from the certifying step itself.

Each level $k \geq 1$ is therefore realizable at cost exactly k , and no trace certifies at level k for less. □ □

| Plain reading. Two parts. Part (1) is by construction: pick a single `LOAD_IMM` with $\delta = k - 1$ (costs $k - 1$) and follow with `CERTIFY 0` (costs 1). Total cost k , ends certified. Part (2) is just Latent μ : total trace cost equals total μ accumulated from 0, so $\mu \geq k$ means cost $\geq k$. No shortcuts.

Corollary 17.3 (`mu_hierarchy_no_upper_bound`). *No fixed μ -budget suffices for all certification levels.*

Proof. Suppose for contradiction that some fixed budget $B \geq 0$ suffices for all certification levels, meaning every certified trace satisfies `vm_mu` $\leq B$. Apply the μ -Hierarchy theorem at $k = B + 1$: there exists a certified trace with `vm_mu` $= B + 1 > B$, contradicting the budget bound. □ □

| Plain reading. For any budget you propose, the hierarchy theorem produces a certified trace that exceeds it by 1. So no finite budget caps all certifications.

What this comes down to is that the ledger does not give discounts. Pick any integer level you like and there is a certified trace that arrives at exactly that level; try to reach it for one unit less and you cannot, not because anybody forbade it but because the cost was already spent on the way up and conservation will not let you unspend it. That is the entire argument, and where it is mechanised the check agrees. Notice what I did not say. I did not say more μ makes the substrate “strictly more powerful”. That phrase belongs to the problem-class reading, the question of whether a bigger budget

cracks problems a smaller one cannot, and this theorem does not touch it. I would rather leave that fence standing in plain sight than collect on a claim I have not paid for.

Here is the classical statement the shape comes from, so you can see exactly where the resemblance starts and where it stops. The time-hierarchy theorem says that for any time bound $T(n)$ there are problems you can solve in time $T(n) \log T(n)$ that you simply cannot solve in time $T(n)$: hand the machine a little more clock and it can decide things it could not decide before. That is a statement about which problems are in reach. The μ -hierarchy keeps the silhouette and trades the cargo: more μ buys strictly more cost levels, which is a statement about how high the ledger can climb, not about which problems come into reach. So treat the resemblance as a way to see the result, not as a smuggled inheritance of the classical theorem's strength. The proof a few lines up is the part actually holding weight; the analogy is just the light I am reading it by.

To falsify: find $k \geq 1$ and a trace reaching `vm_certified = true` and `vm_mu` $\geq k$ with total trace cost $< k$. The theorem says this cannot happen. If it does, the cost accounting is wrong.

18 Discrete Geometry over Partition Graphs

The partition graph can be studied as a combinatorial object with an assigned angle model. The result below is conditional on explicit incidence and boundary-count hypotheses; it requires no physical interpretation.

18.1 Discrete triangulation of the partition graph

A partition graph is not only a bookkeeping structure. You can also read it as a shape, a discrete geometric object, and put geometric questions to it. The honesty of the framing matters here, so I will be plain about what is and is not being claimed. The result below reads the partition graph as a triangulated combinatorial object, a graph cut into triangular pieces, and sets it in an equilateral angle model: every corner of every triangle contributes $\pi/3$ radians, which is 60 degrees, because the three angles of an equilateral triangle are equal and a full turn is 2π radians, or 360 degrees. Those angles are a modeling choice. They are not pulled out of μ costs, and I am not going to dress them up as if they were.

A discrete triangulation defines a curvature-like angle defect. As I understand it (I researched this for the project, so check me on the geometry): on a flat surface, if you draw a triangle, the interior angles sum to 180° . On a curved surface, the angles can sum to more or less than 180° . Curvature is the amount by which triangles deviate from flat. For an interior vertex of a triangulated surface, the usual discrete curvature is 2π minus the incident angle sum. At a boundary vertex, the corresponding boundary turning contribution uses π instead. The quantity used in the Coq result below uses 2π at every vertex; on a surface with boundary it therefore includes an extra boundary

contribution.

I went looking for the right invariant and found the Euler characteristic χ , a number that captures the overall shape of a triangulated surface, independent of how you draw the triangles. For any triangulation with V vertices, E edges, and F triangular faces: $\chi = V - E + F$. As I read it, no matter how you triangulate the same surface, you get the same number. For a sphere, $\chi = 2$. For a torus (donut shape), $\chi = 0$. The continuum Gauss-Bonnet theorem (which I am citing, not certifying) says the total curvature of a closed surface equals $2\pi\chi$: the local geometric information (curvature) integrates to a global topological fact (shape). The discrete analogue is stated below. The continuum result is exposition.

18.2 The restricted angle-defect identity

The definition `well_formed_triangulated` bundles properties of a partition graph, including triangle and incidence conditions and the additional equation $B = 3\chi$. That equation is a hypothesis, not a general theorem about triangulated disks. For example, a disk made by dividing a quadrilateral into two triangles has $V = 4$, $E = 5$, $F = 2$, $B = 4$, and $\chi = 1$, so it fails this extra condition. The predicate does not separately construct a planar embedding or establish every topological property of a disk.

Theorem 18.1 (Restricted angle-defect identity (`discrete_gauss_bonnet`)). *Let g be a partition graph satisfying `well_formed_triangulated`. In addition to its graph and triangle conditions, this predicate requires $3F = 2I + B$, the triangle-incidence degree sum $\sum_v \deg(v) = 3F$, and $B = 3\chi$, with $\chi = V - E + F$. The edge definitions give $E = I + B$. Assign angle $\pi/3$ to every triangle corner and define*

$$D(g) = \sum_v (2\pi - \deg(v)\pi/3),$$

including the same 2π term at boundary vertices. Then $D(g) = 5\pi\chi(g)$.

Proof. The required triangle-incidence sum gives $D = 2\pi V - \pi F$. From $E = I + B$ and $3F = 2I + B$, we get $B = 2E - 3F$. Consequently,

$$D - 2\pi\chi = (2\pi V - \pi F) - 2\pi(V - E + F) = \pi(2E - 3F) = \pi B.$$

The additional hypothesis $B = 3\chi$ now gives $D = 2\pi\chi + \pi B = 5\pi\chi$. □

| Plain reading. The coefficient follows from both the definition of D and the boundary-count restriction. Coq checks their consequence; it does not establish that the restriction is forced by computation, topology in general, or nature. Removing $B = 3\chi$ from the assumptions leaves the more general algebraic expression $D = 2\pi\chi + \pi B$ under the incidence equations, rather than a universal $5\pi\chi$ law.

For a triangulated surface whose boundary is a union of simple cycles, the number of boundary vertices equals B . Replacing 2π by π at those boundary vertices subtracts πB , giving the usual combined interior-curvature and boundary-turning total $2\pi\chi$. This explains the relation to discrete Gauss–Bonnet. It requires identifying the boundary vertices and the appropriate surface hypotheses. The existing Coq theorem proves the defined D identity under its own predicate; it does not formalize that boundary-corrected theorem.

For a simple equilateral triangle, the distinction is visible without any machinery. Summing $2\pi - \pi/3$ at its three vertices gives 5π . Summing the boundary terms $\pi - \pi/3$ gives 2π . The first sum is the quantity defined here; the second is the boundary turning total. Both calculations can be right while answering different questions.

I did not write these proofs by hand. I directed LLMs and used the checker to test the formal statements. That method still leaves me responsible for checking the definitions and the interpretation. The 5π identity is a conditional combinatorial result, not evidence for a new physical constant or a derivation of angles from μ cost.

19 The substrate and its halting-shaped wall

The Thiele Machine is not the 51 opcodes. The 51 opcodes are scaffolding. They make the substrate concrete enough to verify in Coq, run as OCaml, and synthesize to silicon, but they are not the substrate itself. The substrate is the step relation together with the cost-ledger monotonicity rule: every atomic transition either preserves μ or charges it. PNEW, MORPH, CHSH_LASSERT, the whole instruction set. Those are the experimental apparatus that gives the substrate a concrete realization to probe. Strip the opcodes and the substrate is still there. Add different opcodes that respect the substrate’s monotonicity discipline and the substrate is still the same substrate.

The Turing machine has the same shape. Tape, head, and transition table are scaffolding for one realization. Register machines, lambda calculus, counter machines, RAMs, and partial recursive functions all instantiate the same step-relation primitive with different scaffolding; the equivalences between them are theorems, and that this class captures effective computation at all is the Church-Turing thesis. They are all the Turing machine in the sense that matters. Their primitive steps and cost models can differ even when their computable functions agree.

The result that follows is a fact about the substrate, not about the 51 opcodes. The opcodes are one realization. Other substrates that supply the same typeclass fields inherit the same theorem the moment they are presented.

Relationship to Rice’s theorem. The result that follows is Kleene’s 1938 diagonal pattern instantiated at the substrate level. Rice’s 1953 theorem applies the same diagonal to non-trivial extensional properties of recursive functions computed by Turing machines; the substrate-level version below applies the very same diagonal to non-trivial

extensional predicates of substrate programs in any model that exposes an internal recursion theorem and an internal representability predicate. It is the same trick, two floors apart, which is the whole point. The formal content is the diagonal construction; what the substrate-level framing adds is the parameterization (any substrate satisfying the typeclass interface, not just the standard Turing model) and the choice of extensional predicate (`AdmitsShortcut`, branching on `csr_cert_addr` reachability). The result is therefore a substrate-level instance of the pattern Rice used; the result does not fall out as a corollary of Rice’s theorem in its original setting, because Rice’s premises are about partial recursive functions in the Turing-machine model, and nothing in them knows about arbitrary substrates carrying their own recursion theorems and representability predicates. The formalized result should be read with its own interface premises. Its proof structure does not establish a non-reducibility or Turing-degree separation from classical halting.

19.1 The Substrate typeclass

A Coq typeclass is a checklist of operations and properties a type has to supply to count as the thing. The `Substrate` interface used here packages the following data and premises independently of any particular instruction set. It does not itself contain a certification predicate or A2:

- A type `state` of substrate states.
- A type `Program` of substrate programs.
- An outcome function `run : Program → state → option state`. The record does not impose a correspondence between this function and `step`, or a divergence interpretation of `None`; those require an instance contract. The VM instance always returns `Some` of its 1000-fuel result, including when that result has not halted.
- A cost function `mu : state → nat`.
- An atomic step relation `step : Program → state → state → Prop`.
- A cost-ledger monotonicity field `mu_monotone: forall p s s', step p s s' → mu s ≤ mu s'`. This is a separate general-purpose monotonicity property from the kernel’s A2 (which specifically asserts that a step flipping certification false-to-true costs ≥ 1 , and which is carried by the `CertificationSystem` record, not by this typeclass). Both properties hold in the VM: all instruction costs are non-negative and the cert transition costs ≥ 1 . See Section 2.8 for the A2 statement and the related axioms A3 through A6.
- A Gödel coding: functions `encode : Program → state` and `decode_safe : state → Program` with the round-trip property `decode_safe (encode p) = p`. This states recovery by the supplied mathematical decoder; it does not assert that a program in the substrate can execute that decoder or inspect the encoded field.

- An internal-representability predicate `Representable : (Program → Program) → Prop`, scoping which Coq function transformers the substrate can express as programs of its own kind. Concrete substrates instantiate this with a precise meaning; the minimal nat-coded substrate defines `Representable f` as “`f` is the diagonal flip transformer for the substrate’s own decide function,” which is exactly the shape the diagonalization needs.
- A recursion-theorem field, internal form: for every internally representable transformer `f : Program → Program`, there is a program `p` extensionally equivalent to `f p`, where extensionally equivalent means the two give the same result on every input, however differently they are written.

The recursion-theorem field is the load-bearing piece. It is the substrate’s Kleene 1938 recursion theorem, restricted to transformers the substrate can actually express. The restriction is not a hedge; it is the substrate’s own scope. Turing 1936 did not prove “no decider exists in any conceivable mathematical universe”; he proved “no Turing machine decides halting,” because Turing machines were the substrate Turing was studying. The restriction to internally representable transformers is the same scope move at the substrate level: the substrate’s limitative theorem is about what the substrate itself can express, not about every Coq function that exists in the meta-theory.

The recursion theorem is not derivable from `mu_is_initial_monotone` (the cost-functional uniqueness result). Initiality of the cost functional states that a measure initialized to zero and satisfying the exact increment law $M \text{ (vm_apply } s \ i) = M \ s + \text{instruction_cost } i$ agrees with μ on reachable states. It does not select the instruction-price schedule, identify measures on unreachable states, or collapse all monotone measures to one inhabitant. The recursion theorem instead supplies extensional fixed points for transformers satisfying the chosen representability predicate. In the nat construction it is proved separately for each parameter d , whose flip determines that instance’s restricted representability class and code-2 behavior.

19.2 The shortcut-admittance predicate

The substrate becomes interesting for the limitative result when it is equipped with a non-trivial *shortcut-admittance predicate*: a Prop-valued predicate `AdmitsShortcut` on programs, witnessed by at least one program that admits and at least one that does not, and depending only on extensional behavior (programs with the same `run` behavior on every state admit shortcuts iff each other).

The typeclass extension `WithShortcutPredicate` packages this: an `AdmitsShortcut` field, two canonical witnesses `yes_program` and `no_program` with their respective inhabitedness or non-inhabitedness lemmas, and the extensionality field. These are the conditions the diagonalization needs. A substrate that supplies them earns the limitative theorem.

The kernel constructs a concrete nat-coded family and a VM instance parameterized

by its representability class and recurrence premise.

For each $d : \mathbb{N} \rightarrow \text{bool}$, `nat_substrate d` supplies the typeclass fields for a language with distinguished codes 0, 1, and 2. Its representability predicate selects the flip transformer associated with d , and code 2 supplies its fixed point. The theorem `nat_self_undecidable` states that d fails to decide `nat_admits d`. The global assumption report is closed; d remains an explicit mathematical parameter.

The 51-opcode VM substrate `vm_substrate` is parameterized over the VM-specific Gödel encoding (`vm_encode`, `vm_decode_safe`, the round-trip), the VM’s internal-representability predicate (`vm_representable`), and the VM’s internal recursion theorem restricted to representable transformers. The Coq Section discipline is identical to the one the physical bridges use for their named constants: the parameters are not global axioms; closing the surrounding Section discharges them as explicit `forall` premises on every consumer. The encoding is a Section parameter too, though not a live one: the kernel’s closed round-trip lemma (`program_to_nat`, `nat_to_program`, no Hypothesis) discharges it, which is exactly what the `_encoded` corollary below does. The VM’s internal-representability predicate and bounded recurrence premise remain explicit parameters of the encoded theorem. The generic theorem is independent of this particular VM instance, but still requires its own interface premises. The separate family `nat_substrate d` supplies those premises only for its d -dependent semantics and representability class; it does not discharge the VM premise.

The VM-side `AdmitsShortcut` is the extensional predicate “this VM program’s bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`.” `yes_program` is `simple_morph_trace` itself; `no_program` is the empty trace (which produces `init_state`, demonstrably distinct from `simple_morph_final`, since the latter has fired the supra-cert channel and the former has not). The extensionality field is the standard “equal run on every state implies the predicates agree.” The instantiation that assembles these into the `WithShortcutPredicate` interface lives in the kernel.

19.3 The substrate-level limitative theorem

Theorem 19.1 (Structural-axis undecidability, substrate-level). *For every Substrate-instance equipped with a `WithShortcutPredicate` extension, there is no Coq-definable function $\text{decide} : \text{Program} \rightarrow \text{bool}$ whose corresponding diagonal flip transformer is internally representable in the substrate and that for all programs p satisfies: $\text{decide } p = \text{true}$ iff `AdmitsShortcut p`.*

Proof. Assume for contradiction such a `decide` exists. Define the diagonal flip transformer

$$\text{flip}(p) = \begin{cases} \text{no_program} & \text{if } \text{decide } p = \text{true} \\ \text{yes_program} & \text{if } \text{decide } p = \text{false} \end{cases}$$

By hypothesis, `flip` is internally representable in the substrate. Apply the substrate's typeclass field `recursion_theorem` to `flip`: there exists `p` such that `prog_equiv p (flip p)`, where `prog_equiv` is the extensional equivalence “run agrees on every initial state.”

Case-split on `decide p`. *Case A* (`decide p = true`): by the bi-implication, `AdmitsShortcut p`; by definition of `flip`, `flip p = no_program`; by the fixed-point witness, `prog_equiv p no_program`; by extensionality of `AdmitsShortcut` (typeclass field), `AdmitsShortcut p` iff `AdmitsShortcut no_program`. The latter is false (typeclass field `no_program_refuses`). Contradiction.

Case B (`decide p = false`): symmetrically, by the bi-implication $\neg$ `AdmitsShortcut p`; `flip p = yes_program`; `prog_equiv p yes_program`; extensionality yields `AdmitsShortcut p` iff `AdmitsShortcut yes_program`. The latter is true (typeclass field `yes_program_admits`). Contradiction.

Both cases close. No such `decide` exists. `PrintAssumptionsstructural_shortcut_undecidable` returns `Closed` under the global context. $\square$

| Plain reading. Kleene 1938's diagonal, transplanted from time to structure. Same shape: assume a decider, build a flipper, find a fixed point, watch the program disagree with itself in both cases. The work is in the typeclass: once the substrate has its own recursion theorem, two canonical inhabitants on opposite sides of the predicate, and extensionality, the diagonal lands itself. The conclusion excludes correct deciders whose flips belong to the supplied representability class. Its application to a concrete execution model requires that model's fixed-point premise; the generic proof does not establish it.

Theorem 19.2 (Structural-axis undecidability for the nat-coded substrate). *For every Coq function $d : \text{nat} \rightarrow \text{bool}$, the substrate `nat_substrate d` cannot internally decide its own structural-shortcut membership predicate. In particular, `d` itself is not a correct decider for the predicate of the substrate it parameterizes; the corollary `nat_self_undecidable` reads off this self-referential conclusion directly.*

Proof. For each fixed `d`, the substrate `nat_substrate d` supplies every typeclass field constructively: `Program := nat`; `yes_program := 0`; `no_program := 1`; `nat_run d` is a finite Coq pattern match using `d(2)`; `prog_equiv` is pointwise equality of run; `Representable f` is the precise predicate “`f` is the diagonal flip transformer for the substrate's own decide function”; `recursion_theorem` is discharged by exhibiting code 2 as the fixed point of any such transformer (its run is defined to mirror whatever the transformer outputs); `AdmitsShortcut` is extensional by construction; `yes_program_admits` and `no_program_refuses` hold by definitional unfolding of run at 0 and 1. Apply the substrate-level theorem above. The corollary `nat_self_undecidable` specializes the conclusion to `d` itself: `d` cannot be a correct

decider for `nat_admits d`, cannot answer `true` exactly when the predicate holds. Print Assumptions on both `nat_structural_shortcut_undecidable` and `nat_self_undecidable` returns `Closed` under the global context. $\square$

| Plain reading. For each candidate d , the construction supplies an instance and proves that d fails on that instance’s predicate P_d . The instance itself depends on d . This family theorem does not exhibit one fixed effective predicate that defeats every effective decider.

Theorem 19.3 (Structural-axis undecidability for the 51-opcode VM, conditional on the VM’s own recursion theorem). *Assume the VM’s internal recursion theorem restricted to representable transformers: for every transformer f on VM programs with `vm_representable f`, there exists a program p such that `vm_run p` and `vm_run (f p)` agree on every initial state. Under that assumption, there is no Coq function `decide : list vm_instruction → bool` whose diagonal flip transformer satisfies `vm_representable` and that for all programs p satisfies: `decide p = true` iff `vm_admits_shortcut_extensional p`.*

Proof. Instantiate the substrate-level theorem at `vm_substrate`. The Gödel encoding piece (`vm_encode`, `vm_decode_safe`, the round-trip) enters as Section parameters and is then discharged by the kernel’s closed round-trip lemma (`program_to_nat`, `nat_to_program`, no Hypothesis) in the `_encoded` corollary. The VM’s internal-representability predicate `vm_representable` and the VM’s internal recursion theorem restricted to representable transformers are the Section parameters that stay live; closing the surrounding Coq Section discharges them as explicit `forall` premises on every consumer, the same Section-parameter discipline the physical bridges use for their named constants, and the theorem above states that recursion-theorem premise directly rather than leaving it implicit in the proof. The VM-side `AdmitsShortcut` is the extensional predicate “this VM program’s bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`”; `yes_program` is `simple_morph_trace`; `no_program` is the empty trace, which produces `init_state` and so differs from `simple_morph_final` (where the supra-cert channel has fired). The substrate-level diagonal applies; `vm_structural_shortcut_undecidable` is its specialization, carrying the VM-side parameters as premises. The `nat` family above is a separate application with different semantics. This VM specialization remains conditional on its stated bounded recurrence premise and representability class.

`vm_run` returns `Some (run_vm 1000 p s)` even when fuel expires before halting. Equality of its two finite-data results is decidable externally. Under the recurrence premise, the flip of that correct external decider must therefore lie outside the chosen representability class. The unbounded relation `VMUnboundedExec.vm_halts_at` instead requires an existential fuel witness and a halted result. Its determinism and saturation proofs do not identify it with bounded-result equality. A separate develop-

ment now supplies a fixed 60-instruction interpreter over the natural-valued sibling `vm_apply_u`, with executable variable-width encodings of two-counter programs. The guest branches on a successful nonzero decrement and falls through on zero. This convention matters: the earlier zero-branch guest is a weaker machine and is not used as a universality witness. The new `VMUnboundedCM2Bridge.mm2_halt_host_iff` proves equivalence between halting in the pinned Coq Undecidability Library's MM2 semantics and raw execution of this fixed host interpreter on the encoded input. Its correctness, malformed-code exclusion, and divergence contracts are proved independently of any supplied interpreter-correctness premise.

The transferred `actual_unbounded_host_synthetic_undecidability` theorem retains the library's precise synthetic meaning: a Boolean halting decider would enumerate the complement of single-tape binary Turing-machine halting. It is not a proof of the unrestricted proposition $\sim \text{decidable}$, nor a recursion theorem for the full VM ISA. The original word64 machine, bounded equality theorem, and conditional structural-shortcut theorem above are unchanged. No effective-specialization theorem or concrete representability/recurrence instance for the full VM is established by this reduction. $\square$

| Plain reading. The VM theorem is conditional on the bounded recurrence and representability premises printed above. The `nat` family supplies a separate concrete example for each candidate decider. It does not discharge the VM premise. Unbounded execution and a self-interpreter require their own semantics and proofs.

19.4 The proof, in prose

The proof is the standard Kleene-1938 diagonal, transported to the substrate.

Assume for contradiction that `decide : Program → bool` satisfies the bi-implication. Define the flipping transformer

$$\text{flip}(p) = \begin{cases} \text{no_program} & \text{if } \text{decide } p = \text{true} \\ \text{yes_program} & \text{if } \text{decide } p = \text{false} \end{cases}$$

This is a total Coq function from `Program` to `Program`. The theorem also requires that this particular `flip` satisfy `Representable`; total Coq definability alone does not establish that premise. With that premise, apply the substrate's recursion-theorem field to `flip`: there exists a program `p` with `prog_equiv p (flip p)`.

Case-split on `decide p`.

Case A: `decide p = true`. By the bi-implication, `p` admits a shortcut. By definition of `flip`, `flip p = no_program`. By the recursion-theorem witness, `p` is extensionally equivalent to `no_program`. By extensionality of `AdmitsShortcut`, `p` admits iff `no_program` admits. But `no_program` does not admit. Contradiction.

Case B: decide $p = \text{false}$. By the contrapositive of the bi-implication, p does not admit. By definition of `flip`, `flip p = yes_program`. By the recursion-theorem witness, p is extensionally equivalent to `yes_program`. By extensionality, p admits iff `yes_program` admits. But `yes_program` admits. Contradiction.

Both cases close. No correct total decision procedure for `AdmitsShortcut` whose flip satisfies the stated representability premise exists in this instance. $\square$

The proof is short, because the work is in the typeclass, not in the diagonal. The body runs about 38 lines, of which roughly 16 are actual tactic steps; the rest is the case-split structure and the inline narration. Once the substrate supplies the restricted recursion theorem, two canonical inhabitants, an extensional predicate, and representability of the candidate decider's flip, the diagonal is mechanical. The substrate-level work was assembling the right typeclass; the limitative result is what falls out.

The diagonal does not use `A2`, the ledger, or a certification predicate. Its contradiction uses the restricted recursion-theorem field, representability of the candidate flip, and the two witnesses and extensionality of `WithShortcutPredicate`. Certification supplies a separate interpretation in the VM; the `nat` family has no certification content. `A2` neither supplies the recurrence premise nor establishes a correspondence between the bounded program predicate and the CSR state predicate. Those connections must be proved separately wherever an application requires them.

19.5 The two-axis picture this completes

Time axis	Structural axis
steps	μ
HALT (Turing 1936)	<code>AdmitsShortcut</code> (above)
no Turing decider for halting	no correct decider with a representable flip under recurrence

The comparison records the diagonal proof pattern under each model's premises. The `nat` construction gives a candidate-indexed family, and the VM result concerns bounded outcome equality under a restricted recurrence premise. Neither establishes a second fixed effective undecidable problem or a Turing-degree separation. The CSR observation collision below is a separate state-level non-recoverability result.

19.6 What this closes

What the substrate-level theorem closes. A total witness procedure would yield a decider of the same predicate only under an explicit reduction contract: its input is each program p in the theorem's domain; its success is equivalent to `AdmitsShortcut p`; and it terminates with either a witness or a definite negative answer on every such input. To apply the diagonal, the Boolean decider obtained by testing that success must also have a flip in the supplied representability class. Under these premises the theorem excludes the procedure. A procedure returning a `SoundStructuralShortcut` record is not automatically such a reduction: its witness

relation must first be connected to this exact program predicate. Partial synthesis, restricted argument inputs, and externally supplied evidence are not excluded by the total-decider theorem.

What the nat-substrate theorem closes. It constructs a family of instances indexed by the candidate d . For each member, d fails on that member's predicate. The generic theorem applies under its representability and fixed-point premises. A fixed-system effective undecidability result requires another instance argument.

What the VM corollary closes. The 51-opcode VM corollary `vm_structural_shortcut_undecidable` is the substrate-level theorem instantiated at `vm_substrate`: no representable decider answers `true` exactly when a VM program admits the extensional shortcut. It carries the VM-side Section parameters as explicit `forall` premises, in the named-premise style the physical bridges use. The generic theorem does not depend on these particular VM parameters, but its VM application does. The `nat` family supplies a different instance for each d and cannot replace the VM's bounded recurrence premise. The exclusion applies to deciders whose diagonal flips lie in the supplied representability class.

How to falsify. Three routes.

1. A Coq function `decide : Program → bool` whose diagonal flip transformer is internally representable in some Substrate plus `WithShortcutPredicate` instance and that satisfies the bi-implication would force the substrate-level proof to fail at one of its steps, each of which is named in the case-split above.
2. For the nat-coded substrate specifically: exhibit a Coq function `d : nat → bool` that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` forbids this; finding such a `d` contradicts the corollary.
3. For the VM specialization, supply its bounded recurrence premise for the chosen representability class and a correct decider whose particular diagonal flip belongs to that class. A correct external decider alone is not a counterexample: bounded membership is externally decidable, and its flip is excluded by every class satisfying that recurrence premise.

19.7 The wall the classical configuration cannot see

There is one sentence that kills most new “a computer can't decide this” results before they get off the ground, and the first time I thought I had one I said it to myself before anyone else could: *you didn't find a new kind of unanswerable question. Your machine is a Turing machine with some extra memory bolted on, the thing you can't decide is just one more ordinary fact about what a program does, and Rice's theorem already says nobody can decide facts like that. Nothing new, nothing sideways to ordinary computation.* I went looking for where that sentence breaks the result, because if it does, I'd spent months on a footnote

to Rice. It doesn't. The reason is plain once you see it: a classical machine never gets the whole state. It gets a stripped-down version, the part it can read and write, and my unanswerable question turns on a piece the stripping throws away. Rice's theorem is about an answer a machine can't compute from what it's handed. This is about an answer that was never in what it's handed.

A classical decider has, as its entire input, the Turing configuration `forget s` : `TMSnapshot`, the same four-field projection from the structural-axis section. If the thing it is asked to decide is not a function of `forget s`, then no classical decider can be right about it, and not for want of horsepower: what it is deciding is not in what it is given. That is the whole shape of this wall, and it is why it sits beside Rice's rather than under it. Two independent obstructions, on two independent axes.

Theorem 19.4 (Structural-shortcut admittance is not classical, `structural_shortcut_not_function_of_classical`). *There is no function `decode : TMSnapshot → bool` such that, for every Thiele state s ,*

$$\text{decode}(\text{forget } s) = \text{admits_structural_shortcut_bools},$$

where `admits_structural_shortcut_bools` is the reading “ s 's certification channel has fired” (`csr_cert_addr ≠ 0`).

Proof. Two states reachable from `init_state` by executing a trace: `run_cert_set` runs the minimal `MORPH_ASSERT` closure and fires the channel (`csr_cert_addr = 650`); `run_cert_unset` runs the same two-instruction prefix, then a graph-preserving no-op whose μ -cost is chosen to match, leaving `csr_cert_addr = 0`. The matched cost makes the two runs land on the same $(pc, \mu, \text{regs}, \text{mem})$, so `forget run_cert_set = forget run_cert_unset` by computation. A `decode` satisfying the equation would have to return both `true` and `false` on that one input. Contradiction. □ □

| Plain reading. These are reachable witnesses for the CSR predicate, with the same four-field `forget` projection. The predicate is decidable from full state and cannot be recovered from that projection. The bounded program-equality predicate used by the diagonal is a different definition; no equivalence between these predicates is asserted.

This is the first rung of a short ladder, and I want to be exact about how far each rung reaches, because the honest version is sharper than the inflated one.

Observation-dependent oracles. `structural_axis_survives_any_classical_oracle` quantifies over oracle answers that are functions of `forget` alone. Composing such an oracle with a decoder preserves the collision. An oracle supplied independent evidence or full state has a different information domain.

The two non-determination directions. `axes_mutually_independent` supplies reachable witnesses: the structural channel is not determined by `forget`, and the program counter is not determined by that channel. `structural_membership_decidable` computes the channel predicate from full state. These are information-loss statements about specified observations. Rice and Turing-degree conclusions require an effective program model and are not consequences of this state-level collision.

The concrete VM encoding. `vm_structural_shortcut_undecidable_encoded` uses the proven `program_to_nat/nat_to_program` round trip, with the code stored in `vm_logic_acc`. It still quantifies over a representability predicate and a fixed-point premise for the bounded `vm_run`. An unbounded self-interpreter would concern another execution semantics and would require a correspondence theorem before it could discharge this premise. The separate B3 interpreter uses executable data encoding for an unbounded arithmetic/control fragment; it does not supply this bounded-semantics bridge. Furthermore, `no_logic_acc_encoded_interpreter` proves an input-access obstruction for the current encoding: every executable step and run commutes with replacing `vm_logic_acc`. A fixed VM program cannot use that encoded input to reproduce even a simulated program's final certification bit through an output observation independent of the retained input code. This rules out the proposed encoding for that interpreter contract; it does not rule out an executable alternative encoding or an architectural extension.

| Plain reading. The state-level theorem excludes recovery from the specified observation, including observation-dependent oracle extensions. The program-level theorem excludes deciders whose flip transformers belong to a representability class satisfying the stated recurrence premise. The `nat` example is a family parameterized by the candidate decider. These results retain their separate domains and quantifiers.

20 What Coq is, and why I used it

Coq is a proof assistant, which is a phrase I had to go look up. Underneath the phrase it is a programming language, but one whose type system is strict enough to do a thing ordinary languages cannot: you write a proof down as a program, and the compiler refuses it unless the program genuinely matches the statement you claimed. You cannot talk it into a maybe.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means `x` is an integer. In Coq, types are propositions. The type `n > 0` means “the proof that `n` is positive.” A function from `n > 0` to `n + 1 > 0` is a proof that if `n` is positive, `n + 1` is too.

The compiler checks whether the types line up. If your proof has a hole and you write “trust me,” it rejects you. You can't bluff a proof term past it. But here is the part people skip, and it's the part that matters: the compiler checks that the proof matches

the statement. It does not check that the statement says what you meant. Hand it a statement that means nothing and it'll prove it for you, three characters, no complaint. So the compiler accepting a proof is not the same as the thing being true. It means this proof matches this statement, and whether that's the right statement is on me, not the machine.

So I didn't use Coq to make anything true. I used it because I don't trust my own eye to catch a gap in an argument I want to believe. I spent months convinced by my own reasoning, pushed the proof through, and watched a key step come apart in my hands. The machine caught where I'd fooled myself. That is what it's for, and it's the same source the running machine and the chip get extracted from. Gap-catcher and extractor. Not the proof. The proof is the argument, and an argument has to close in a way a person can follow, or it hasn't closed at all.

20.1 Reproducing the source and proof contracts

Repository: <https://github.com/sethirus/The-Thiele-Machine>. The review workspace starts from commit 69b968906eeb88622f6480cd65a011a7d021b890. The changes described here are supplied by the companion artifacts/review_revision/source.patch; source_manifest.json identifies its bytes and the reviewed source files. The base commit alone does not contain these changes. Publication of a final release commit remains separate from this local review artifact.

The verified local toolchain is Coq 8.18.0, OCaml 4.14.1, and Python 3.12.1, with the vendored Kami and bbv dependencies built. From a checkout of that base commit, apply the companion patch with `git apply source.patch`. Then run the following commands from the repository root in the prepared toolchain:

```
export COQPATH="$PWD/vendor/bbv/src:$PWD/vendor/kami"
cd coq
coq_makefile -f _CoqProject -o Makefile
make -j2
cd ..
python3 scripts/check_review_contracts.py \
  > artifacts/review_revision/contract_report.txt
bash monograph/build_monograph.sh
```

The Coq build and contract probe must exit zero. Inspect the report's theorem types as well as its global dependencies; a closed report does not discharge explicit premises. The monograph build prints `Build Successful` and regenerates `monograph.pdf` and `monograph.txt`. The full-corpus reports remain in `artifacts/print_assumptions_all_proofs.*`; their stored counts belong to the source version recorded by that audit, not automatically to every later edit.

RTL generation is a separate command, `bash scripts/kami_extract.sh`, requir-

ing the Bluespec compiler and its standard library in addition to the Coq/OCaml toolchain. Its logs, generated Verilog, synthesis report, and any bitstream must carry the source identity they actually used. A successful Coq build supplies no fresh synthesis or timing measurement. The saved Inquisitor report likewise retains its generation date and waiver census.

20.2 The Inquisitor

Beyond Coq’s type checker, the project has a custom proof auditor called the Inquisitor, shipped with the proof corpus. It runs on every commit and emits 86 distinct rule codes. The main categories:

- **No holes.** No `Admitted`, `admit`, or `give_up` anywhere in the active proof files. No unproved `Axiom` or `Parameter` declarations in project code.
- **Vacuity heuristics.** No theorem whose conclusion is literally `True`, `0 = 0`, or any other tautology that proves nothing. No “theorem” of the form $P \rightarrow P$. No existence claims backed by constant witnesses. Coq’s kernel already prevents circularly dependent proofs (it checks definitional well-foundedness); the Inquisitor catches separately the case where a proof is technically valid but epistemically empty, proving `True` by `trivial`, for instance.
- **Physics-definition heuristics.** No physics quantity defined as a constant placeholder. No physics claim without a corresponding invariance condition. No theorem stated about a physics concept if that concept is not genuinely formalized.
- **No trivial true proofs.** If a theorem’s entire proof is `trivial` or `tauto` and the conclusion reduces to `True`, the theorem is rejected.
- **Scope enforcement.** The proof tree is split into tiers. Tier 1 is the kernel files: the core proof of the machine. Tier 1 files may not import from exploratory or research-side files. Theorem connectivity: every file in the active proof tree must reach the cost or semantics foundation through its import chain. If a file is disconnected, it does not get to claim the kernel’s pedigree.
- **Comment hygiene.** No `TODO`/`FIXME`/`WIP` markers in active proof comments.

The saved `INQUISITOR_REPORT.md`, generated 11 September 2026, records 297 scanned Coq files, zero HIGH, MEDIUM, and LOW unsuppressed findings, and 341 suppression markers across 140 files. This is a historical report for its recorded source state. Regenerate it for a changed tree. The rule checks and waiver census are heuristic evidence; they do not prove semantic applicability or absence of every vacuous premise.

21 Zero Admitted: what it actually means

“Zero Admitted” is literal. Nowhere in the proof tree did I write “trust me” and walk on. Every step the checker could see, it saw, end to end, with no “and the rest is obvious.” That is a hygiene fact about the gap-catcher, not a verdict on the argument: it says I didn’t let myself skip a step where the machine was watching. It does not say the statements are the right statements. That part stays on me, which is what the rest of this section is about.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele substrate, as formalised in Coq. Whether that formalisation describes the world you live in is a separate question, answered by experiments, not by proof checkers.
- It does not mean the Coq axioms themselves are consistent. Coq sits on a formal foundation called the Calculus of Inductive Constructions. The key idea is the Curry-Howard correspondence: proofs and programs are the same thing. A proof that P is true is literally a program of type P . Checking the proof is type-checking the program. The “inductive” part means you can define your own recursive types (naturals, lists, trees) inside the system instead of having them as magical primitives handed down from above. That matters. The foundation is explicit. Nothing is smuggled.

Now the honest limitation. The sources I read on Coq’s foundation say it is believed to be logically consistent (meaning you cannot derive a contradiction from it), but this has not been proven within Coq itself. The way I understand it, that is not a weakness specific to Coq. The label I dug up is Gödel’s incompleteness theorem from 1931: a consistent, effectively axiomatized theory containing sufficient arithmetic cannot prove its own standard arithmetized consistency statement under the usual derivability conditions. And an inconsistent system proves everything, its own consistency included. So Coq’s foundation is trusted, not proved-from-within. “Trusted” has substance here. The same foundation has verified the CompCert C compiler and the Four Color Theorem (Gonthier 2005, a landmark fully machine-checked proof of a major mathematical theorem, one first settled on paper-plus-computer in 1976), among other artifacts. That prior use is evidence about the foundation’s track record in practice. It is not a proof of logical consistency, and a successful large-scale application would not necessarily surface every possible inconsistency; an inconsistent system only breaks visibly if some proof actually exploits the inconsistency, and nothing here rules that out. I am relaying what I read. I have not personally audited CompCert or Gonthier’s proof. “Zero Admitted” means every theorem follows from those axioms. It does not mean those axioms are themselves in-system guaranteed. It means they are the same axioms a substantial body of verified

software has relied on without breaking, which is track record, not a consistency proof.

- Hardware correspondence is conditional on its model and representation contract. `driven_step_wf` proves intermediate model equality under `WFDDrivenPrecondition`; executable RTL transfer adds finite-range and retirement obligations and toolchain trust. The four Q_{1+AB} instructions remain outside the synthesized inventory.
- It does not mean the OCaml runner binary is proven inside Coq to be bitwise identical to the Coq semantics. The kernel proves Coq-side observable facts. The parity suite checks the extracted binary empirically. One is a theorem. The other is a test.

A kernel-checked theorem proves its complete type, including quantified premises and record obligations. A Section hypothesis can become a theorem argument after generalization. `Print Assumptions` reports global dependencies; `Closed` under the `global context` does not remove those arguments. Assessing an intended instance requires its statement, expanded load-bearing definitions, global assumption report, and witnesses for the premises.

The bridges add mathematical premises and external trust boundaries. Physical interpretations require their named quantum or Landauer–Unruh calibration premises. Executable hardware transfer also depends on extraction, the OCaml pretty-printer, BSC, generated RTL, and the simulation harness; synthesis and place-and-route add further tools when silicon claims are made. The Coq model theorem does not verify that whole chain. External OCaml and RTL parity are test results with their own revision and input domains.

22 How to break this

Every claim in this document comes with the exact shape of the thing that would kill it. I went to the trouble on purpose. A claim you can't break is a claim that wasn't saying anything, and I would rather hand you a loaded gun pointed at my own work than a sentence too vague to aim at. So the ones below are written as instructions, not as boasts: here is the trace to find, here is the object to build, here is the theorem to exhibit. Build one and the proof tree falls, and I mean that as the better outcome of the two, because a break tells me something I didn't know and a year of nobody finding one only tells me people got tired of looking. If you build any of these, that is the message I want in my inbox.

Falsify No Free Insight.

Find a VM trace that:

- starts with `vm_certified = false` and `vm_mu = 0`

- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the claim is false: `certification_requires_positive_mu` breaks at the step that prices the cert-flip. Add the counterexample as a theorem and the check refuses to compile on top of that.

Falsify μ -monotonicity.

Find a state s and instruction i such that $(\text{vm_apply } s \ i).\mu < s.\mu$. The Coq proof of `vm_apply_mu` falls.

Falsify categorical separation.

Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. That kills `categorical_separation`.

Falsify Turing strictness.

Exhibit a *classical* program, a Thiele instruction trace restricted to the classical opcode fragment (`is_classical_program`, run on the same substrate), whose execution changes the partition graph (`pg_next_id`) or the morphism map. Or prove that the witness step `PNEW` leaves `pg_next_id` fixed. Either kills the strictness half of `D5_thiele_strictly_extends_classical`. This is the semantic claim, that the classical opcode fragment cannot reach the structural states one structural opcode reaches. It is not the computability claim. A Turing machine can of course simulate a Thiele run by encoding the graph as bytes on its tape; that simulation exists and is not a counterexample.

Falsify structural entitlement.

Prove that for every pair of Thiele VM states with the same classical shadow and different morphism structure, no probe preserves the distinction; equivalently, refute the concrete witness pair the theorem exhibits. `shadow_strictly_lossy` falls. Or produce a strict feasible-set subset with a distinguishing observation that does not yield stronger membership predicates. The feasible-subset theorem falls with it.

Falsify universal certification cost.

Build a `CertificationSystem` satisfying the one-step rule `cs_cert_costs`, then give a trace from uncertified to certified with total cost 0. `universal_nfi_any_substrate` falls. If you instead drop the one-step rule, you have found a free-certification system, not a counterexample.

Falsify substrate-level structural-axis undecidability.

Three routes (Section 19 expands).

- Exhibit a Coq function `decide : Program → bool` whose diagonal flip

transformer is internally representable in some Substrate plus WithShortcut-Predicate instance and that satisfies the bi-implication. The substrate-level proof of `structural_shortcut_undecidable` must fail at some step; identify which.

- For the nat-coded substrate specifically: exhibit a Coq function $d : \text{nat} \rightarrow \text{bool}$ that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` says no such d exists; build one and you have contradicted it head-on, which is exactly the kind of break I want to hear about.
- For the VM specialization, supply its bounded recurrence premise for the chosen representability class and a correct decider whose particular diagonal flip belongs to that class. A correct external decider alone is not a counterexample: bounded membership is externally decidable, and its flip is excluded by every class satisfying that recurrence premise.

Falsify the classical CHSH bound.

Exhibit a locally factorizable strategy that achieves $|S| > 2$. `chsh_stat_violation_not_local` falls. So does known mathematics.

Falsify the CHSH-NPA bridge.

The biconditional `column_contractive_iff_quantum_realizable` falls to a counterexample in either direction: a column-contractive correlator whose specified zero-marginal, zero-within-party-moment matrix is not PSD (forward), or a PSD specified zero-marginal, zero-within-party-moment matrix whose correlators are not column-contractive (reverse). For the Tsirelson corollary, exhibit a slice-coherent (column-contractive) correlator with $|S| > 2\sqrt{2}$ and `state_column_contractive_implies_tsirelson` falls. The PR box and the Turing-classical point $(1, 0, 1, 0)$ are *not* counterexamples: both fail column contractivity by direct arithmetic, which is what the theorem requires, not what would break it.

Falsify the elliptope completion.

Three targets, one per load-bearing theorem of Section 16.8. Exhibit signs a_0, a_1, b_0, b_1 with squares 1 and a vector on which the completed Gram form goes negative; `deterministic_strategy_elliptope` falls. Exhibit an elliptope-realizable correlator tuple with $S^2 > 8$; `elliptope_tsirelson` falls, and with it the claim that the completion cannot stretch both Cauchy–Schwarz budgets at once. Exhibit real x, y making the PR box’s completed 5×5 matrix PSD; `pr_box_not_elliptope` falls. The $\mu = 0$ all-ones witness and $(1, 0, 1, 0)$ are *not* counterexamples here either, in the opposite direction from the bridge entry above: the elliptope theorems admit them, which is what the theorems require.

Falsify the elliptope gate.

`elliptope_check_full_sound` claims soundness with the check’s passing as its

only hypothesis: any passing combination of witness counters, completion buckets, or LDL certificate entails elliptope membership of the witness-derived correlators. Exhibit inputs where `elliptope_check_full` returns `true` while the correlators lie outside the set (violating $S^2 \leq 8$ suffices), and the soundness theorem falls, taking `elliptope_full_gate_never_accepts_pr_box` with it. Trap-side behavior is not a target, and a rejection is never a counterexample to soundness. This gate’s certificate format requires rational completion or LDL data (Section 16.8), so by construction it cannot accept a boundary point whose every PSD completion is irrational. That is a fact about this particular certificate format, not a theorem that any exact integer check whatsoever must reject exactly those points and no others; no such general claim is made or needed here.

The Inquisitor standard.

Run the Inquisitor on the repository and inspect its report. The current report has 0 HIGH, 0 MEDIUM, and 0 LOW unsuppressed findings. The command fails on HIGH or MEDIUM findings; LOW findings are reported without independently causing failure. In-source suppression markers and their justifications are recorded separately. This is a syntactic and heuristic audit, not a complete check of theorem applicability or surrounding prose. A new finding requires review rather than an automatic conclusion that a theorem is false.

Falsify the axiom audit.

If running `Print Assumptions` (Section 2: the Coq command that lists everything a theorem depends on) on any named theorem in the kernel tier shows a project-local `Axiom` or `Parameter` in the dependency tree, the audit has degraded. The current full-corpus probe covers 4,028 named theorems: 2,979 close under the global Coq context outright, and 1,049 lean only on Coq standard-library axiom families (functional extensionality and the classical-reals construction). Of those, 67 depend on the excluded middle, which is the classical-logic axiom $P \vee \neg P$ (“either P is true or P is false, with no third option”); those uses come from the Coq standard library’s real-analysis lemmas, entering through the physics-bridge families (the Holevo, Bekenstein, Landauer, and gravity bridges), not from project code; the CHSH algebra itself leans only on the classical-reals construction and functional extensionality. Zero project-local axioms appear in any of the 4,028 dependency trees. Any project-local axiom or parameter appearing in any of them is a bug.

Falsify the hardware bisimulation.

A counterexample to `driven_step_wf` must satisfy `WFDrivenPrecondition` and falsify its stated intermediate model equality. A cosimulation mismatch on a valid represented input challenges the tested RTL transfer for that revision. Diagnose encoding, observation timing, extraction, pretty-printing, BSC, and harness behavior separately. A rejected input outside the theorem’s domain or a toolchain mismatch does not refute the conditional Coq theorem.

Falsify the convergence answer to parametricity.

The separation/minimality/uniqueness trio is schema-parametric: any field a step rule carries that classical state does not determine supports all three theorems, word for word, and I concede that in full. What singles out certification is convergence: the verifier factorisation result (`v_does_not_factor_through_classical`), the clean-room floor demonstration (`nofi_demo`), and the NPA moment-matrix certificate (`chsh_lassert_no_trap_implies_quantum_realizable`) land on the same field from vocabularies that owe each other nothing. Exhibit another field with three convergent bridges of the same standing, or break one of these, and certification's distinguished seat falls back to an aesthetic choice. Whether certification is the event a model is *forced* to price remains the named open problem either way; Section 25 states the conjectured criterion (redundant record proliferation) that a successor project would have to prove or refute, with its own attack surface listed there.

Falsify the proof-of-stake reduction.

Construct a trace in the formal finality model satisfying the stated slashing schedule, with unfinalized initial state, finalized final state, and zero summed modeled charge. This contradicts `slashing_finality_floor`. A deployment with different transition or accounting semantics instead challenges its proposed interpretation.

Falsify the gas-metering reduction.

Construct a schedule satisfying both the commitment floor and no-overcharge whose charging predicate differs from the commitment predicate on some reachable step. `gas_schedule_exactness` falls, and the kernel's `exact_commitment_pricing_characterization` with it.

Falsify the attestation reduction.

Build a sound and complete attestation verifier for the μ -dependent claim that reads only the bare classical transcript, a function of the report's log field alone. `attestation_cannot_factor_through_bare_transcript` falls, and behind it the verifier-factorisation theorem.

Falsify the transparency-log reduction.

Build a sound and complete log-free verifier with the same soundness target. `log_free_verifier_impossible` falls, and the bare-setting impossibility with it.

Falsify the proof-carrying reduction.

Construct a level- k certified proof script with total μ below k , or a sound and complete zero-round bare verifier. `level_k_verification_floor` or `bare_pcc_impossible` falls, and the μ -hierarchy or the bare-setting impossibility behind them.

23 The full claim ledger

The following claims are checked in Coq under their complete formal premises. The crosswalk below records the principal domains and instance boundaries; the named source declarations give the full quantified statements.

1. *no_free_certification*: `csr_cert_addr` going $0 \rightarrow \text{nonzero}$ in one step requires cost ≥ 1 .
2. *no_free_certification_mu*: same, with μ increment ≥ 1 .
3. *no_free_certification_trace_mu*: trace-level version.
4. *no_free_certification_certified*: `vm_certified` going $\text{false} \rightarrow \text{true}$ requires cost ≥ 1 .
5. *certification_requires_positive_mu*: both channels covered.
6. *universal_nfi_any_substrate*: any abstract certification system satisfying the one-step cost rule has trace-level non-free certification.
7. *thiele_universal_nfi_cert_addr*: universal theorem instantiated for Thiele's `csr_cert_addr` channel.
8. *thiele_universal_nfi_certified*: universal theorem instantiated for Thiele's `vm_certified` channel.
9. *thiele_represents_simulating_cert_system*: any certification system with a simulation morphism into Thiele transfers certification into a Thiele certified execution.
10. *forget_kernel_is_eq_on_classical*: the classical snapshot quotient forgets exactly the non-classical fields.
11. *blindness_non_injective*: distinct Thiele states can have the same classical snapshot.
12. *certification_is_lost*: certification can be in the kernel of the classical forgetful map.
13. *shadow_strictly_lossy*: the classical shadow forgets morphism structure.
14. *mu_ledger_necessity*: no strict-classical oracle on $(\text{mem}, \text{regs}, \text{pc})$ recovers the pair $(\mu, \text{certified})$.
15. *vm_mu_not_classically_determined* / *vm_certified_not_classically_determined*: neither ledger balance nor certification status is a function of strict classical state.
16. *mu_ledger_mutual_independence*: μ and `vm_certified` are each independent of strict classical state and of each other under cost-only/cert-only projections.
17. *mu_ledger_minimality* / *P_full_complete_neither_mu_nor_cert_droppable*: the full $(\text{mem}, \text{regs}, \text{pc}, \mu, \text{certified})$ projection is complete, and dropping either μ or certification destroys the corresponding completeness.
18. *thiele_nfi_pc_indexed*: PC-indexed execution version.
19. *mu_is_initial_monotone*: μ is unique given the initial value and fixed instruction-cost schedule.
20. *vm_apply_mu*: exact ledger identity $(\text{vm_apply } s \ i). \mu = s. \mu + \text{instruction_cost}(i)$. The cost is not just δ , it is δ for ordinary instructions, $S(\delta)$ for cert-setters, $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT, etc.
21. *kernel_certified_implies_positive_mu*: from zero, reaching certified implies $\mu > 0$.
22. *feasible_strict_subset_implies_strict_predicates*: feasible-set narrowing plus a distinguishing observation yields a strictly stronger predicate.

23. *strengthening_requires_structure_addition*: certified predicate strengthening requires a structure-addition event.
24. *b4_information_reduction_derives_strict_predicates*: feasible-set reduction can generate the strict-predicate premise used by NoFI.
25. *info_priced_cert_executions_bound*: executed cert-setting events are bounded by $\Delta\mu$.
26. *observation_partition_reduction_implies_posterior_representative_reduction*: observation classes package into the posterior-representative witness for structural entitlement.
27. *info_priced_arbitrary_feasible_reduction_bound*: with a decision-tree/representative witness, feasible-set compression is bounded by $\Delta\mu$.
28. *info_priced_weighted_feasible_reduction_bound*: the $\Delta\mu$ lower bound extends to nat-weighted feasible sets.
29. *structural_entitlement_representation*: strict narrowing plus distinguishability, certified posterior admissibility, and an explicit posterior-representative reduction imply a strictly stronger predicate, a structure-addition event, and a $\Delta\mu$ lower bound.
30. *every_sound_structural_shortcut_lands_here*: every packaged `SoundStructuralShortcut` instantiates the structural-entitlement representation theorem.
31. *structural_shortcut_undecidable*: for every `Substrate` plus `WithShortcutPredicate` instance, no Coq function whose diagonal flip transformer is internally representable in the substrate decides the structural-shortcut admittance predicate. The substrate's own halting-shaped wall, by Kleene 1938 diagonal restricted to the substrate's internal expressive power.
32. *nat_recursion_theorem / nat_structural_shortcut_undecidable / nat_self_undecidable*: for each candidate d , the associated nat instance supplies its restricted fixed point and defeats d on `nat_admits d`. The parameter d remains in the theorem type despite a closed global assumption report.
33. *vm_structural_shortcut_undecidable*: the bounded VM specialization retains its representability and recurrence premises. The encoded corollary discharges the code round trip. `vm_bounded_shortcut_decide_correct` supplies an external decider; `vm_bounded_decider_flip_not_representable` proves its exclusion from a class satisfying bounded recurrence.
34. *program_to_nat / nat_to_program / nat_to_program_program_to_nat*: closed Gödel encoding of `list vm_instruction` as a nat with a proven left inverse, all 51 opcode constructors handled, no Hypothesis. Composes the existing binary encoders with a `list bool ↔ nat` bijection through `positive`.
35. *categorical_separation*: classical-identical states distinguishable by morphism probe.
36. *classical_observer_cannot_separate*: no classical function separates morphism-distinct states.
37. *shadow_proj / shadow_separation_theorem*: formal surjection, strictly lossy.
38. *D2_faithfulness*: classical programs embedded faithfully in Thiele.
39. *D3_conservativity*: classical programs leave structural layer untouched.
40. *D4_strictness*: there exist Thiele steps that escape the classical fragment.
41. *D5_thiele_strictly_extends_classical*: Thiele strictly exceeds classical semantics.

42. *degenerate_projection_theorem*: shadow projection is the classical equivalence quotient.
43. *no_classical_certification_decider*: no classical machine can decide morphism-certification.
44. *chsh_stat_violation_not_local*: CHSH-violating statistics are non-local.
45. *structural_trace_preserves_cert_addr*: traces with no cert-address setter preserve `csr_cert_addr`.
46. *partition_refinement_nonfree*: certified partition knowledge cannot be free.
47. *partition_free_but_certification_nonfree*: partition structure can be built at $\delta = 0$, but certified partition knowledge cannot be acquired for free.
48. *witness_insight_nonfree_general*: certified non-local witness insight requires $\mu \geq 1$.
49. *witness_insight_complete_taxonomy*: full three-tier taxonomy proven.
50. *separable_coupling / non_separable_coupling*: Functional (separable) and non-functional (non-separable) coupling predicates with concrete witnesses.
51. *CHSH coupling bridge*: Bell's theorem in morphism coupling language. A locally consistent strategy produces a separable coupling and has $|S| \leq 2$. $S > 2$ rules out every locally factorizable morphism coupling.
52. *graph_certify_morphism_lookup*: morphism certification cost utility, including lookup correctness.
53. *exists_covering_tree*: for any feasible-set reduction, a covering complete binary tree exists constructively.
54. *info_priced_reduction_no_tree_hypothesis*: entropy bound without requiring the tree as an explicit input.
55. *sound_shortcut_from_components*: assembly lemma for SoundStructuralShortcut from its component witnesses.
56. *mu_budget_decidable / mu_budget_verifiable / classical_mu_budget_decidable*: predicates classifying decision problems by their polynomial- μ envelope, plus the inclusion that every zero- μ classical solver is *mu_budget_decidable*. Not a class separation. See Section 24 for what these predicates do and do not say.
57. *column_contractive_iff_quantum_realizable*: slice coherence $\iff$ PSD realizability of the specified fixed completion. A biconditional with no project-local axioms (its `Print Assumptions` reports only Coq's standard classical-reals and functional-extensionality `stdlib` axioms). Forward (*zero_marginal_npa_column_contractive_implies_psd*): column-contractive $\rightarrow$ NPA PSD. Reverse (*npa_psd_implies_column_contractive*): NPA PSD $\rightarrow$ column-contractive, proved by vector specialization plus the *quadratic_nonneg_discriminant* lemma. Zero-marginal NPA realizability is the orthogonal-observables slice of the quantum elliptope ($\rho_{AA} = \rho_{BB} = 0$), not the full elliptope. Classical correlations like $(1, 0, 1, 0)$ have $|S| \leq 2$ but fail column contractivity; the gate is a slice-membership test, not a truth test, and it rejects all sixteen deterministic local strategies. The classical polytope and the slice-coherent set intersect but neither contains the other.
58. *state_column_contractive_implies_tsirelson*: slice-coherent $\Rightarrow |S|^2 \leq 8$, i.e., $|S| \leq 2\sqrt{2}$. The proof factors through quantum realizability: *state_column_contractive_implies_quantum_gram* composes with *quantum_realizable_implies_tsirelson_bound*. The row-bounds lemma *tsirelson_from_row_bounds* is the algebraic

- Cauchy–Schwarz step inside this chain. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not slice-coherent) follows immediately. The converse ($|S| \leq 2\sqrt{2} \Rightarrow$ slice-coherent) is false and is not claimed.
59. *Concrete falsifications by direct arithmetic.* The PR box ($E_{00} = E_{01} = E_{10} = 1, E_{11} = -1$) fails the first column-contractivity condition: $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Any super-quantum value $|S| > 2\sqrt{2}$ falls outside the slice-coherent set by the contrapositive of `state_column_contractive_implies_tsirelson`. The Turing-classical correlation $(1, 0, 1, 0)$ also fails condition 1. These are direct consequences by `lra` once the biconditional is destructed.
 60. *Elliptope completion (Section 16.8).* `elliptope_realizable`: correlator membership in the full elliptope as existential completion of the cross moments; `deterministic_strategy_elliptope`, `elliptope_convex`, `elliptope_finite_mixture`, and `lhv_mixture_elliptope` put every LHV correlator inside (with `classical_tightness_witness_elliptope` and `turing_point_elliptope` naming the two points the slice gate rejects); `elliptope_tsirelson` bounds the whole set by $S^2 \leq 8$ via `psd_cauchy_schwarz`; `pr_box_not_elliptope` excludes the PR box; `beyond_classical_elliptope` exhibits $S = 12/5 > 2$ inside, making `classical` $\subset$ `elliptope` strict. Print Assumptions on all of them: the standard classical-reals and functional-extensionality `stdlib` families, zero project-local axioms. `elliptope_check_full_sound` (in `ElliptopeGate.v`) adds the decidable two-branch $\mathbb{Z}$ -arithmetic gate (fraction-free Sylvester on completion buckets, rational LDL^T certificate reaching singular/boundary completions): a passing check entails membership, with computed acceptances (`gate_accepts_all_ones`, `gate_accepts_turing_point`, `gate_accepts_beyond_classical`, and on-boundary `gate_accepts_pythagorean_boundary`) and the universal refusal `elliptope_full_gate_never_accepts_pr_box`. Correlator level only; the runtime opcode is unchanged. This one-way soundness theorem does not establish completeness for the full set of quantum behaviors.
 61. *`mu_hierarchy_theorem`:* for every $k \geq 1$, there exists a level- k certified trace with cost exactly k , and every trace that ends certified having executed a `CERTIFY` at level k (charge $S(\delta) \geq k$) pays total cost $\geq k$. Unbounded strict μ -cost levels: no finite budget caps every certification. Not a problem-class separation (Section 24).
 62. *`mu_hierarchy_no_upper_bound`:* no fixed μ -budget suffices for all levels.
 63. *Structural-invariant Qed group:* the 10 opcodes `PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_ASSERT`, `MORPH_GET`, `COMPOSE`, `MORPH_TENSOR` carry Qed bisimulation proofs under the inductive invariant `coupling_wf` ($=$ `coupling_desc_bounded` $\wedge$ `coupling_pairs_in_range` $\wedge$ `coupling_pairs_fully_populated`) and `morph_table_wf`. The preservation theorems `coupling_wf_kami_step_preserved` and `morph_table_wf_kami_step_preserved` show these invariants hold inductively at hardware reset and through every `kami_step` operation, so the precondition is discharged in practice for any state reachable by machine execution.
 64. *RTL trust boundary:* the Section Variable `rtl_step_correct` states that the synthesised RTL implements the Kami step function. It is backed empirically by the `cosim` suite (38 tests, covering every opcode) and the parametrized cross-layer fuzz suite. The kernel proves `rtl_step_correct` for the `COQKAMI` model and names `bsc_kami_compilation_trusted` as the residual trust boundary covering the BSC

compiler / pretty-printer pipeline.

65. *nothing_at_stake_is_free_forgery*: a finality gadget whose finalize step carries zero stake-at-risk admits no A2 field. Nothing-at-stake is the kernel’s free forgery, stated in proof-of-stake vocabulary (Section 15).
66. *slashing_finality_floor*: under any slashing schedule (finalize risks ≥ 1), every run from unfinalized to finalized carries total stake-at-risk ≥ 1 ; `universal_nfi_any_substrate` instantiated at the slashing gadget.
67. *gas_schedule_exactness*: a gas schedule satisfies the commitment floor and no-overcharge exactly when its charging predicate is the commitment predicate with unit pricing; the kernel machine itself inhabits the class (`thiele_vm_commit_pricing_is_exact`).
68. *attestation_cannot_factor_through_bare_transcript*: a sound and complete attestation verifier of the μ -dependent claim cannot factor through the report’s bare transcript; the replay attack is the witness pair (`replay_is_a_two_preimage_witness`).
69. *level_k_verification_floor*: k certification events cost at least $k \mu$ in any covered proof system, with tightness witnessed; bounds certification events only, not gate counts, circuit size, prover time, or proof length.

Bridge-tier claims (checked in Coq, but resting on named hypotheses or documented trust boundaries):

- **Hardware correspondence**: `driven_step_wf` and `driven_trace_commutates` prove intermediate model equality under `WFDrivenPrecondition` and `WFDrivenRun`. Section 14 states the additional finite RTL contract and distinguishes the 47 synthesized instructions from four nonsynthesized checks. Allocation, overflow, and retirement obligations remain part of assessing an executable instance.
- **OCaml extraction**: `ocaml_bisimulation_closure` proves NoFI, μ -monotonicity, and totality for the Coq-side extracted observable. The external OCaml binary equivalence is a parity-test trust boundary, not a kernel theorem.
- **Structural advantage**: the kernel proves exact μ costs and the arithmetic time-tax facts for the factored-search witness; executable tests check the measured VM behavior.
- **Cost formula forcing**: `cost_necessity` + `cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.

23.1 Main-result contract crosswalk

Each row identifies the result’s domain, conclusion, load-bearing premises, and instance status. A symbol prefixed by `Kernel` is in its namesake file under `coq/kernel/`; `KamiHW` names files under `coq/kami_hw/`. The companion artifacts/`review_revision/contract_sources.json` maps every row to its source file and line. The `contract_probe.v` file in that directory loads the exact modules. Its report combines `Check`, expanded records and definitions via `Print`, and

Print Assumptions. The source declaration and its imported definitions remain the complete mathematical contract.

Prose label and formal symbol	Domain, conclusion, premises, and status
Certification floor <code>Kernel.UniversalCertificationCost.universal_nfi_any_substrate</code>	CertificationSystem states and instruction-list traces. An initially false, finally true certification indicator requires positive total cost. The record supplies the one-step floor. VM instances are proved.
Exact commitment pricing <code>Kernel.CommitmentPredicateAdequacy.exact_commitment_pricing_characterization</code>	LocalPredicatePricedSystem steps. The lower bound and no-overcharge characterize unit pricing of the designated state-dependent event. The record and reachability premises remain explicit.
Ledger uniqueness <code>Kernel.MuInitiality.mu_is_initial_monotone</code>	VM instruction-list execution and CostFunctional. Equality requires the specified initial value and instruction-increment law. This fixes the schedule, not the unique event every model must price.
Structural independence <code>Kernel.NecessityAbstract.mu_ledger_mutual_independence</code>	Specified projections of VMState. The theorem gives the displayed non-determination witnesses. It is neither statistical independence nor recovery of every field from a graph-free projection.
Verifier obstruction <code>VerifierExhaustiveness.V_does_not_factor_through_classical</code>	Verifier functions on the stated evidence domain. Soundness, completeness, and the supplied collision prevent factorization through the named bare projection. Authenticating external evidence requires its own contract.
Structural entitlement <code>Kernel.HonestNoFI_TheoremsWithoutAssumptions.structural_entitlement_representation</code>	Finite feasible lists and a supplied representative reduction. Initial/final certification, strict narrowing, distinguishability, covering and payment obligations are retained. Packaged witnesses prove this conditional statement.
Abstract diagonal <code>Kernel.StructuralUndecidability.structural_shortcut_undecidable</code>	Substrate and WithShortcutPredicate. No correct decider has a representable flip when the record supplies its fixed point and distinct yes/no witnesses. The conclusion is relative to that class.
Nat family <code>Kernel.NatSubstrateInstance.nat_self_undecidable</code>	For every candidate d , d fails on <code>nat_admits d</code> in the instance constructed from d . The family is proved. A fixed effective undecidable problem is not supplied by changing this quantifier order.
Encoded VM diagonal <code>Kernel.VMSubstrateEncoded.v_m_structural_shortcut_undecidable_encoded</code>	Bounded <code>vm_run</code> outcomes. The code round trip is proved; representability and bounded recurrence remain premises. An applicable unbounded interpreter is not an instance of this statement without a bridge.
Bounded external decider <code>Kernel.VMBoundedDecidability.v_m_bounded_shortcut_decide_correct</code>	VM instruction lists. A total Boolean computation decides equality with the bounded reference outcome. Proved by decidable equality of finite VM records.

Prose label and formal symbol	Domain, conclusion, premises, and status
Representability boundary <code>Kernel.VMBoundedDecidability.vm_bounded_decider_flip_not_representable</code>	Any transformer class satisfying bounded recurrence excludes the correct external decider's flip. Proved; this does not construct an unbounded interpreter.
Input-field step law <code>Kernel.VMEncodedInputAccess.vm_apply_logic_acc_commutes</code>	Every VM opcode commutes with replacement of the logic accumulator. Proved for all states and replacement values.
Input-field run law <code>Kernel.VMEncodedInputAccess.run_vm_logic_acc_commutes</code>	Every finite-fuel run preserves that replacement relation. Proved for every program and fuel.
Encoded interpreter obstruction <code>Kernel.VMEncodedInputAccess.no_logic_acc_encoded_interpreter</code>	No fixed interpreter using the current encoding reproduces all unbounded final-certification observations through an output observation invariant under replacement of the retained input accumulator. Other encodings and architectures are outside this conclusion.
Unbounded halting <code>Kernel.VMUnboundedExec.vm_halts_at_deterministic</code>	Existential fuel plus a halted state. Halting outcomes are unique. The relation and saturation foundation are proved. The separate B3 interpreter and B4 Rice results below use an explicit unbounded guest fragment.
Self-interpreter (B3) <code>Kernel.VMSelfRun.self_interpreter_correct</code>	One fixed 122-instruction host interprets twelve arithmetic/control instructions over four unbounded guest registers. Executable data encoding, positive step simulation and both directions of raw result correctness are checked. Separate contracts cover malformed words and live divergence. Guest structural fields remain ambient and unchanged.
Rice reduction (B4) <code>Kernel.VMSelfRiceUndec.self_rice</code>	Over the B3 guest model, extensional predicates separating the divergent program from a well-formed program are undecidable. Guest-program deciders and the named halting-on-zero/returning-zero predicates are covered. This closes the alternative-construction contract, without an internal recursion theorem or discharge of bounded recurrence.
Reachable CSR collision <code>Kernel.StructuralAxisOrthogonality.structural_shortcut_not_function_of_classical</code>	Reachable VM states and four-field forget. No decoder recovers the CSR predicate from that observation. This state predicate differs from bounded program-outcome equality.
Shadow-local execution <code>Kernel.TuringClassicalEmbedding.D2_classical_shadow_preserved</code>	The source predicate restricts the instruction fragment and its six-field observation. Graph preservation alone is insufficient when hidden graph or CSR data can be read into visible fields.

Prose label and formal symbol	Domain, conclusion, premises, and status
Elliptope gate soundness <code>Kernel.ElliptopeGate.elliptope_check_full_sound</code>	Witness counters and the supplied completion/certificate data. Passing the actual gate implies the defined elliptope membership. Rejection and completeness are outside this one-way conclusion.
Charge levels <code>Kernel.MuHierarchyTheorem.level_k_certification_cost_floor</code>	Executed traces from the specified initial state containing an instruction certified at level k . The ledger meets the charge floor. This differs from k certification events and from a complexity-class separation.
One-step model correspondence <code>KamiHW.GraphReconstructionBridge.driven_step_wf</code>	Gallina KamiSnapshot and VMState through the stated abstraction. Exact equality requires WFDriivenPrecondition. Physical RTL retirement, range, and toolchain obligations are separate.
Executed-run correspondence <code>KamiHW.GraphReconstructionBridge.driven_trace_commutes</code>	PC-indexed fuelled runs. WFDriivenRun checks the instruction premise at each visited snapshot. This proves the intermediate trace equality, with no claim about unbounded finite-hardware capacity.
Local fixed-bit CHSH <code>Kernel.CHSH.KernelCHSH.local_strategy_chsh_between_neg2_2</code>	Four Boolean local response bits imply the integer bound between -2 and 2 . The population mixture argument in the text additionally uses a setting-independent shared-variable distribution. Finite empirical samples have a different domain.
Fixed-completion PSD <code>Kernel.QuantumPartitionPSD.column_contractive_iff_quantum_realizable</code>	Four real correlators and the specified fixed matrix. PSD is equivalent to the three column-contractivity inequalities. This is an algebraic biconditional, with the real-library assumptions shown by the probe.
Integer gate <code>Kernel.QuantumPartitionPSD.chsh_assert_no_trap_implies_quantum_realizable</code>	A CHSH assertion step that advances without error yields the fixed-matrix predicate. The actual integer check supplies the premise; A2 alone does not select or imply its polynomial guard.
Relational category laws <code>Kernel.CategoryLaws.relational_compose_assoc</code>	Coupling relations modulo membership equivalence. Composition is associative; diagonal identities require the source theorem's region bounds. Allocation IDs, labels, and list equality are separate runtime observations.
Unbounded charge levels <code>Kernel.MuHierarchyTheorem.mu_hierarchy_theorem</code>	Positive k has a single CERTIFY($k-1$) witness with exact cost k and the stated lower bound. This proves unbounded instruction-charge levels, with the source initialization and execution premises.
Restricted geometry <code>Kernel.DiscreteGaussBonnet.discrete_gauss_bonnet</code>	Graphs satisfying the expanded well-formed triangulation predicate, including $B=3$ chi and the incidence identities. The defined angle sum is 5π chi. This restricted identity does not derive its premises from arbitrary topology.

24 Scope of the claim: what is established, what is not

This is where I draw the line around what I have proved. The named projections lose distinctions that matter to the richer machine. A2 gives an accounting law, and the cost and initiality results have the contracts stated earlier. The questions below ask how much further that argument goes. They stay open until there is an argument that closes them.

Physical interpretation of μ . The ledger is unique for the chosen initial value and schedule. That does not turn it into entropy or energy. A physical bridge needs an information-bearing state model, jointly satisfiable premises, and a justified relation between the formal charge and dissipation. The full-ISA F1 premise pair fails that first applicability test: its universal collapse premise contradicts zero-cost jumps.

Class separation. The proved witnesses lose designated structural fields under specified projections. They do not prevent a conventional simulator from retaining those fields in an encoding. No strict separation of standard computational complexity classes follows from these observation collisions or from unbounded charge levels.

Program predicates and state observations. The state-level CSR collision and the conditional program-outcome diagonal have different domains. A theorem linking them remains an additional obligation. The nat family and bounded VM theorem retain their explicit quantifiers and representability premises.

Feasible-set cost bounds. The representation theorem assumes a supplied tree and representative reduction, including its covering and numeric payment obligations, initial uncertified state, and final certification. It does not automatically extract the tree from arbitrary deterministic execution or establish a Shannon bound for every trace.

Full quantum behavior. The fixed-matrix biconditional concerns the specified zero-marginal, zero-within-party-moment completion. Elliptope existence permits free within-party moments at the correlator level. Full behaviors with marginals and operator realizations require further identifications. Runtime certificate soundness does not supply completeness for those larger sets.

The preceding items state the limits of the individual results. Structural, conditional, and consistency results must keep their hypotheses attached. The foundational proposal is broader than certification, but no theorem here establishes an unavoidable physical metric on all observable changes.

Trace initiality and foundational interpretation. The proved universal property concerns instruction-list evaluation, as explained in Section 3.7. Ledger uniqueness assumes a schedule. Exact pricing assumes an event and the reference count defining overcharge. These theorems give precise content to the construction without establishing that its chosen category or event is physically necessary.

The broader research question. Certification is a worked witness. A family of observations may reveal other useful structural laws, but the event definitions, evidence interfaces, and cost semantics need justification in each case. The existing separation and oracle results concern information retained by specified maps; they do not establish infinitely many independent dimensions, zero mutual information, or costs for changing coordinates.

25 The pointer-observable conjecture

Why put an event on the meter? The substitution contest fixes the event and compares its chargers. I want to go after the choice itself. My candidate is record proliferation: a distinction matters across an ecosystem when separate observers keep enough evidence to recover it. That gives me a criterion to test, and something specific to be wrong about.

Here is the criterion, stated as a conjecture and nothing more.

Conjecture (the pointer-observable criterion). Among the events a step rule could meter, certification is singled out by redundant record proliferation: a certification event is the kind of event whose record gets copied, checked, and stored across independent substrates that do not otherwise share state, while rival meterable events leave no such redundant trail. The event a substrate discipline is forced to price is the one whose records proliferate.

The name is borrowed deliberately. In the decoherence literature, a pointer observable is the one the environment keeps redundant records of: many independent fragments of the world each carry enough information to reconstruct the pointer value, and that redundancy, not any intrinsic label, is what makes those observables the objective, classical-looking ones. I am conjecturing that certification stands to computation the way pointer observables stand to quantum states: not privileged by fiat, privileged by being the thing everything else keeps copies of. Whether that analogy is load-bearing or just load-shaped is exactly what the conjecture is for; I am naming it, not claiming it.

Section 15 gives the idea five concrete settings: proof-of-stake finality, gas metering, trusted-execution attestation, certificate transparency, and proof-carrying verification. These disciplines supply the motivation. I supply the formal models, including what each observer sees and which events compete for the meter. That choice is part of the argument. The proofs cannot make it for me. A public certificate may be checkable without every observer keeping a copy; a paid instruction may have no independently stored record. The criterion has to face those cases.

How to attack it, because a conjecture without a target painted on it is just a mood. Exhibit a deployed, independently evolved metering discipline that prices a non-

certification event with the same conviction these five price certification, an event whose records do *not* proliferate, and the criterion is dead. Or show the criterion fails to discriminate: that multiplication counts or head-reversals proliferate redundantly in some substrate ecosystem by the same standard I just applied, so the criterion picks out more than one event and decides nothing. Or formalize the criterion and show certification fails it. Any of the three would teach me more than another instantiation would.

`PointerObservable.v` defines the objects: an `Ecosystem` of independent observers each holding a boolean fragment of the state; `records`, one observer's fragment deciding an event; `redundantly_proliferating`, every observer recording it independently; and `unique_pointer_among`, proliferating while no named rival event does, with uniqueness deliberately relative to the rival family because absolute uniqueness is false for free (an event's complement proliferates whenever it does). The conjecture itself is stated as a schema (`pointer_criterion_holds`): over any class of ecosystems with designated metered events and rivals, every metered event is the unique pointer among its rivals.

`PointerObservableReductions.v` supplies five minimal observer models and proves `five_disciplines_are_pointers`. The choice of observable fragments, metered events, and rival families is explicit. These are five formal instances, not five independent empirical validations of the criterion.

The five proofs establish the criterion inside those five models. To use them as an argument about deployed systems, I owe the correspondence: why those fragments describe the observers, why those rivals are the relevant rivals, and why their records should govern the price. Five proofs about chosen models do not settle those choices. That is where the conjecture has to earn its reach.

Counterexamples are part of the argument. `PointerObservableCounterexamples.v` models observers inspired by deniable authentication, symmetric authentication, and object capabilities. The examples separate an abstract forgery-resistance condition from universal record proliferation. They check observer models; they do not prove cryptographic security or establish deployment facts. A rule claiming that forgery resistance alone forces every observer to retain evidence fails at this level already.

Public verifiability and actual record proliferation are different properties. Availability of a verification procedure does not imply that every observer retains a record. The abstract verifier examples supply three sufficient ways of extending evidence, with different guarantees; they do not prove that these are the only mechanisms or that only one can involve metering.

The question is whether a justified choice of observer ecosystem and rival events identifies useful metering disciplines. `V_does_not_factor_through_classical` proves that a sound and complete verifier for the distinguishing claim cannot use

only the colliding bare observation. It does not select a unique physical mechanism for supplying the missing evidence.

The conjecture asks which events deserve a cost law. The accounting theorems tell me what follows once that law is specified. I want both parts of the argument on the table, with neither borrowing a proof from the other.

26 Conclusion

Two pillars close this document, one at the instance level and one at the substrate level.

At the instance level, the sharpest thing I found is this: an integer-arithmetic check on the CHSH outcome counters forces the NPA moment matrix to be positive semidefinite, and the argument from the one to the other never once reaches for physics. No Hilbert space, no operators, no observables anywhere in the chain. That is the claim, and it is algebra you can follow line by line. Where I checked it is `chsh_lassert_no_trap_implies_quantum_realizable`, and the check is where I confirmed I hadn't smuggled physics in without noticing: `Print Assumptions` on it leans on exactly three standard real-number axioms from Coq's own library: Dedekind-reals decidability (`sig_forall_dec`, `sig_not_dec`) and functional extensionality, and nothing of mine. The substrate-level pillar below leans on even less.

The abstract diagonal excludes a correct decider when its flip transformer is representable and the instance supplies the required fixed point. The `nat` construction proves a family result: for each candidate d , the associated instance defeats d on its predicate P_d . The VM corollary retains explicit bounded recurrence and representability premises. The separate reachable-state collision proves loss of the CSR predicate under `forget`. Neither the global assumption report nor the shared terminology identifies these different claims.

These results support the investigation of a structural account of computation. The A2 theorem applies to arbitrary systems satisfying its pricing premise. The irrecoverability theorem concerns selected projections. Neither requires treating the concrete VM as the entire subject, and neither establishes that all computation must adopt this law. The choice of foundational interpretation remains part of the argument under investigation.

Each of these results carries an explicit falsification condition, and they are gathered in one place (Section 22): a zero- μ trace from uncertified to certified, a column-contractive correlator with a non-PSD NPA matrix, a cost-rule-respecting `CertificationSystem` that certifies at total cost zero, a `nat` function that decides `nat_admits` for its own substrate. Any one of them collapses the matching theorem. You can hunt for any of them with a pencil, and the mechanised ones would stop the checker dead on top of that. I hunted. I came up empty.

I do not want this to sound like I am declaring I am right. I want to push myself, and after investigating everything, maybe the field too.

I built this. The definitions are here, the proofs are here, and the machine runs. If I have claimed more than a theorem gives me, that sentence needs fixing. If a proof fails, the proof needs fixing. I want to know where. I also want the larger question to survive the inspection: what have we left out when we describe a computation only by the answer it returns? Certification gave me one place to get a grip on that question. I intend to keep going.